

Оси 2024

Created by @SwanPoi, @at231

Файловая подсистема: общие сведения

Понятие файла и файловой системы

Уровни иерархии файловой системы

Действия, предоставляемые файловой системой

Стандарт иерархии файловых систем (организация дискового адресного пространства)

Дисковое пространство, разделы

Структура VFS

Основа интерфейса VFS: 4 основные структуры

Монтирование

Связь основных структур: superblock, inode, dentry, file.

Схема: раздел жесткого диска смонтированной файловой системы EXT2

Ext2

Superblock

Структура superblock

Структура super_operations

Inode

Структура inode

Структура inode_operation

Пример 1

Пример 2

Dentry

Структура dentry

Состояния dentry

Кеш объектов dentry

Структура dentry_operations

Inode кеш

File

Структура file

Структура path

Структура file_operations

Создание виртуальной файловой системы

Пример

Slab

Смысл Slab: основные идеи

Смысл Slab 2

Пример

Два вида кешей

Просмотр slab

Кеш

Как происходит выделение

Контроль за выделением слабов

Функции, определенные на слабе

Разница между kmalloc и kmem_cache_alloc()

О флагах структуры kmem_cache

DMA

Открытые файлы

struct fs_struct

struct files_struct

Связь структур ядра

Прерывания

Прерывания в различных системах: 16, 32 и 64 бита

Обратная совместимость

Обозначения на схеме:

MSI

PCI

Написание обработчиков прерываний

Быстрые и медленные прерывания

Быстрые прерывания

Медленные прерывания

О смысле отложенных действий

Softirq

Определение

В настоящее время определено 10 типов soft irq

Работающие soft irq — `/proc/softirqs`

Обработчик отложенных прерываний

Включение в перечень softirq своего обработчика

Запуск отложенных действий

Правила выполнения softirq

NET_RX_SOFTIRQ

Тасклеты

Почему возникла необходимость в тасклетах?

Структура тасклета

Планирование тасклета на выполнение

Регистрация тасклета

Удаление тасклета

Написание собственной функции-обработчика тасклета

Очереди работ

Понятие очереди работ

Структуры для управления очередями работ

Создание очереди работ

Способы создания работ

Обработчик отложенного действия

Планирование на выполнение

Сравнение softirq, tasklets, очередей работ

Связь структур для одного сри — сри0

Специальные файлы устройств

Типы драйверов в Linux

Задача идентификации устройств в системе

Типы файловых устройств

Идентификация файлов устройств

Старший номер

Наиболее часто используемые значения старших номеров

Младший номер

dev_t

Определение (выделение) старших и младших номеров

dev_t и подводка к USB (rdev в inode)

USB

Структуры для работы с внешними устройствами общего вида

Опять о DMA

USB топология

Архитектура USB драйвера

Типы передачи данных

struct usb_driver vs struct usb_device_driver

Plug and play

Сокеты

Парные сокеты

Сходства с pipe

Отличия от pipe:

Код из лабораторной

Сокеты BSD/Беркли

Создание сокетов: системный вызов `socket()`

Структура `socket`

Сокет как файл

Адресация сокетов

Структура для IPv6

Сетевой стек

Системные вызовы

ЛР сервер только получает сообщения

ЛР сервер получает и посылает сообщения

ЛР `epoll`, сокеты `AF_INET`

Модели ввода/вывода

Блокирующий [синхронный] ввод/вывод приложения

Polling — опрос. **Неблокирующий синхронный ввод/вывод.**

Multiplexing — уплотнение.

Мультиплексирование ввода/вывода — асинхронный блокирующий

Почему `datagram`?

Альтернатива мультиплексированию

4 модель. Ввод/вывод, управляемый сигналом. Асинхронный/блокирующий. `Signal driven IO`

Асинхронный ввод/вывод

Классификация

Загружаемый модуль ядра

Инициализация модуля

Выгрузка модуля

Пример загружаемого модуля

Работа с загружаемым модулем

Лр

Приставка `current`

Задача взаимодействия загружаемых модулей

Общие точки входа ядра:

Важен порядок загрузки модулей

Виртуальная файловая система `proc`

Для чего нужна?

Что из себя представляет:

Что видели в лабе (есть в билете на экзе!)

Программа, которая выводит на экран содержимое файла `environ`

Какую информацию еще предоставляет

[Proc: структуры](#)

[Proc dir entry](#)

[Создание proc](#)

[Передача данных из ядра в юзер и обратно](#)

[Лр фортуны](#)

[Sequence файлы](#)

[Single Open](#)

[Пример: helloworld из ядра в user. Single File](#)

[Код лабораторной с итератором](#)

[Итератор](#)

[Код лабораторной с single_open](#)

[Numa](#)

[Буферизованный/небуферизованный ввод/вывод](#)

[Код VFS](#)

[Код тасклеты](#)

[Код очереди работ](#)

Файловая подсистема: общие сведения

Понятие файла и файловой системы

Файл — информация, хранящаяся во вторичной памяти или вспомогательном ЗУ с целью ее сохранения после завершения выполнения задания(?) или преодоления ограничений, связанных с объемами основного ЗУ (рабочие файлы).

В файле могут содержаться данные, программы, тексты и любая другая информация.

Файл — любая последовательная область памяти, находящаяся во вторичной памяти.

Файловая система — порядок, определяющий способ организации, хранения, именования и доступа к данным на вторичных хранителях.

Задачи файловой системы:

1. Именованье файлов

2. Обеспечение программного интерфейса для работы с файлами пользователей и приложений.
3. Отображение логической модели или логического представления файлов на физическую организацию данных на соответствующих носителях.
4. Обеспечение надежного хранения данных, доступа к ним и защиты их от несанкционированного доступа.
5. Обеспечение совместного использования файлов.

Уровни иерархии файловой системы

В соответствие с описанными задачами в файловой системе выделяются следующие уровни иерархии.

Символьный
Базовый
Логический
Физический
Подсистема ввода/вывода
Физическое устройство

1. **Символьный уровень** — уровень именования файлов строками (массивами символов). Способ, удобный для человека.

Каталоги так же именуются.

Каталоги — организация хранения файлов (с точки зрения пользователя). В разных ФС имена файлов могут иметь разные ограничения.

Каталоги (directory=folder) позволяют систематизировать.

2. **Базовый уровень** — уровень дескрипторов файлов.

В Unix/Linux дескриптором является структура inode.

Номер inode — идентификатор файла.

3. **Логический уровень** — файл имеет логическое представление: адресное пространство файла непрерывно. Когда файл открывается, указатель файла имеет нулевое смещение. Каждая операция чтения или записи сдвигает его на определенное число байт.

С точки зрения Си — системные вызовы. В Unix/Linux, Windows — одинаково.

С точки зрения операционной системы файлы бывают:

a. Байт-ориентированные

b. Блок-ориентированные

(типы внешних устройств: символьные, блочные)

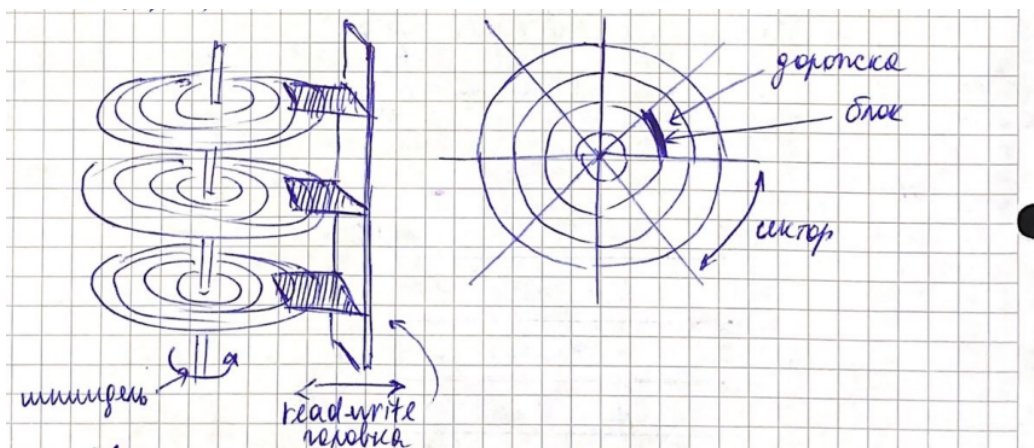
a. Символьные файлы представляют строго последовательный доступ к содержимому

b. Блок-ориентированные файлы — определена функция FSEEK. От начала файла сдвигается на какое-то количество байт

Этот уровень нужен для организации доступа к данным в файле, не думая об особенностях физического хранения файла.

4. **Физический уровень** — организация хранения файлов на внешних хранителях.

Магнитный диск (винчестер)



Разработчики Unix рассчитывали на существование больших файлов, поэтому файлы хранятся в **разброс**, а не подряд.

Дисковый inode хранит адреса блоков данных конкретного файла.

Действия, предоставляемые файловой системой

Любая файловая система будет предоставлять в распоряжение пользователя одинаковые действия:

1. Чтение, запись
2. Перемещение по папкам
3. Переименование
4. Открытие, закрытие

⇒ решение разработчиков Unix/Linux: система способна поддерживать большое количество файловых систем.

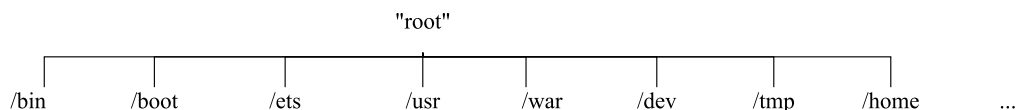
Для этого был создан интерфейс VFS(virtual file system)/vnode (в Linux без vnode).

В Linux все файл, нет различий между файлом и каталогом, так как каталог - это файл, содержащий имена других файлов.

Каталог — файл, который содержит имена других файлов.

Существуют следующие VFS: ext2, nfs, xfs, ufs, ntfs, ms-dos, fat, fat32.

В Unix/Linux всегда имеется корневая файловая система типа root fs.



Стандарт иерархии файловых систем (организация дискового адресного пространства)

Существует стандарт FHS (стандарт иерархии файловых систем), который определяет структуру каталогов и их содержимое в дистрибутивах Linux. 1993 — начало разработки FHS. 14 февраля 1994 года опубликован FHS. Ubuntu поддерживает этот стандарт.

В соответствии с этим стандартом корневая файловая система , то есть корневой каталог и его ветви должны составлять единую ФС, расположенную на одном разделе диска (partition).

Дисковое адресное пространство делится на разделы и каждая файловая система занимает на диске 1 раздел.

Одна и та же файловая система может быть подмонтирована несколько раз, то есть может иметь несколько разделов на диске.

В любой файловой системе операции над файлами будут подобны.

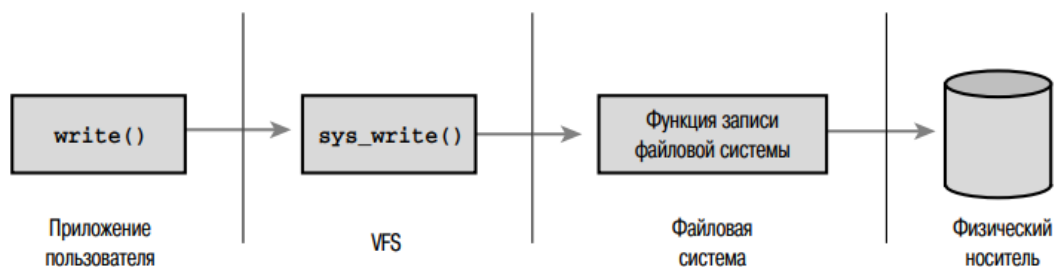


Рис. 13.2. Схема прохождения данных из пользовательского приложения, в котором вызывается функция `write()`, через вызов общей системной функции `VFS`, к специфической функции записи данных файловой системы и поступления их в конечном счете на физический носитель

В приложении вызывается функция записи в файл (системный вызов), он приводит к вызову соответствующей функции конкретной файловой системы.

Дисковое пространство, разделы

Файловые системы - важная часть любой ОС. Без файловой системы полноценной системы быть не может.

Большая часть дискового пространства — для хранения файлов (r - regular files, 1 из 7 типов), информация зависит от формата, а формат зависит от назначения. Меньшая часть — для своппинга или пейджинга (файлы, отображаемые в память. Файл на диске занимает адресное пространство, появилась идея пейджинга).

Исполняемые файлы, или регулярные файлы - обычные файлы. В Windows исполняемый файл выполняется как файл, отображаемый в собственное адресное пространство (*Рихтер*)

Как обеспечить работу виртуальной памяти, если нет памяти? read-write

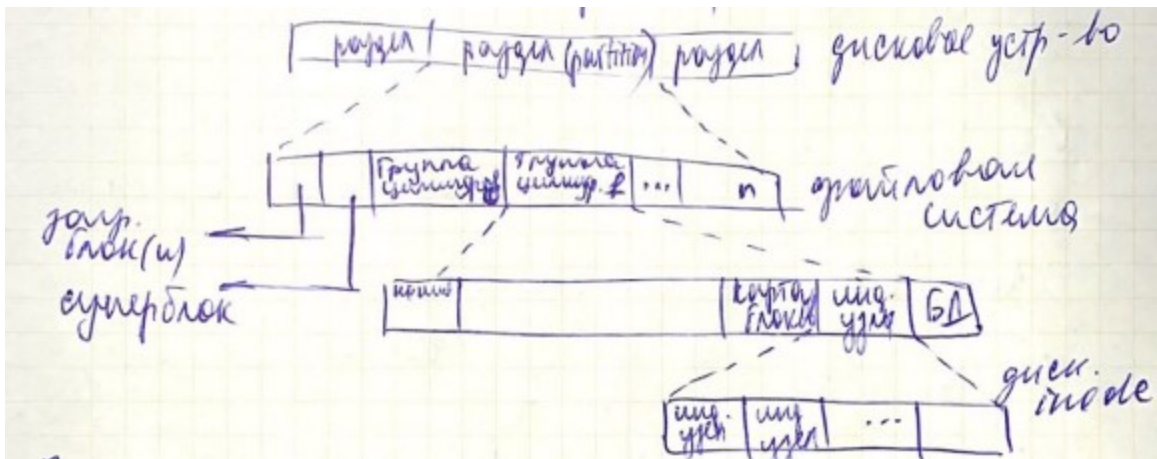
В Unix все является файлом. Обычный файл (программа до выполнения (запуска)) - во вторичной памяти.

В современных системах файлы хранятся вразброс. Это блоки, имеющие адрес (дисковый inode), можно выполнить своппинг и пейджинг (одноуровневая память)

Директории также хранятся на диске, так как это важная информация для доступа к файлу (он по fullname, а есть shortname), без каталогов имена файлов будут потеряны (⇒ Unix — иерархическая файловая система)

Каждый раздел содержит одну файловую систему.

Имя файла — символьная строка, которая содержит элементы каталога (директории)



Суперблок описывает подмонтированную файловую систему. Файлы могут храниться только на подмонтированных файловых системах. Одна и та же файловая система может быть подмонтирована несколько раз.

Карта блоков — блок занят/свободен

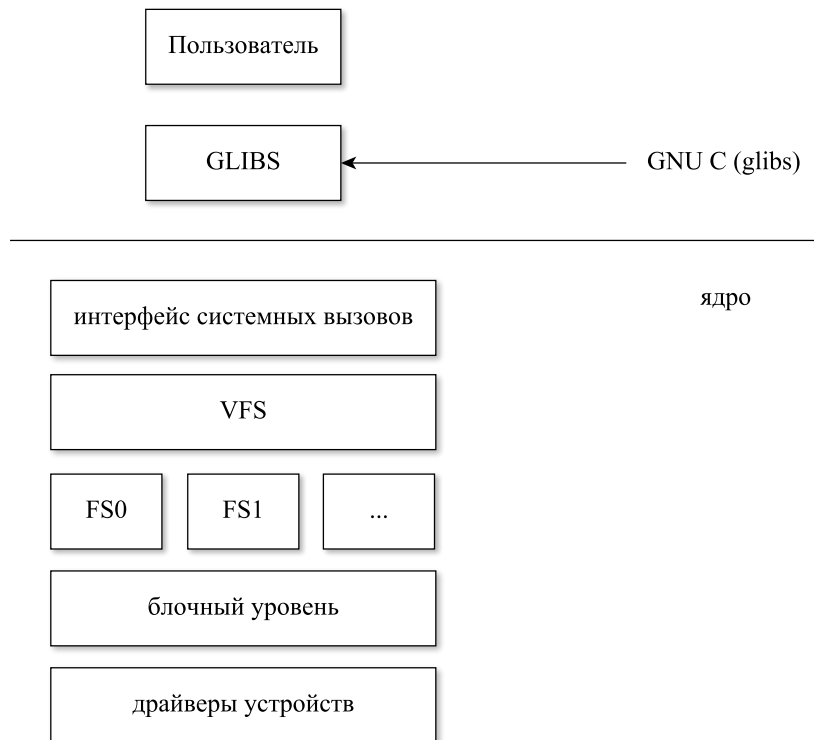
Дисковые inode содержат адреса дисковых блоков, в которых хранится файл.

Если мы находимся в директории, то эта директория является рабочей.

Unix/Linux поддерживает текущую директорию - система сама дописывает путь к имени файла (short name)

Структура VFS

Основа интерфейса VFS: 4 основные структуры



В основе интерфейса VFS лежат четыре объявленные в ядре структуры:

1. Суперблок (superblock) — представляет определенную смонтированную файловую систему.

В Unix определена операция монтирования файловой системы, следовательно

экземпляр `struct superblock` описывает смонтированную файловую систему.

В этой структуре есть поле, необходимое для определения, какая ФС подмонтирована, для ее контроля и обеспечения безопасности.

Основная задача суперблока — содержать информацию, необходимую для доступа к файлам, которые хранятся в подмонтированной файловой системе, то есть к inode.

2. Индексный узел (inode)

Экземпляр

`struct inode` описывает на диске физический файл.

inode — дескриптор файла.

Существует 2 версии

inode : inode, который хранится в ядре и inode на диске:

1. Дисковый inode — содержит адреса блоков данных конкретного файла. Обращение к inode идет через структуру superblock.

▼ **Пример: структура дискового inode для файловой системы ext**

```
struct ext2_inode {
    __le16  i_mode;      /* File mode */
    __le16  i_uid;      /* Low 16 bits of Owner Uid
*/
    __le32  i_size;      /* Size in bytes */
    __le32  i_atime;     /* Access time */
    __le32  i_ctime;     /* Creation time */
    __le32  i_mtime;     /* Modification time */
    __le32  i_dtime;     /* Deletion Time */
    __le16  i_gid;      /* Low 16 bits of Group Id
*/
    __le16  i_links_count; /* Links count */
    __le32  i_blocks;    /* Blocks count */
    __le32  i_flags;     /* File flags */
    union {
        struct {
            __le32  l_i_reserved1;
        } linux1;
        struct {
            __le32  h_i_translator;
        } hurd1;
        struct {
            __le32  m_i_reserved1;
        } masix1;
    } osd1;              /* OS dependent 1 */
    __le32  i_block[EXT2_N_BLOCKS]; /* Pointers to blocks */
    __le32  i_generation; /* File version (for NFS) */
    __le32  i_file_acl; /* File ACL */
}
```

```

__le32 i_dir_acl; /* Directory ACL */
__le32 i_faddr; /* Fragment address */
union {
    struct {
        __u8 l_i_frag; /* Fragment number
*/
        __u8 l_i_fsize; /* Fragment size */
        __u16 i_pad1;
        __le16 l_i_uid_high; /* these 2 field
s */
        __le16 l_i_gid_high; /* were reserved
2[0] */
        __u32 l_i_reserved2;
    } linux2;
    struct {
        __u8 h_i_frag; /* Fragment number
*/
        __u8 h_i_fsize; /* Fragment size */
        __le16 h_i_mode_high;
        __le16 h_i_uid_high;
        __le16 h_i_gid_high;
        __le32 h_i_author;
    } hurd2;
    struct {
        __u8 m_i_frag; /* Fragment number
*/
        __u8 m_i_fsize; /* Fragment size */
        __u16 m_pad1;
        __u32 m_i_reserved2[2];
    } masix2;
} osd2; /* OS dependent 2 */
};

```

2. Ядреный inode — содержит все те же поля. **В ядреном inode нет адресной информации.**

При работе с файлами вся информация кешируется. Как обратились к `inode` — он заносится в кеш `inode`. Обращение к части пути — заносится в кеш `dentry`. За счет этого добиваются ускорения, обеспечения быстрого доступа к информации о конкретном физическом файле.

В ядренном `inode` есть поля, которых нет в дисковом `inode`:

- a. Логический номер устройства
- b. Номер индекса, так как это поле не нужно, поскольку на диске индекс хранится в одномерном массиве и ядро идентифицирует `inode` по его смещению.
- c. Нет ссылок на другие индексы, для организации хеш-очереди в ядре.
- d.

Файл может не иметь дискового `inode`.

3. Объект элемент каталога (`dentry` — `directory entry`) — представляет определенный элемент, являющийся компонентом пути.

Экземпляр структуры описывает конкретный элемент пути.

4. Объект файл (`file`)

Экземпляр описывает открытый файл, связанный с процессом `struct file` — дескриптор открытого файла.

Открытый файл — файл, с которым работает процесс.

Имеется одна глобальная таблица открытых файлов, каждый процесс имеет свою.

Монтирование

В Linux/Unix обратиться к файлу можно только на смонтированной файловой системе.

Монтирование — системное действие, в результате которого файловая система становится доступной. Для этого используется команда `mount`.

Ключи команды

`mount` (ключи — глобальные опции для данной команды):

1. `-t` — тип файловой системы
2. `-o` — опции файловой системы
3. устройство — путь к физическому устройству или каталогу
4. каталог назначения — имя каталога, к которому будет подмонтирована конкретная файловая система

```
sudo mount -o loop -t myfs ./image ./dir
```

loop — такой драйвер “диска”, который пишет данные не на физическое устройство, а в файл (образ диска)

image — образ диска

dir — точка монтирования

Подмонтированную файловую систему можно отмонтировать с помощью команды `umount`.

Любая файловая система должна иметь корневой каталог.

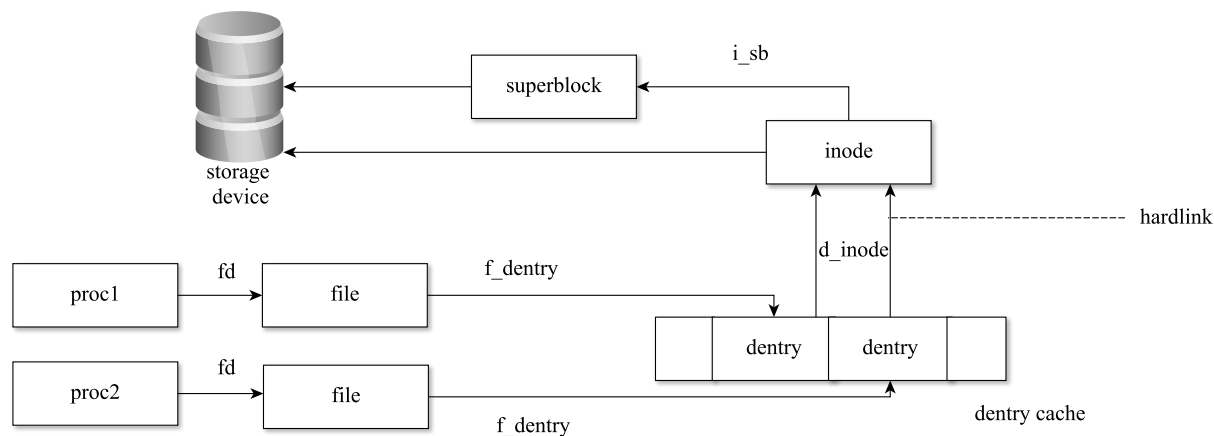
Файловая система, которая не монтируется на физическое устройство, называется виртуальной.

Связь основных структур: superblock, inode, dentry, file.

Каждая из этих структур указывает на другую структуру (непосредственно или посредством структур).

В системе имеется одна глобальная таблица открытых файлов. Каждый процесс имеет собственную таблицу открытых файлов. Причем один и тот же файл может быть открыт много раз ⇒ будет много дескрипторов (столько, сколько раз был открыт).

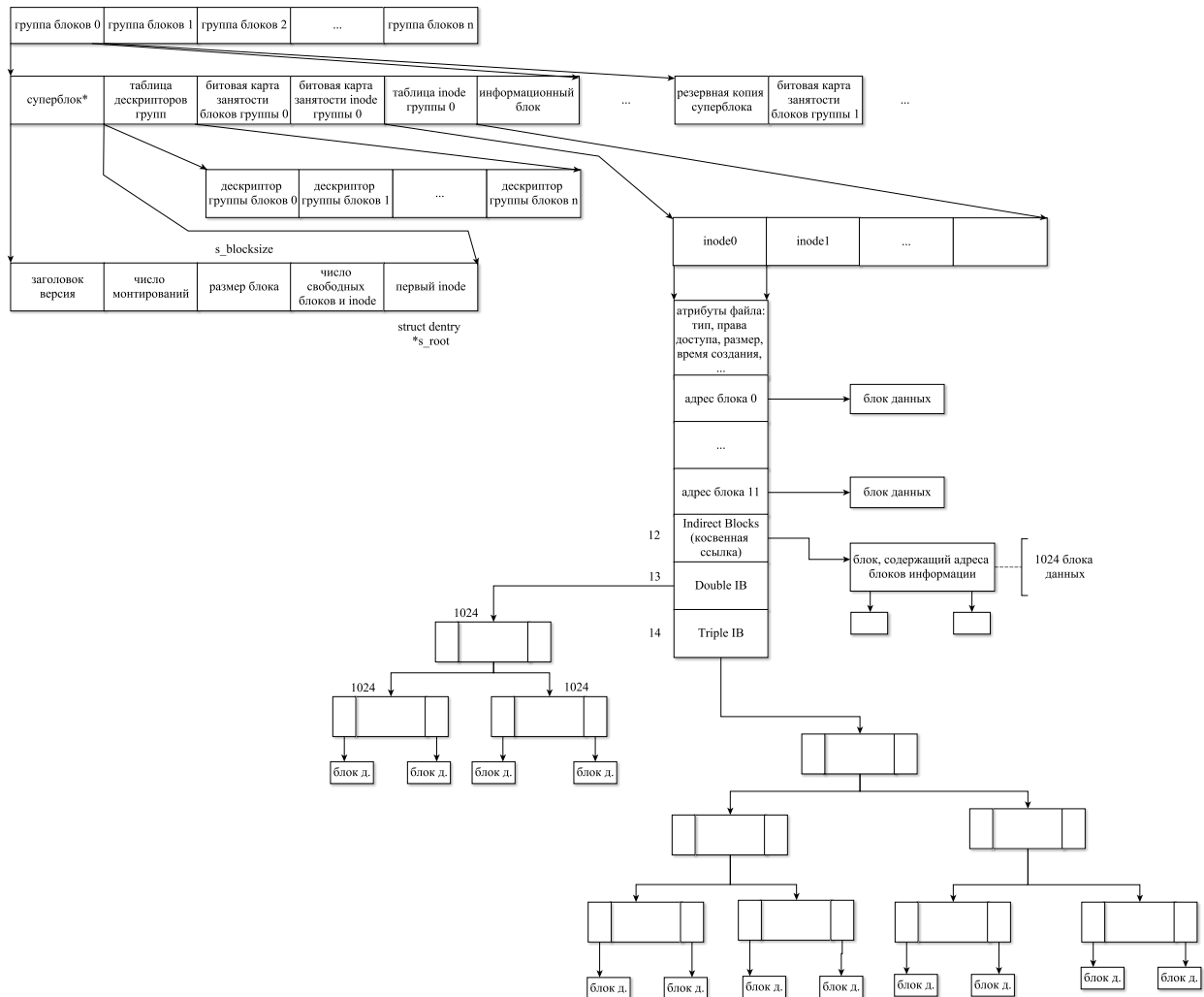
В этой схеме отправной точкой является процесс!



1. `fd` — файловый дескриптор таблицы открытых файлов конкретного процесса. Он ссылается на дескриптор открытого файла в системной таблице открытых файлов.
2. `i_sb` — указатель на superblock
3. Элемент пути (имя каталога до `/`) кешируется, то есть в оперативной памяти существует кеш `dentry` (так как они изначально хранятся на диске) ⇒ в первый раз происходит обращение к диску.
4. В структуре `dentry` есть поле `d_inode` — ссылается на `inode` конкретного файла (?)
5. `inode` имеет поле `i_sb` , которое ссылается на суперблок.

Каждое открытие файла приводит к созданию файлового дескриптора в системной таблице.

Схема: раздел жесткого диска смонтированной файловой системы EXT2



Начиная с 0 смещения этой группы находится проинициализированная структура superblock.

Инициализируем ее в загружаемом модуле ядра в приложении: заполняли s_type, s_op, s_flags, s_magic, указатель s_root.

Ниже раскрывается содержимое структуры:

1. размер блока — sblocksize
2. `struct dentry * s_root ;`

Битовая карта занятости блоков группы — дисковая информация.

i_list — битовая карта занятости inode группы

Далее идет таблица inode — `struct list_head s_inodes ;` — двусвязный список.

То есть супер блок на диске должен хранить эту информацию, чтобы обеспечить доступ к файлам.

Для дискового же inode кроме всей информации об inode есть еще информация об адресах блоков, что показано на схеме. В блоках хранится информация файла.

Прямая адресация, косвенная, двойная косвенная, тройная косвенная.

То есть уже ext2 был написан так, чтобы обеспечить доступ к сохранению очень больших файлов.

Как обеспечить доступ? Предоставить адрес блоков $\Rightarrow$ его надо хранить.

Группа блоков — последовательность блоков, имеет значение физическая организация данных (физическая структура диска)

Битовая карта — двоичное число, но каждый бит которого имеет определенное значение. Самый компактный способ передачи информации.

-
1. * — структура суперблок — экземпляр этой структуры для конкретной подмонтированной файловой системы. В ее полях содержится определенная информация: заголовок, число монтирований, размер блока, число свободных блоков и inode, первый inode.
 2. Время создания указывается для всех объектов ядра. Имеет огромное значение для системы для контроля правильности доступа к объектам.
 3. Блок, содержащий адреса блоков информации — 4 кб, страница (размер указателя 4 байта $\Rightarrow 1024 * 4 = 4\text{кб}$).
 4. Начиная с ext2 Unix поддерживает очень большие файлы.
Ext2 — Second Extended File System.
12 блоков прямой адресации. То есть 12 блоков содержат непосредственно адреса блоков данных.
 5. Чтобы обратиться к блоку данных, нужно многократно обращаться к диску за получением информации. Не делается аппаратно, выполняется драйвером $\Rightarrow$ время кратно увеличивается!
 6. Любая файловая система имеет инициализированный экземпляр структуры superblock. Этот экземпляр описывает файловую систему, которая подмонтирована.

Суперблок считывается в область данных ядра системы при монтировании файловой системы и находится в памяти ядра до размонтирования.

7. Существует файловая система, которая монтируется при инициализации системы, то есть при ее загрузке.

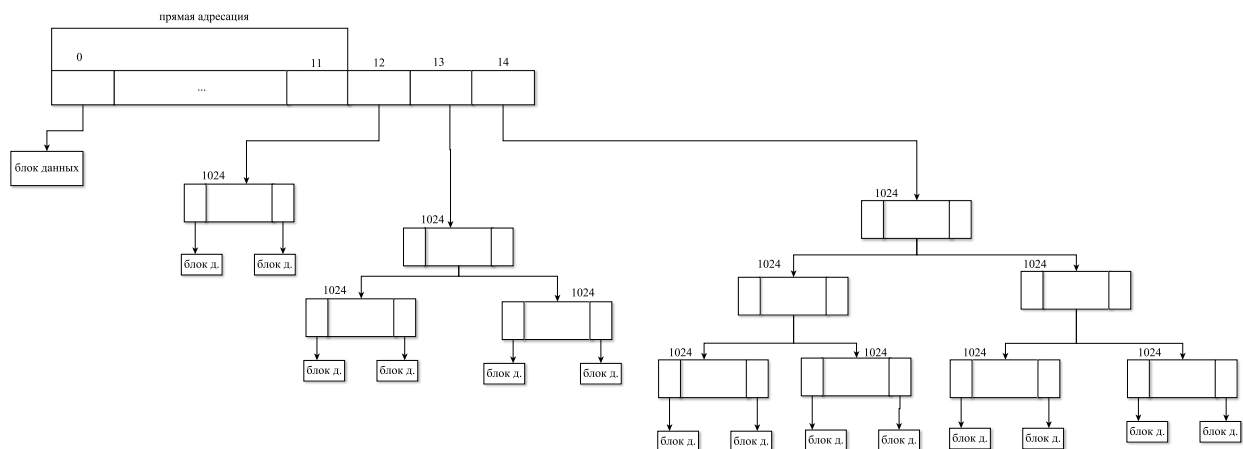
Ext2

Inode содержит 15 блоков-указателей

▼ ext2

```
#define EXT2_NDIR_BLOCKS      12
#define EXT2_IND_BLOCK        EXT2_NDIR_BLOCKS
#define EXT2_DIND_BLOCK       (EXT2_IND_BLOCK + 1)
#define EXT2_TIND_BLOCK       (EXT2_DIND_BLOCK + 1)
#define EXT2_N_BLOCKS         (EXT2_TIND_BLOCK + 1)
```

```
struct ext2_inode {
    ...
    __le32  i_block[EXT2_N_BLOCKS]; /* Pointers to blocks
    */
    ...
};
```



Superblock

Структура superblock

Суперблок считывается в адресное пространство ядра при монтировании файловой системы и находится там до ее демонтирования.

В каждой ФС один суперблок, который расположен в начале раздела (видели выше).

С суперблоком связаны две структуры из четырех: inode и dentry.

```
struct super_block {
    struct list_head    s_list; // список подмонтированных Ф
    C
    unsigned long      s_blocksize; // размер блока в байтах
    loff_t              s_maxbytes; // информация о максимальном
    размере файла
    struct file_system_type *s_type; // тип файловой системы
    (может быть в системе только 1).
    const struct super_operations *s_op; // перечисляются ф
    ункции для работы с суперблоком.

    unsigned long      s_flags; // система руководствуется з
    начениями флагов
    unsigned long      s_magic; // магическое число для иден
    тификации суперблока
    struct dentry       *s_root; // точка монтирования
    struct rw_semaphore s_umount; // любая структура ядра сод
    ержит поля для средств взаимногоисключения
    int                s_count;
    char                s_id[32];

    const struct dentry_operations *s_d_op;

    struct list_lru     s_dentry_lru;
```

```

    struct list_lru    s_inode_lru;
    struct list_head    s_inodes; // все inode
};
i

```

1. Любая структура ядра содержит поля для организации взаимoisключения
2. До поля `s_inodes` есть поле `spinlock_t` (активное ожидание). Обеспечивает доступ к inode
3. Таблица inode — связанный список структур inode.
Каждый inode описывает один физический файл на диске ⇒ важными действиями являются выделение, уничтожение и освобождение inode.

Структура `super_operations`

```

struct super_operations {
    struct inode *(*alloc_inode)(struct super_block *sb);
    void (*destroy_inode)(struct inode *);
    void (*free_inode)(struct inode *);

    void (*dirty_inode) (struct inode *, int flags);
    int (*write_inode) (struct inode *, struct writeback_control *wbc);
    int (*drop_inode) (struct inode *);

    void (*put_super) (struct super_block *);
    int (*statfs)(struct dentry *, struct kstatfs *);
};

```

Обозначение inode как dirty требует его(структуры) перезаписи на диск.

dirty inode — сначала изменяется связанный с информацией о файле блок, ядреный inode помечается как dirty, чтобы в определенный момент эту информацию отразить в дисковый inode.

1. `alloc_inode` создает и инициализирует новый объект `struct inode`, который связан с **конкретным** суперблоком.
2. `destroy_inode` — уничтожение указанный inode
3. `dirty_inode` — вызывается VFS, если в inode были внесены изменения, чтобы не потерять изменения и отразить их в дисковом inode
4. `write_inode` — записывает указанный inode на диск (в ранней версии второй параметр `int wait` — должна ли данная операция выполняться синхронно)
5. `drop_inode` — вызывается VFS, когда удаляется последняя ссылка на inode. В обычных файловых системах Unix эта функция никогда не определяется. В таких случаях подсистема VFS просто удаляет индекс.
6. `put_super` — вызывается VFS при размонтировании файловой системы для освобождения указанного суперблока
 - а. `statfs` — вызывается VFS для получения информации о файловой системе, так называемой статистики. (статистика указанной файловой системы записывается в структуру `statfs`)

В перечне функций `super_operations` нет функции создания суперблока, но безусловно такая функция есть.

```
static struct super_block* alloc_super(struct file_system_type* type,
                                      int flags,
                                      struct user_name_space
* user_ns)
{
    struct super_block* s = kzalloc(sizeof(struct super_block), GFP_USER);
    static const struct super_operations default_op;
    ...
}
```

Выделяется структура `super_block`, инициализируются ее поля и определяются операции на **конкретном** экземпляре суперблока.

Определение операций на конкретном объекте структуры суперблока — важнейшее действие!

Функция

`kzalloc` возвращает указатель на инициализированную структуру суперблока или `NULL`. Если `NULL`, то действие завершилось аварийно.

Inode

Структура inode

На диске хранятся регулярные файлы(обычные) и директории. Но в Unix глобальная идея — в Unix все файл $\Rightarrow$ структура inode описывает большое количество разных типов файлов (7 в Unix). В том числе сокеты в пространстве файловых имен, пайпы и так далее.

Название inode смысла не имеет, исторически — индексный узел.

Номер inode — идентификатор файла — метаданная (имя файла не есть идентификатор, у файла может быть много имен).

inode описывает большое количество разных типов файлов

abstract inode — ядреный inode, описывающий файл в ядре.

Обычно, когда говорят про inode, говорят об обычных файлах и dentry.

2 типа inode:

- дисковый
- ядреный

Копия индекса, хранящаяся в оперативной памяти, содержит поля, которых нет в дисковом индексе:

- логический номер устройства
- номер индекса (так как это поле не нужно в дисковом inode: на диске индексы хранятся в одномерном массиве и ядро идентифицирует inode по его смещению)
- ссылка на другие индексы (для организации хеш-очереди)

Линукс позволяет создавать виртуальные файловые системы по device. Соответственно файлы, которые могут быть созданы в такой файловой системе, не находятся на диске, а находятся в ОЗУ. То есть нет местной зависимости, что обычный файл обязательно на диске. Но для **долговременного** хранения надо задать энергонезависимое устройство, которым является диск.

```
struct inode {
    umode_t      i_mode;
    kuid_t      i_uid;
    kgid_t      i_gid;
    unsigned int  i_flags;

    const struct inode_operations *i_op;
    struct super_block *i_sb; // указатель на суперблок (см. 1)
    unsigned long i_ino; //Номер inode – то, что видели в
    union {
        const unsigned int i_nlink;
        unsigned int _i_nlink; // количество жестких
    }

    dev_t      i_rdev; // 2
    loff_t      i_size;
    struct timespec64 __i_atime; //время последнего обращения
    struct timespec64 __i_mtime; //время последнего изменения
    struct timespec64 __i_ctime; // время изменения inode, то
    spinlock_t  i_lock; // 4
    unsigned short i_bytes; // количество использованной
    u8          i_blkbits; // размер блока в байтах
    blkcnt_t     i_blocks; // размер файла в блоках

    atomic64_t    i_version; // версия
    atomic_t      i_count; // счетчик ссылок на inode
};
```

1. Любой inode имеет указатель на суперблок, так как любой файл принадлежит определенной смонтированной файловой системе. Только смонтированная файловая система позволяет получить доступ к файлу.

Любой файл можно открыть/создать только в подмонтированной файловой системе.

⇒ структура: дескриптор физического файла ссылается на суперблок.

2. real device — связанное устройство, то есть устройство, на котором находится подмонтированная файловая система, к которой относится файл.
3. atime, mtime, ctime — указываются для всех объектов ядра!
4. В любой структуре ядра имеются определенные для нее средства взаимoisключения. Структура может содержать следующие средства взаимoisключения:
 - a. семафоры
 - b. мьютексы
 - c. спинлоки — активное ожидание

То есть некоторые части кода ядра не могут блокироваться, для них может быть установлен только spinlock.

Структура inode_operation

```
struct inode_operations {
    struct dentry * (*lookup) (struct inode *, struct dentry *, int);
    int (*permission) (struct mnt_idmap *, struct inode *, int);
    int (*create) (struct mnt_idmap *, struct inode *, struct dentry *,
                  umode_t, bool);
    int (*link) (struct dentry *, struct inode *, struct dentry *);
    int (*unlink) (struct inode *, struct dentry *);
    int (*symlink) (struct mnt_idmap *, struct inode *, struct dentry *,
                   const char *);
    int (*mkdir) (struct mnt_idmap *, struct inode *, struct dentry *,
                  umode_t, bool);
};
```

```

        umode_t);
int (*rmdir) (struct inode *,struct dentry *);
int (*mknod) (struct mnt_idmap *, struct inode *,struct dentry *,
        umode_t,dev_t);
int (*rename) (struct mnt_idmap *, struct inode *, struct dentry *,
        struct inode *, struct dentry *, unsigned int);
int (*atomic_open)(struct inode *, struct dentry *,
        struct file *, unsigned open_flag,
        umode_t create_mode);
} ____cacheline_aligned;

```

1. **lookup** — выполняет поиск inode в указанном каталоге. При этом inode должен соответствовать имени файла, хранящемуся в указанном объекте элемента каталога

2. **permission** — проверяет, разрешен ли указанный режим доступа к файлу, на который ссылается объект inode.

Функция возвращает нулевое значение, если доступ разрешен и отрицательное в случае ошибки.

Для большинства файловых систем поле указателя на данную функцию содержит значение NULL. При этом используется общий метод VFS, который просто сравнивает биты поля режима доступа файлового индекса с указанной маской

3. **create** — вызывается подсистемой VFS из системных функций create() и open()

для создания нового файлового индекса с указанным в параметре mode первоначальным режимом доступа, который связан с заданным в параметре dentry элементом каталога.

То есть из тех системных вызовов, которые пользователь вызывает в своем приложении.

Когда в своем приложении мы хотим создать новый файл — fopen — указываем параметр "w" — определяем доступ — можем потерять ранее записанные данные, так как такой вызов приводит к созданию файла:

создает inode, идентификатор файла и будут выполнены действия из приложения.

Операции на inode определяют

общую практику работы с файлами.

Create создает новый inode, для физического файла будут указаны права доступа, и будет выполнена связь с каталогом dentry.

Последний параметр указывается для распределенных файловых систем.

4. `atomic_open` — данная функция вызывается опционально и открывает файл одной неделимой операцией. При этом такие действия возможны, если это symlink или какая-то ФС позволяет это делать.
5. `__cacheline_aligned` — указывает, что данная информация кешируется.
6. `link` — вызывается из системного вызова `link()` для создания `hardlink` к `inode`. Указывается старый `dentry` и новый `dentry`.
7. `mkdir` — вызывается из системной функции `mkdir()` для создания нового каталога с указанным первоначальным режимом доступа `mode`. В параметрах `inode`, так как создает новую директорию, новый `dentry`.
`umode_t` — права доступа.
8. `rmdir` — вызывается из системной функции `rmdir()` для удаления каталога, на который указывает элемент каталога `dentry` из каталога `dir`. При удалении права доступа не нужны, поэтому два параметра.
9. `rename` — вызывается подсистемой VFS для перемещения указанного элемента каталога `old_dentry` из каталога `old_dir` в каталог `new_dir` с новым именем, указанным в параметре `new_dentry`. Параметр `int` как счетчик ссылок.

Пример 1

Структура inode каталога

inode number 3470036	
.	3470036
..	3470017

inode number 3470036	
folder1	3470031
file1	3470043
file2	3470023
folder2	3470024
file3	3470065

В левом столбце указываются имена файлов и каталогов, в правом — номера inode соответствующих файлов и директорий.

Директория состоит из имен inode и соответствующих inode файлов и директорий.

Числа условны.

Пример 2

Пример демонстрирует, что элементы пути(каталоги) хранятся на диске!

Каталоги должны храниться на диске, так как иначе после перезагрузки будет потеряна эта информация и мы не сможем обеспечить доступ к файлу в иерархической системе ⇒ каталоги храним аналогично файлам на диске.

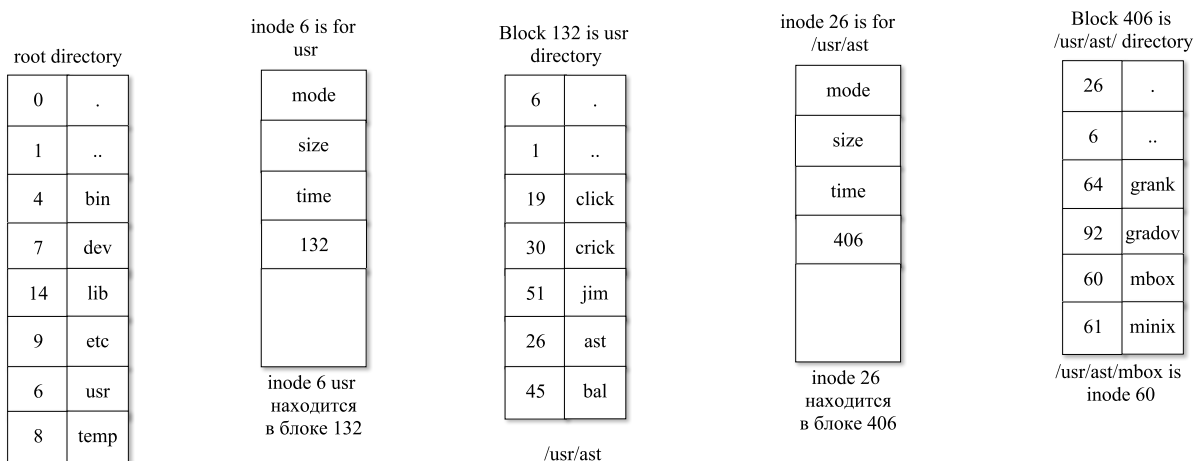
Этот пример показывает доступ к файлу.

Демонстрирует как система осуществляет доступ к конкретному файлу.

Демонстрируется необходимый объем действий, выполняемый системой, чтобы система в итоге обеспечила доступ к конкретной директории или файлу.

Доступ к /usr/ast/mbox

/ — root, то есть будет использовано 4 объекта dentry.



ФС ищет компонент пути `usr` в корневом каталоге, чтобы найти номер `inode` файла `/usr/`.

По номеру `inode` система может найти адрес блока.

Получив адрес блока система получает содержимое директории `usr`.

Нас интересует конкретная директория, у нее есть `inode`.

Далее в этой директории система ищет каталог `ast`, по номеру `inode` находит адрес следующего блока.

Получив адрес блока система получает содержимое директории `ast`.

Из этого `Inode` система ищет файл `mbox`.

Соответственно у любого файла есть соответствующий каталог, файл хранится в соответствующей директории.

В `inode` отражается несколько уровней, в том числе символичный.

Здесь показан проход системы по элементам пути. Элемент — имя до слеша.

Путь начинается с корневого каталога файловой системы.

В Linux: с корневого каталога смонтированной файловой системы.

Для каждого файла инициализируется `inode` и ее поля.

Block — блок с данными файла.

Dentry

Структура dentry

Подсистема VFS представляет каталоги так же, как и файлы. В имени пути `/bin/vi`, и элемент `bin`, и элемент `vi` — это файлы, только `bin` — это специальный файл, который является каталогом, а `vi` — это обычный файл. Объекты файловых индексов служат для представления обоих этих компонентов. Несмотря на такую полезную унификацию, подсистеме VFS также необходимо выполнять операции, специфичные для каталогов:

1. Поиск компонента пути по его имени
2. проверка того, что указанный элемент пути существует
3. переход на следующий компонент пути

Для решения этих задач в ядре существует структура `dentry`. Из `dentry` формируется дерево каталогов.

Для решения этой задачи в подсистеме VFS реализована концепция элемента каталога (directory entry, или `dentry`). Элемент каталога является особым компонентом пути.

Например, все компоненты из предыдущего примера (`/`, `bin` и `vi`) являются

элементами каталога, или объектами `dentry`. Причем первые два — это каталоги, а последний — обычный файл. Важный момент заключается в том, что все элементы пути, включая файлы, являются объектами `dentry`.

`dentry` — определенный элемент пути.

Элемент пути включает точки монтирования.

Объект `dentry` не соответствует какой-либо структуре данных на жестком диске.

Подсистема VFS создает объект `dentry` налету (в памяти) — по строковому представлению пути к файлу.

Так как объекты элементов пути (каталога) не хранятся на физическом диске ⇒ `dentry` не имеет флагов, что объект был изменен.

Объект `dentry` хранится в кеше: `dentry` кеш (`dentry_cache`).

В результате объекты `dentry` создаются и уничтожаются вызовами

`kmem_cache_alloc()` и `kmem_cache_free()` .

```

struct dentry {
    unsigned int d_flags;
    struct hlist_bl_node d_hash;
    struct dentry *d_parent; // Объект dentry родительского каталога
    struct qstr d_name; //Имя dentry
    struct inode *d_inode; //Связанный inode

    unsigned int d_count;
    spinlock_t d_lock;

    unsigned long d_time;
    void *d_fsdata;

    unsigned char d_iname[DNAME_INLINE_LEN]; //короткое имя файла

    const struct dentry_operations *d_op;
    struct super_block *d_sb;

    struct list_head d_hash;
    struct list_head d_lru; // указатель на список неиспользованных dentry
    struct list_head d_children; //указатель на список объектов dentry
    //родительский каталог
    struct list_head d_subdirs; // для директории список объектов dentry
    int d_mounted; //Является ли объект точкой монтирования
};

```

1. d_count - счетчик указателей на структуру d_inode (счетчик использований)
2. d_vfs_flex — флаг dentry кеша
3. Указатель на d_inode — связанный inode
Связанный: dentry хранятся на диске и соответственно имеют inode
4. d_lru — unused list, то есть **двусвязный** список, в list_head два поля (next и prev).

Двухсвязный список неиспользуемых и противоречивых объектов dentry, упорядоченный по времени использования (last recently used, LRU). Вставка элементов в этот список выполняется с головы, поэтому самые новые элементы находятся в его начале. Как только ядру нужно удалить элементы каталогов для освобождения памяти, эти элементы берутся из конца списка. Там находятся элементы, которые использовались очень давно и для которых меньше шансов, что они понадобятся в ближайшем будущем.

5. d_children — поле, содержащее связный список объектов dentry, которые включены в этот каталог
6. d_subdirs — поддиректории данного dentry. Если у этого dentry есть поддиректории, они будут указаны в этом list_head.
7. dentry_operations — для элементов пути определены соответствующие операции.
8. Любой файл/директория относятся к смонтированной файловой системе ⇒ указатель на super_block
9. d_mounted — является ли объект точкой монтирования
10. qstr d_name — qstr: quick string. Имя dentry. Из элексира: quickstring упрощает передачу параметров, но, что более важно, сохраняет "метаданные" о строке (т.е. длину и хэш).

Состояния dentry

Каждый объект dentry может находиться в 1 из 4 состояний:

1. free — свободный
Такой объект dentry не содержит достоверной информации и не используется в ВФС.
Соответствующая область памяти обрабатывается распределителем slab.
2. in_use
Используемый элемент каталога соответствует существующему inode (то

есть поле `d_inode` указывает на связанный объект типа `inode`), и в поле `d_count` содержится положительное значение (то есть он используется один или более раз).

Используемый элемент каталога задействован в VFS, кроме того, он также указывает на существующие данные, поэтому не может быть удален.

3. `unused`

Неиспользуемый объект `dentry` также соответствует существующему объекту `inode` (поле `d_inode` указывает на объект `inode`), но VFS в данный момент не использует этот `dentry` (поле `d_count` содержит нулевое значение).

Поскольку элемент каталога продолжает указывать на существующий объект, он сохраняется на случай, если вдруг окажется нужным.

4. `negative`

`Dentry` в противоречивом состоянии не связан с существующим `inode` (поле `d_inode` равно `NULL`), потому что или `inode` был удален, или соответствующий элемент пути никогда не существовал.

Тем не менее такие объекты элементов каталогов сохраняются, чтобы в будущем поиск элемента пути по имени проходил быстрее. В качестве примера рассмотрим фоновый серверный процесс (`daemon`), в котором постоянно выполняются попытки открытия и чтения несуществующего файла конфигурации. При этом из системной функции `open()` будет сразу же возвращаться код ошибки `ENOENT`. Ядру больше не нужно будет каждый раз создавать элементы пути, анализировать дисковые структуры и проверять, существует ли файл.

Поскольку даже такая операция поиска несуществующего файла требует достаточно

большого количества ресурсов, запоминание "отрицательных" результатов имеет определенный смысл. Хотя противоречивые элементы каталога и могут пригодиться, но при отсутствии достаточного количества оперативной памяти их можно удалить, поскольку на самом деле они нигде не используются.

`dentry` из состояния `in_use` может стать `negative`, если удалена последняя жесткая ссылка.

Кеш объектов dentry

dentry — кэшируемый объект

В структуре dentry есть поле d_hash — dentry, которые записаны в кэш (для быстрого доступа к dentry, являющихся элементами пути) + d_lru;

Зачем кэшировать?

Так как чтение и запись каталога с диска и создание соответствующего объекта dentry требует значительного времени, имеет смысл хранить объекты dentry в памяти (даже если работа с ними завершена, чтобы обратиться к тому же файлу при обращении к пути)

Полное имя = путь + имя файла

Чтобы получить доступ к файлу, система проходит все элементы пути, начиная с root

При первом обращении часть элементов пути может быть некеширована (будут созданы объекты dentry), из кэша они не удаляются.

Для максимально эффективной работы с элементами путей dentry Linux использует кеш dentry, который состоит из двух видов структур данных:

1. Список объектов dentry в состояниях in_use, которые связаны с определенным inode через поле `i_dentry` объекта inode.
Поскольку указанный файловый индекс может иметь несколько ссылок, ему может соответствовать несколько объектов dentry, а следовательно, используется связанный список

+ двухсвязный список unused и negative — d_lru.

2. Хеш-таблица и хеш-функция, которые позволяют быстро преобразовать заданный путь в связанный объект dentry.

Поиск в хеш-таблице выполняется с помощью функции `d_lookup()`. Если в кеше найден соответствующий объект, то возвращается указатель на него. В противном случае возвращается значение NULL.

Кеш `dcache` также является внешним интерфейсом к кешу файловых индексов `icache`.

Кеш `dentry` действует аналогично контроллеру для кеша `inode`.

Объекты `inode`, связанные с объектами `dentry`, не освобождаются до тех пор, пока значение их счетчика использования больше нуля.

Это, в свою очередь, позволяет объектам `dentry` удерживать связанные с ними объекты `inode` в памяти.

Иными словами, если элемент каталога находится в кеш-памяти, то соответствующий ему файловый индекс также находится в кеше.

Следовательно, если поиск в кеше некоторого имени пути прошел успешно, соответствующие файловые индексы также уже будут находиться в кеш-памяти.

Структура `dentry_operations`

Содержит перечень функций, определенных на `dentry`

```
struct dentry_operations {
    int (*d_revalidate)(struct dentry *, unsigned int);
    int (*d_hash)(const struct dentry *, struct qstr *);
    int (*d_compare)(const struct dentry *, unsigned int, const
    int (*d_init)(struct dentry *);
    void (*d_release)(struct dentry *);
    char *(*d_dname)(struct dentry *, char *, int);
};
```

1. `d_revalidate` — Определяет, является ли указанный объект элемента каталога действительным.

Подсистема VFS вызывает эту функцию, когда она пытается использовать объект `dentry` из кеша `dcache`.

Причем большинство ФС оставляют это значение как `NULL`, так как все их `dentry` в `d_cache`.

(поэтому и говорят, что `dentry` — кешируемый объект)

2. `d_hash` — Вычисляет значение хеш-ключа на основании указанного объекта `dentry`.
Вызывается VFS, когда необходимо добавить `dentry` в хеш таблицу.
При этом 1 `dentry`, который заносится в `d_hash` — это родительский каталог, в котором в дальнейшем должно быть хешировано имя.
3. `d_compare` — очевидно, что когда выполняется проход по элементам пути, должна выполняться операция сравнения
Поле `uint` нужно для указания номера `inode`.

Первый `dentry` — родительский, второй — дочерний.
`char*` — заданное имя `dentry`, с которым производится сравнение
`qstr` — имя для сравнения

4. `d_init` — используется для размещения `dentry` в `d_cache`
5. `d_release` — вызывается подсистемой VFS, когда она собирается освободить указанный объект `dentry` и используется для освобождения места в `d_cache`
6. `d_name` — вызывается, когда должен быть сгенерирован путь к `dentry`
Написана специально для некоторых псевдофайловых систем: `sockfs`, `pipefs`.
Причем функция вызывается для задержки генерации имени пути, как отложенное действие. Вместо того, чтобы генерировать путь при создании `dentry`, система делает это только тогда, когда путь необходим, то есть при обращении к файлу.

Кроме функций из `struct dentry_operations` на `dentry_cache` определены некоторые функции, которые позволяют ФС манипулировать `dentry`, то есть выполнять какие-то действия.

(указатель на функцию есть, функция описана, а в структуре ее нет).

```
struct dentry *__d_lookup(const struct dentry *parent, const st
```

Так как имеем дело с деревом каталогов и имена файлов каталогов могут повторяться в разных директориях, то для обеспечения правильного поиска

передается имя родительского каталога и тот каталог, который ищем. Если dentry найден, то связанный счетчик инкрементируется. При этом вызывающий процесс может использовать функцию, которая закрывает обработчик dentry — функция `d_put()`.

В этом случае счетчик будет декрементирован и если счетчик станет равным нулю, то dentry сбрасывается. Но dentry остается в родительском кеше.

Inode кеш

В системе должен существовать inode кеш. inode кеш имеет следующую структуру:

1. Глобальная хеш таблица `inode_hashtable`, в которой каждый inode хешируется по значению указателя на суперблок и 32-битному номеру inode.

Если у inode нет суперблока (поле

`inode->i_sb == NULL`), то вместо хеш-массива inode добавляется к двусвязному списку `anon_hash_chain`.

Примером таких анонимных inodes могут служить сокеты, созданные вызовом функции `sock_alloc()`

2. Глобальный список `inode_in_use` — inode в состоянии `in_use`.

В этом списке содержится inode с полями `i_count > 0` и `i_nlink > 0`.

Только что созданный inode добавляется как раз в этот список.

3. Глобальный список `inode_unused`, который содержит допустимые inode с `i_count = 0`.

4. `per_superblock (sb->s_dirty)`

Список для каждого суперблока (

`sb->s_dirty`), который содержит inodes с `i_count>0`, `i_nlink>0` и `i_state & I_DIRTY`.

Когда inode помечается как "грязный", он добавляется к списку `sb->s_dirty` при условии, что он (inode) хеширован.

5. Inode cache суть есть - SLAB cache, который называется `inode_cache`. Объекты inode могут создаваться и освобождаться, вставляться и

изыматься из SLAB cache
Слаб кешей в ядре много.
Slab — брусок.

Через поле `inode->i_list` с inode вставляется в список определенного типа, через поле `inode->i_hash` - в хеш-массив. Каждый inode может входить в хеш-массив и в один и только в один список типа (in_use, unused или dirty).

File

Структура file

Одна из 4 основных структур ядра (file, superblock, inode, dentry)

Эта структура, которая создана как **дескриптор открытого файла**.

То есть inode описывает физический файл, который тихо лежит на диске (чаще всего), хранит информацию и ждет, когда понадобится.

Файл может понадобиться процессу, например, для работы с данными в этом файле.

Следовательно, процесс открывает файл: или для чтения, или для записи, или для дозаписи.

Кроме того, процесс может создать новый файл. И это — открытие файла.

Открытые файлы описываются структурой file.

В системе есть одна общая для всех типов файлов таблица открытых файлов. Каждая строка таблицы —

```
struct file .
```

Глобальная таблица.

```
struct file {
    spinlock_t      f_lock; // средство взаимного исключения
    fmode_t         f_mode; // режим доступа
    atomic_long_t    f_count; // счетчик (жестких) ссылок на
    struct mutex     f_pos_lock; // блокирующее действие
    loff_t           f_pos; // логическое смещение
```

```

unsigned int      f_flags;
struct fown_struct f_owner; // владелец

struct path      f_path;
struct inode      *f_inode; /*cached value*/
const struct file_operations *f_op;

u64              f_version;
struct address_space *f_mapping;
};

```

1. `f_lock` — средство взаимного исключения. Какие-то действия в ядре могут блокироваться, а какие-то не могут ⇒ только активное ожидание.
2. `f_count` — счетчик жестких ссылок на файл.
3. `loff_t` — long offset `f_pos`;
У файловой системы много уровней: символьный, логический, базовый, физический (обращение к вторичной памяти).
`f_pos` — логическое смещение.
Любой файл как и процесс считает, что он начинается с 0 адреса.
Каждая операция сдвигает указатель файла на определенное количество байт.
Изначально `f_pos = 0`.
4. `f_flags` — флаги определяют, что будем делать с файлом:
писать(дописывать или переписывать, теряя предыдущую информацию),
только читать, + флаг `create` — создать новый файл. (`read`, `write`, `append`, `create`)
5. `f_owner` —владелец, нет в `proc_ops`

Так же как и объект элемента каталога, файловый объект не соответствует никакой

структуре, хранящейся на жестком диске. Поэтому в приведенной выше структуре нет

никакого флага, который бы указывал на то, что объект изменен (`dirty`) и требует сохранения на диск.

Структура path

В структуре `f_path` (поле `dentry`) файлового объекта хранится указатель на связанный с ним объект элемента каталога. Объект `dentry` в свою очередь содержит указатель на связанный с ним файловый индекс, в котором хранится информация о том, изменен ли файл.

```
struct path {
    struct vfsmount *mnt;
    struct dentry *dentry;
};
```

В `vfsmount` первое поле `root`.

Файл всегда принадлежит файловой системе.

Родными в Unix/Linux являются `ext`.

Какие-то файловые системы подмонтированы в процессе загрузки.

Но обратиться к файлу можно только через корневой каталог той подмонтированной файловой системы, который принадлежит файлу.

(Путь к файлу начинается с корневого каталога той файловой системы, которой он принадлежит)

Также в `vfsmount` есть указатель на суперблок.

Структура file_operations

```
struct file_operations {
    struct module *owner;
    loff_t (*llseek) (struct file *, loff_t, int);
    ssize_t (*read) (struct file *, char __user *, size_t, loff_t);
    ssize_t (*write) (struct file *, const char __user *, size_t, loff_t);

    int (*mmap) (struct file *, struct vm_area_struct *);
    int (*open) (struct inode *, struct file *);
    int (*flush) (struct file *, fl_owner_t id);
```

```
int (*release) (struct inode *, struct file *);  
};
```

Каждое поле этой структуры является функцией, определенной на файле.

1. mmap — memory mapping — Отображает указанный файл на область памяти в указанном адресном пространстве и вызывается из системной функции mmap().
2. flush — стереть — Вызывается подсистемой VFS, когда уменьшается счетчик ссылок на открытый файл.
3. Функции read/write — указатель только на file ⇒ могут работать только с открытыми файлами
4. open — **указатель на inode** — открываем физический файл. Любой открытый файл работает с физическим файлом.
5. close — библиотечная функция, release — системный вызов.

Все таблицы ядра связаны между собой указателями.

Но тут нет указателя на суперблок, inode, который описывает физический файл, будет связан с суперблоком.

Создание виртуальной файловой системы

Unix/Linux поддерживают много файловых систем ⇒ написан интерфейс VFS (В Unix vnode/VFS).

Поскольку в операционной системе Linux поддерживается множество файловых систем,

в ядре должна быть предусмотрена специальная структура данных для описания возможностей и характеристик каждой файловой системы.

В этой структуре нет указателя на superblock, так как с ним связан inode
Структура

```
file_system_type .
```

В define если первая буква F, то это флаг. Если E — error.

dev - device

```

struct file_system_type {
    const char* name; /* Название файловой системы */
    int fs_flags; /* Флаги типа файловой системы */
    #define FS_REQUIRES_DEV 1
    #define FS_USERSNS_MOUNT 8 /*can be mounted by usersns root

    struct dentry* (*mount)(struct file_system_type*, int, const
    /* Эта функция используется для прекращения доступа к супер
    void (*kill_sb)(struct super_block*);
    struct module* owner; /* Модуль, владеющий файловой системой
    struct file_system_type* next; /* Следующая файловая система
    struct hlist_head fs_supers; /* Список объектов типа супер

    /* Оставшиеся поля используются для проверки блокировок во время
    программы */
    struct lock_class_key s_lock_key;
};

```

Данная структура объявлена в ядре.

Инициализируя поля этой структуры, мы определяем собственный тип файловой системы.

После инициализации полей этой структуры мы можем передать ее функции `register_filesystem` и зарегистрировать собственную файловую систему.

Для каждого типа файловой системы существует только одна структура

`file_system_type`

, независимо от того, сколько таких файловых систем смонтировано и смонтирован

ли хотя бы один экземпляр соответствующей файловой системы

Пример

```

struct file_system_type my_fs_type =
{
    .owner = THIS_MODULE,

```

```

    .name = "my_fs",
    .mount = my_mount,
    .kill_sb = my_kill_sb,
    .fs_flags = FS_REQUIRES_DEV
};

```

Регистрируем нашу функцию mount + собственную функцию kill_sb.

Эту структуру передаем в register_filesystem. Эта функция вызывается в init.

Mount вызывается когда в командной строке вызываем команду mount.

Для того, чтобы определить файловую систему, необходимо заполнить поля структуры file_system_type: owner, name и вызвать функцию register.

Чтобы смонтировать — написать свою функцию mount, при этом основные действия выполняет функция fill_super.

Что она делает?

Создает структуры inode, dentry, root

в struct super_block есть поле s_root, которое надо инициализировать, для этого создаем inode

```

static int __init my_init(void)
{
    int ret;
    ...
    ret = register_filesystem(&my_fs_type);
    if(likely(ret==0))
        ...

    return ret;
}

```

Во всех структурах:

- superblock + sb_open
- dentry + d_open

- inode + inode_open

В `file_system_type` ничего почти не определяется, кроме функций для работы с этим типом.

Основные точки входа, что ФС заработала.

Только смонтированные файловые системы предоставляют доступ к своим файлам.

Часто функцию `exit` называют `cleanup`:

```
static void __exit my_exit(void)
{
    int ret;
    ret = unregister_file_system(&my_fs_type);
    ...
}
```

Чтобы увидеть нашу файловую систему в перечне файловых систем ядра:

```
cat /proc/filesystems
```

В ядре 4 функции для монтирования файловых систем. Все функции имеют модификатор `extern`. Возвращают значение типа `struct dentry`.

```
extern struct dentry* mount_nodev(struct file_system_type* fs
_type,
                                int flags,
                                void* data,
                                int (*fill_super)(struct sup
erblock*,
                                                void*,
                                                int));
```

`file_system_type` — предобъявленный тип, мы только можем его проинициализировать.

Эту (`fill_super`) функцию пишем мы. Ее задача проинициализировать поля структуры `struct superblock`. Чтобы проинициализировать поля надо выполнить доп действия:

```
struct superblock {
    struct dentry *s_root;
}
```

Этот root — для него мы должны создать inode. То есть чтобы заполнить это поле структуры struct superblock, мы должны для root создать inode. После написания mount можем подмонтировать файловую систему и увидеть ее root в дереве каталогов.

alloc_pid - аллокация

Paging пожирает много времени.

SLAB - способ организации повторного использования памяти (сохраняют и ищут объекты в slab)

Необходимо написать свои my_mount и my_kill_sb.

В ядре есть функции-хелперы (начинаются со слова generic_xxx_xxx)

`generic_shutdown_super` — common helper for kill_sb, делает всю необходимую работу, чтобы удалить суперблок.

Структура суперблока содержит всю необходимую информацию о подмонтированной файловой системе.

```
void generic_shutdown_super(struct super_block *sb)
{
    // хелпер, не зависящий от файловой системы
    /*
        строки по работе с объектами, определенными в str
uct super_bloc
    */
    struct inode *inode;

    spin_lock(&sb->s_inode_list_lock);
    list_for_each_entry(inode, &sb->s_inodes, i_sb_li
st) {
        ....
    }
}
```

```

        }
        spin_unlock(&sb->s_inode_list_lock);

        ...
    }

```

Кроме generic существуют simple-функции (simple_xxx_xxx)

- `simple_fill_super`

Также существуют litter

```

void kill_litter_super(struct super_block *sb)
{
    if (sb->s_root)
        d_genocide(sb->s_root);
    kill_anon_super(sb); //
}
EXPORT_SYMBOL(kill_litter_super);

```

```

void kill_anon_super(struct super_block *sb)
{
    dev_t dev = sb->s_dev;
    generic_shutdown_super(sb);
    kill_super_notify(sb);
    free_anon_bdev(dev);
}
EXPORT_SYMBOL(kill_anon_super);

```

Например, для файловой системы необходимо определить, какие функции для работы с файлами необходимо реализовать (есть “костыли” —

`simple_dir_operations`)

```

const struct file_operations simple_dir_operations = {
    .open      = dcache_dir_open,
    .release   = dcache_dir_close,

```

```

        .llseek      = dcache_dir_lseek,
        .read        = generic_read_dir,
        .iterate_shared = dcache_readdir,
        .fsync       = noop_fsync,
    };
    EXPORT_SYMBOL(simple_dir_operations);

```

Для того, чтобы работать с файлами ФС, она должна быть подмонтирована. Это задача функции `myfs_mount` - заполнить поля структуры `struct super_block` и вернуть указатель на `struct dentry`

```

static struct dentry *myfs_mount(struct file_system_type
    *type, int flags, char const *dev, void *data)
{
    struct dentry* const entry = mount_bdev(type, flags, dev,
                                            data, myfs_fill_sb);
}

```

`mount_bdev` - функция ядра, если нужно блочное устройство (все внешние устройства блочные)

Инициализацию выполняет функция `fill_sb` (можно использовать `simple_fill_super`)

```

int simple_fill_super(struct super_block *s, unsigned long magic,
    const struct tree_descr *files)
{
    struct inode *inode;
    struct dentry *dentry;
    int i;

    s->s_blocksize = PAGE_SIZE; // (1)
    s->s_blocksize_bits = PAGE_SHIFT;
    s->s_magic = magic;
    s->s_op = &simple_super_operations;
}

```

```

s->s_time_gran = 1;
// Важный шаг - создание inode
inode = new_inode(s); // передается указатель на супербло
к
if (!inode)
    return -ENOMEM;
/*
 * because the root inode is 1, the files array must not
contain an
 * entry at index 1
 */
// Нужно инициализировать поля struct inode
inode->i_ino = 1;
inode->i_mode = S_IFDIR | 0755; // права доступа
simple_inode_init_ts(inode);
// Нужно определить операции на файле
inode->i_op = &simple_dir_inode_operations;
inode->i_fop = &simple_dir_operations;
set_nlink(inode, 2);
s->s_root = d_make_root(inode);
if (!s->s_root)
    return -ENOMEM;
for (i = 0; !files->name || files->name[0]; i++, files++)
{
    if (!files->name)
        continue;

    /* warn if it tries to conflict with the root inode
*/
    if (unlikely(i == 1))
        printk(KERN_WARNING "%s: %s passed in a files arr
ay"
                "with an index of 1!\n", __func__,
                s->s_type->name);

    dentry = d_alloc_name(s->s_root, files->name);

```

```

        if (!dentry)
            return -ENOMEM;

        inode = new_inode(s); // (2)
        if (!inode) {
            dput(dentry);
            return -ENOMEM;
        }
        inode->i_mode = S_IFREG | files->mode;
        simple_inode_init_ts(inode);
        inode->i_fop = files->ops;
        inode->i_ino = i;
        d_add(dentry, inode);
    }
    return 0;
}
EXPORT_SYMBOL(simple_fill_super);

```

1. Так как `vfs` не монтируется на диск, а связано с оперативной памятью, а память выделяется страницами

Slab

Смысл Slab: основные идеи

1. Инициализация любого объекта ядра является затратным действием, поскольку связана инициализация с выделением структуры, то есть для структуры выделяется память и инициализируются поля конкретной структуры.
2. При этом в ядре нет так много структур. Ядро оперирует ограниченным набором структур.
То есть объект в памяти занимает определенный объем памяти. После его выделения с ним какой-то процесс работает и завершает работать.

Чтобы не тратить время на аллокацию, объект не удаляется, а сохраняется в виде бруска в slab кеше.

Это позволяет избежать фрагментации памяти (скрытая фрагментация возникает из-за неполного использования страницы — неэффективное выделение памяти)

Например, `struct inode` — одна из наиболее часто используемых структур ядра ⇒ для них есть slab кеш.

Когда в следующий раз какому-то процессу надо создать `inode` — этот кирпичик берется из slab кеша.

Смысл Slab 2

1. Автор — Джеф Бонвик
2. Slab кеши впервые были созданы для SunOS.
2. Slab кеш/Slab распределитель строится вокруг кеширования объекта
3. В ядре существует ограниченный набор часто используемых структур. Структуры ядра используются ⇒ создается соответствующий объект ядра и для него выделяется память.
4. Идея slab — время, затраченное на инициализацию известного объекта ядра превышает время, необходимое для его выделения и освобождения.

Проинициализированная конкретная структура не возвращает память системе, а оставляет эту память в проинициализированном состоянии в расчете на то, что она скоро будут использована.

Пример

Выделение памяти для мьютекса: вызов `mutex_init()`.

Такую инициализацию надо выполнить один раз, когда память впервые выделена для `mutex`.

Например 1 раз в сессии системы.

Последующие обращения приложения для создания `mutex` не приведут к вызову `mutex_init`, поскольку структура в ядре уже проинициализирована.

Два вида кешей

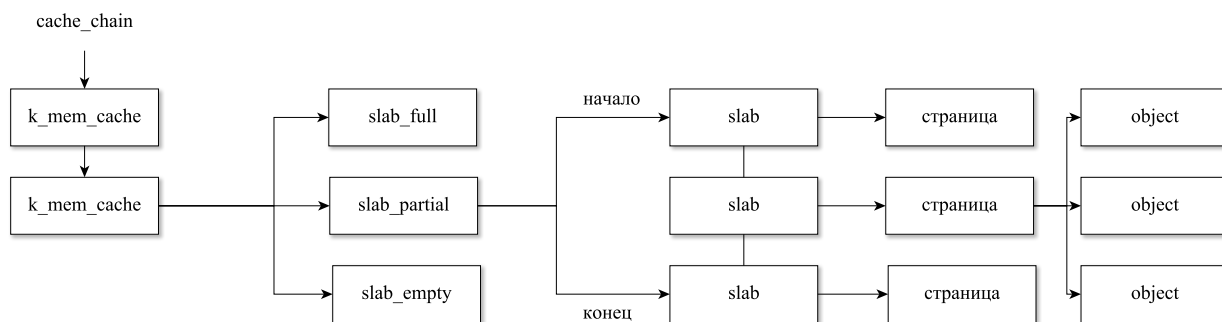
1. Кешы, которые создаются в ядре для часто используемых объектов: `struct task_struct`, `struct inode`, ... Структуры, выделенные в этом кэше, инициализируются, и когда они освобождаются, они остаются инициализированными, так что следующее выделение будет быстрее.
2. Кеш общего назначения для кирпичиков часто используемых размеров.

Просмотр slab

Слаб можно посмотреть специальной командой в ядре:

```
cat /proc/slabinfo
```

Кеш



Каждый кеш представляется в виде структуры `kmem_cache`.

Распределитель блоков состоит из переменного количества кэшей, которые связаны друг с другом в двусвязном циклическом списке, называемом `cache_chain` (цепочка кешей).

```
struct kmem_cache {
    struct kmem_cache_cpu __percpu* cpu_slab;
    slab_flags_t flags;

    struct list_head list; /*list of slab caches*/
}
```

```
struct kmem_cache_node *node[MAX_NUMNODES];
};
```

1. Слаб каким-то образом связан с конкретным ядром — per cpu.

▼ struct kmem_cache_cpu

```
struct kmem_cache_cpu {
    void **freelist;    /* Pointer to next available ob
    ject */
    unsigned long tid;  /* Globally unique transaction
    id */
    struct slab *slab;  /* The slab from which we are a
    llocating */
#ifdef CONFIG_SLUB_CPU_PARTIAL
    struct slab *partial; /* Partially allocated froz
    en slabs */
#endif
    local_lock_t lock; /* Protects the fields above */
#ifdef CONFIG_SLUB_STATS
    unsigned stat[NR_SLUB_STAT_ITEMS];
#endif
};
```

2. struct list_head list — двусвязный список kmem_cache
3. struct kmem_cache_node *node[] — массив структур kmem_cache_node

▼ интересен дефайн константы:

```
#ifdef CONFIG_NODES_SHIFT
#define NODES_SHIFT    CONFIG_NODES_SHIFT
#else
#define NODES_SHIFT    0
#endif

#define MAX_NUMNODES    (1 << NODES_SHIFT)
```

Каждый кеш делится на блоки (slab). Slab занимает одну или несколько физически смежных страниц памяти. Каждый кеш может содержать несколько slab.

В каждом слабе находится некоторое количество объектов, которые представляют собой кешируемые структуры данных.

Каждый slab может находиться в одном из трех состояний:

1. slab full — полностью распределенный

В таком слабе нет свободных объектов, поскольку все они распределены и используются.

2. slab partial — частично распределенный.

В таком слабе есть как выделенные, так и свободные объекты.

3. slab empty — пустой, незаполненный

В таком слабе нет ни одного распределенного объекта, все его объекты свободны для последующего использования.

```
struct kmem_cache_node {
    spinlock_t list_lock;
    unsigned long nr_partial;
    struct list_head partial;
#ifdef CONFIG_SLUB_DEBUG
    atomic_long_t nr_slabs;
    atomic_long_t total_objects;
    struct list_head full;
#endif
};
```

В этих списках находятся все блоки, связанные с данным кешем. Каждый блок описывается с помощью структуры типа slab, которая является его дескриптором

Как происходит выделение

Как только из некоторой части ядра поступает запрос на новый объект, он выделяется из частично распределенного блока, если таковой имеется.

В противном случае объект выделяется из пустого блока.

Если больше не осталось пустых слабов, они будут созданы по мере необходимости.

Такая стратегия выделения памяти существенно снижает фрагментацию памяти.

Контроль за выделением слабов

1. Слабы создаются в адресном пространстве ядра системы, которое является дефицитным.
2. Ядро всегда минимизирует выделение памяти, в частности и под слаб. Нет бесконтрольного создания слабов в АП системы. То есть система минимизирует создание слабов.
3. Только в том случае, когда слаб заполнен, то есть получил значение slab full, новые экземпляры структур будут записываться в новый slab partial.
4. Когда в структурах исчезает необходимость, то они перемещаются в slab empty, и этот slab может быть ликвидирован. То есть пустой слаб является кандидатом на удаление, чтобы не занимать память.

Функции, определенные на слабе

На slab определены следующие функции.

1. Создание нового кеша

```
struct kmem_cache* kmem_cache_create(const char* name,
                                     unsigned int size,
                                     unsigned int align,
                                     slab_flags_t flags,
                                     void (*ctor)(void*))
```

1. name — строка, содержащая имя кеша. Используется VFS proc следующим образом: /proc/slabinfo.
То есть позволяет идентифицировать slab кеш
2. size — определяет размер объектов, которые могут находиться в кеше

3. `align` — определяет выравнивание
4. `flags` — определяет разрешенные опции

В случае успешного выполнения функция `kmem_cache_create()` возвращает указатель на созданный кеш. В противном случае возвращается `NULL`. Эта функция не может вызываться в контексте прерывания, так как она может переводить процесс в состояние ожидания.

2. Для удаления кеша используется функция: она аннулирует кеш, указанный в качестве параметра. Обычно эта функция вызывается при выгрузке модуля из памяти, в котором был создан свой кеш.

```
int kmem_cache_destroy(kmem_cache_t *cache);
```

В случае успешного выполнения функция возвращает нуль, в других случаях возвращается ненулевое значение.

3. После создания кеша из него можно получить объект, вызвав приведенную ниже функцию. Эта функция возвращает указатель на объект, выделенный из кеша, на который указывает параметр `cachep`.

```
void *kmem_cache_alloc(kmem_cache_t *cache, int flags);
```

4. Чтобы освободить ненужный объект и вернуть его в блок, из которого он был выделен, используется следующая функция. Она помечает объект, на который указывает параметр `objp`, в кеше `cachep` как свободный.

```
void kmem_cache_free(kmem_cache_t *cache, const void *obj);
```

Разница между `kmalloc` и `kmem_cache_alloc()`

`kmalloc` выделяет непрерывный регион физической памяти и затем этот регион освобождается функцией `kfree`. Эти действия затратны. Особенно, если постоянно вызывается `kmalloc` — нужно постоянно выделять регионы для объектов.

`kmem_cache_alloc` — получение адресного пространства для объекта (память выделяется страницами)

`kmem_cache_create` — создает слаб и может выделять в слабе разного размера подрегионы для объекта.

В формальных параметрах `kmem_cache_create(size)` `size` — размер.... Однако все равно будет выделена минимум страница.

`kmem_cache_alloc` — получение в этом регионе адресного пространства подрегиона для объекта.

Если знаем, что часто создаем и освобождаем, то создаем slab, чтобы при освобождении объекты попадали в slab, а не в пул свободной памяти, с дальнейшим переиспользованием

Освобожденные в слабе объекты не возвращают память в пулл, и при следующем вызове будут возвращены нов.....

Если система поддерживает слаб — с ним надо работать.

О флагах структуры `kmem_cache`

Определено большое количество флагов, однако мы рассмотрим один:

```
#define SLAB_CACHE_DMA
```

Этот флаг указывает, что требуется использовать для slab DMA-совместимую память.

DMA

Direct memory access (DMA) — прямой доступ к памяти — механизм передачи данных от устройства в память, из сети в память.

Передачи напрямую в память минуя регистры процессора освобождает процессор.

Существует специальный контроллер DMA.

Каждая система подсистемы может иметь собственный контроллер DMA (поддержку).

Открытые файлы

С каждым процессом в системе связаны список открытых им файлов, корневая файловая система, текущий рабочий каталог, точки монтирования и т.д.

Перечисленные ниже 2 структуры данных связывают вместе подсистему VFS и процессы, которые выполняются в системе. Это структуры `files_struct`, `fs_struct`.

Структура `files_struct` определена в файле `<linux/fdtable.h>`. Адрес этой структуры хранится в поле `files` дескриптора процесса:

```
struct task_struct
{
    ...
    struct fs_struct *fs;
    struct files_struct *files;
    ...
}
```

struct fs_struct

Информация о файловой системе, к которой относится процесс

Любое приложение до запуска является файлом конкретной файловой системы.

Это из `elixir`:

```
struct fs_struct {
    int users;
    spinlock_t lock;
    seqcount_spinlock_t seq;
    int umask; /* Стандартные права доступа к файлу */
    int in_exec;
    struct path root, pwd;
} __randomize_layout;
```

А это с лекции:

```
struct fs_struct
{
    atomic_t count;
    ...
    struct dentry *root; /* Корневой каталог */
    struct dentry pwd; /* Текущий рабочий каталог */
    ...
    struct vfsmount *rootmnt;
    ...
}
```

В этой структуре содержится путь к текущему рабочему каталогу (он выводится в ответ на команду `pwd`) и путь к корневому каталогу данного процесса.

struct files_struct

В данной структуре хранится вся информация об открытых процессом файлах и файловых дескрипторах.

В этой структуре есть массив, в котором регистрируются файлы, открытые процессом

```
/*
 * Open file table structure
 */
struct files_struct {
    /*
     * read mostly part
     */
    atomic_t count; // счетчик ссылок
    bool resize_in_progress;
    wait_queue_head_t resize_wait;

    struct fdtable __rcu *fdt;
```

```

    struct fdtable fdtab;
/*
 * written part on a separate cache line in SMP
 */
    spinlock_t file_lock ____cacheline_aligned_in_smp;
    unsigned int next_fd; // число открытых процессом файлов
    unsigned long close_on_exec_init[1];
    unsigned long open_fds_init[1];
    unsigned long full_fds_bits_init[1];
    struct file __rcu * fd_array[NR_OPEN_DEFAULT];
};

```

1. `atomic_t count` — счетчик ссылок.
Отсюда следует, что ссылки на один и тот же экземпляр структур `files_struct` или `fs_struct` могут содержаться в нескольких дескрипторах процессов. В поле `count` каждой структуры находится счетчик использования, который предотвращает уничтожение структуры данных, если она используется хотя бы в одном процессе.
2. `unsigned int next_fd` — число открытых процессом файлов (кэш следующего доступного файлового дескриптора)
3. `fd_array[NR_OPEN_DEFAULT]` — массив файловых объектов: в этой таблице может быть 256 открытых файлов. Если открывается больше файлов, то создается еще одна таблица

В системе есть таблица открытых файлов `struct file` (Она одна, в ней все открытые файлы. Даже те, что не имеют отображения на диске: pipe, socket)

```

struct path {
    struct vfsmount *mnt;
    struct dentry *dentry;
};

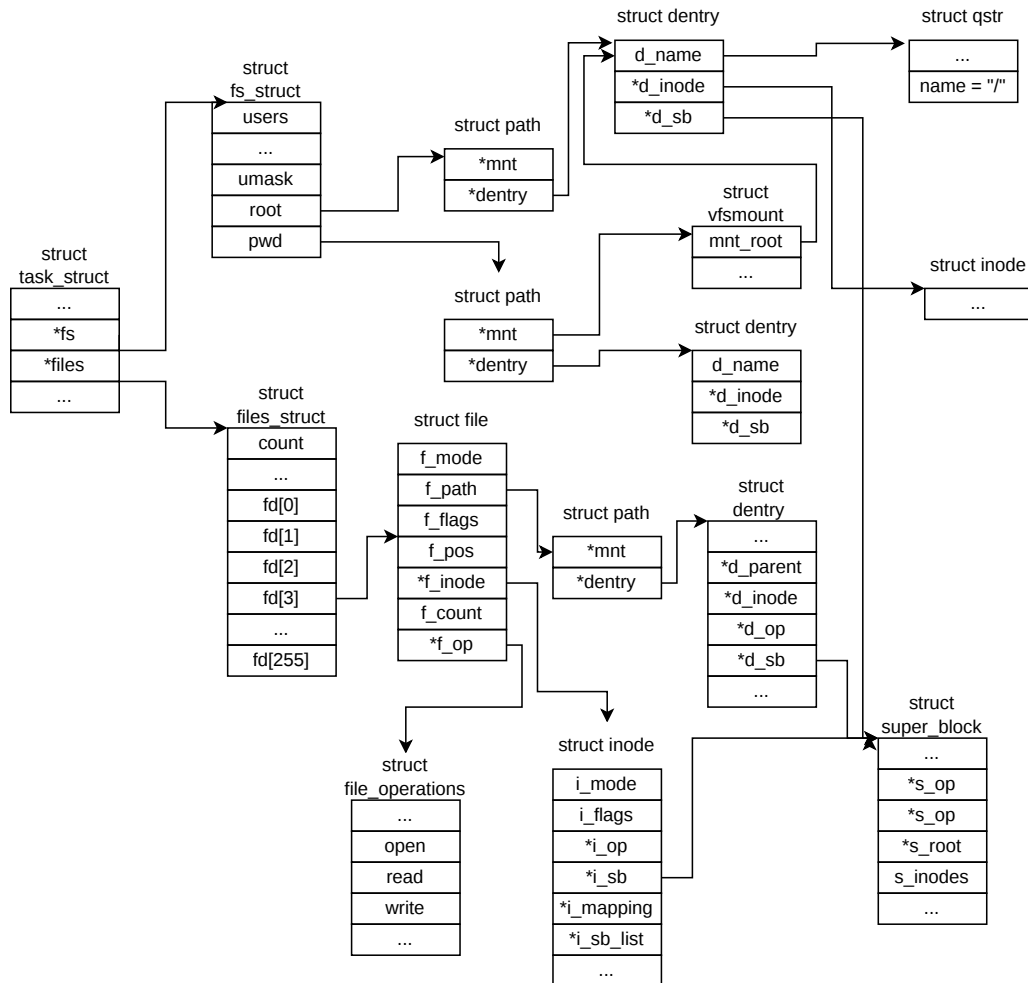
```

`fopen()` - пользовательская функция буферизованного ввода-вывода

`FILE *f` — такой структуры нет, это define над `struct _IO_FILE`

Связь структур ядра

Из лабораторной: чуть измененная, чем из лекции



Прерывания

В определенных частях код ядра, связанный с обработкой прерываний, модифицировался.

Наибольшие изменения после разработки SMP-архитектуры (до нее флаги начинались с SA_xxx, после: IRQF_xxx)

Аппаратное прерывание — единственный асинхронный параллелизм в системе

При этом обработка аппаратных прерываний является принципиально **аппаратно-зависимым (архитектурно-зависимым) действием**

Прерывания в различных системах: 16, 32 и 64 бита

1. В MS-DOS: таблица векторов прерываний, начиная с 0 адреса
2. В защищенном режиме:
 - таблица дескрипторов прерываний IDT (Interrupt Descr. Table)
 - регистр IDTR (находится начальный адрес IDT)
 - указатель на обработчик 8 байт
 - селектор лежит в дескрипторе = сегмент + смещение (точка входа в обработчик прерывания)
3. В режиме long:
 - аналогично 32-разрядному режиму,
 - Но есть 16 специальных прерываний, доступ к которым осуществляется через специальный список

Чтобы избавиться от аппаратной зависимости Linux стремятся ввести логическую схему обработки аппаратных прерываний.

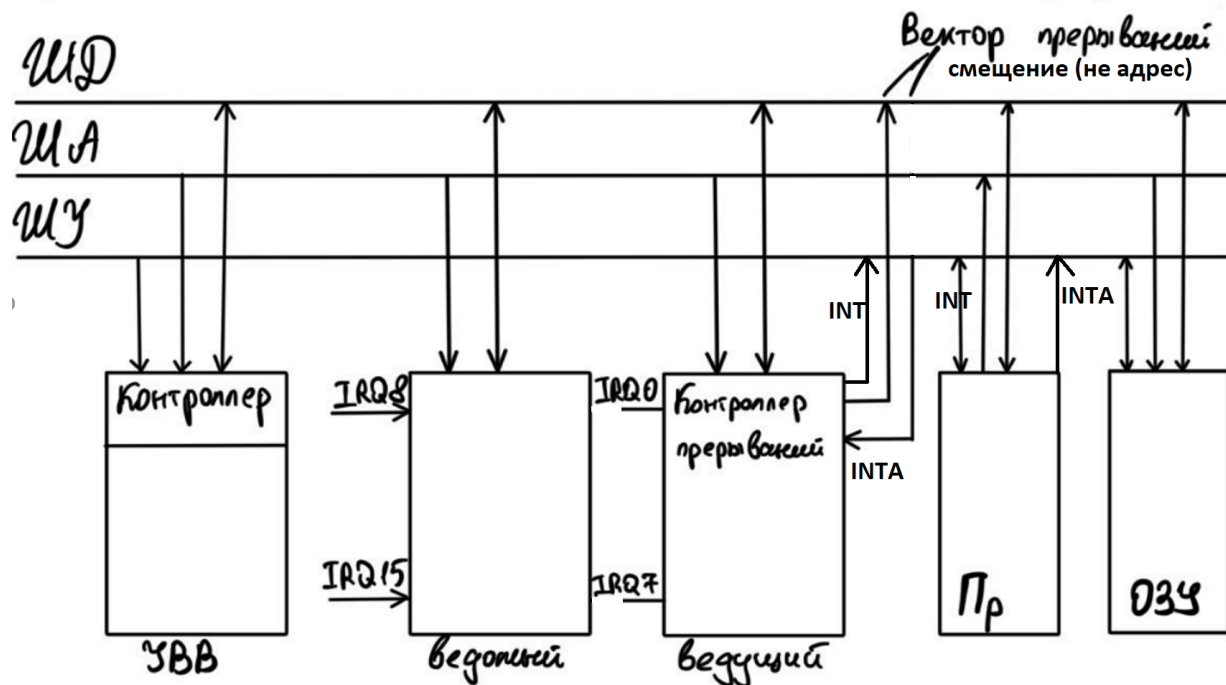
Начиная с 3 поколения ЭВМ реализована идея распараллеливания функций: процессор отключен от управления внешними устройства.

Соответственно, процессор надо информировать о завершении ввода/вывода, для этого и нужны аппаратные прерывания.

Обратная совместимость

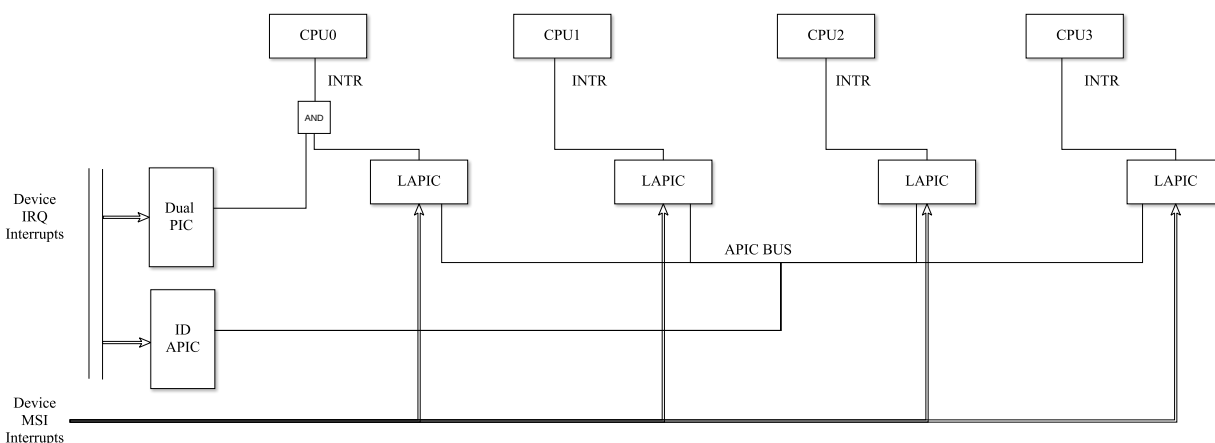
Изначально была ISA шина — индустриальный стандарт. С ней связана работа программного контроллера прерываний. К современным системам можно подключить, используя слоты шины ISA, старые устройства. Затем в трех шинной архитектуре был программный контроллер PIC — peripheral interrupt controller, который работал быстрее ISA.

Простейшая концептуальная схема трекишной орк-ры:



Затем IRQ и MSI.

Таким образом в общем подходе, в котором формируется обратная совместимость, PIC, IRQ и MSI объединяются в одну логическую схему.



Обозначения на схеме:

1. PIC - peripheral interrupts controller
2. APIC — Advanced PIC — контроллер для 32-разрядных систем

3. LAPIC — Local APIC

На схеме присутствует старый программный контроллер для обеспечения совместимости с 16-разрядными системами (dual pic — ведущий, ведомый).

Устройства, поддерживающие MSI, должны поддерживать (pin) IRC с целью обеспечения обратной совместимости

MSI

MSI — message signaled interrupts — прерывание, сигнализируемое сообщением.

Устройства сигнализируют о прерывании, отправляя/записывая сообщение по определенному адресу в памяти.

Так как MSI генерируется в форме сообщения, записываемого в память, необходимо, чтобы поддерживались все условия транзакций, такие как:

1. retry
2. Master abort
3. Target abort
4. Нормальное завершение

Для записи такого сообщения устройство может иметь до 32 закрепленных за ним областей памяти.

Используется механика захвата шины — bus mastering. Механизм чаще всего описывается по отношению к DMA

Аналогично маскированию прерываний в контроллере прерываний, маскированные прерывания, которые пришли из PIC, в MSI могут быть замаскированы. Все это указано в драйвере устройства.

PCI

В шинах PCI X, PCI express вообще не используются физические линии прерываний, прерывания доставляют как MSI

(PCI – для нее в UNIX были определены структуры и они существуют в ядре до сих пор, хотя в чистом виде PCI не исп., PCI EXPRESS)

Системы, предоставляемые пользователю – абстракции, приведение к старому подходу.

Сделано для обратной совместимости.

Написание обработчиков прерываний

Необходимо научиться самим писать обработчики аппаратных прерываний (вопрос приоритетов, решения)

Ядро предоставляет возможность написания собственных функций, которые будут обработчиками прерываний, и их регистрации в системе

Для этого в ядре определена функция, позволяющая **регистрировать** обработчик прерывания

`request_irq` - обертка

```
extern int __must_check
request_threaded_irq(unsigned int irq, irq_handler_t handler,
                    irq_handler_t thread_fn,
                    unsigned long flags, const char *name, void *dev);

/**
 * request_irq - Add a handler for an interrupt line
 * @irq:      The interrupt line to allocate
 * @handler:   Function to be called when the IRQ occurs.
 *            Primary handler for threaded interrupts
 *            If NULL, the default primary handler is installed
 * @flags:    Handling flags
 * @name:     Name of the device generating this interrupt
 * @dev:      A cookie passed to the handler function
 *
 * This call allocates an interrupt and establishes a handle
 * r; see
 * the documentation for request_threaded_irq() for details.
 */
```

```
static inline int __must_check
request_irq(unsigned int irq, irq_handler_t handler, unsigned
long flags,
            const char *name, void *dev)
{
    return request_threaded_irq(irq, handler, NULL, flags, na
me, dev);
}
```

Есть более точечные версии этой функции (persrcu, немаскируемые)

Наиболее часто используемая функция, которая используется для регистрации - `request_threaded_irq`

`unsigned int irq` - номер линии прерывания, понятие сохранено несмотря на существование MSI

- 0, 1, 12 — PCI связан с этими линиями, это можно увидеть в `/proc/interrupts`

Удаление обработчика:

```
extern const void *free_irq(unsigned int irq, void *dev);
```

- `dev` - device, описывается инициализированной структурой. `void *` позволяет передавать указатель на любую структуру, описывающую device

Флаги:

- `#define IRQF_TRIGGER_HIGH`
- `#define IRQF_SHARED` — разделение линии IRQ с другими обработчиками прерывания (одна и та же IRQ разделяется несколькими разными обработчиками прерывания)
- `#define IRQF_TIMER` — показывает, что прерывание является прерыванием от таймера
- Существовал флаг `SA_INTERRUPT` - быстрое прерывание. Сейчас самое быстрое - от системного таймера

Быстрые и медленные прерывания

Быстрые прерывания

Единственным быстрым прерыванием в системе является **прерывание от системного таймера**

Быстрые прерывания выполняются от начала до конца ⇒ никакие другие действия выполняться не могут ⇒ вытеснение такого потока невозможно ⇒ никакая работа выполняться не может, если обработка длительна (признак (следствие) — снижение отзывчивости системы)

Поэтому в обработчике прерывания системного таймера все действия - отложенные, то есть производится постановка в очередь на выполнение

Основная функция обработчика прерывания от системного таймера:

1. декремент кванта
2. инициализация переключения (по кванту), то есть инициализация работы диспетчера.

Медленные прерывания

У прерываний обычно приоритет 49 и обычно он самый приоритетный. Поэтому обработчики не должны быть долгими

Остальные определяются как **медленные прерывания**. Они делятся на верхнюю и нижнюю половины (TOP, BOTTOM)

- TOP — аппаратное прерывание
- BOTTOM — может быть реализовано в виде 3 типов
 - soft irq
 - tasklets
 - work queue

Фактически задачей обработчика прерывания является:

- выполнение минимального количества (объема) действий
- планирование отложенных действий

Тем не менее обработчик прерывания все же должен выполнить некоторые действия.

Например, почти всегда обработчик прерывания должен отправить устройству уведомление о том, что прерывание получено. Ему также может понадобиться скопировать некоторые данные из внутренней памяти аппаратного устройства в память компьютера.

Так как эта работа критична ко времени выполнения, имеет смысл выполнить ее в самом обработчике прерывания.

То есть ВОТТОМ — отложенные действия

Пример:

Клавиатура → кнопки → нажатие клавиши/отпускание клавиши

Код нажатой клавиши попадает в буфер (INT 9H)

1. Получение кода из буфера клавиатуры
2. Помещение кода в область ядра
3. Планирование действия, которое закончит обработку этого прерывания

После отпускания клавиши → контроллер → прерывание IRQ1 → приб-ся вектор прерываний → выделяется ножка процессора → процессор после выполнения каждой команды проверяет наличие прерывания на выделенной ножке → обработка

О смысле отложенных действий

Каждое внешнее устройство как правило имеет обработчик прерываний. Любое аппаратное прерывание выполняется на самых высоких уровнях приоритетов.

При этом в x86 на процессоре, где обрабатывался обработчик аппаратного прерывания — все прерывания запрещаются, а на всех других процессорах запрещаются прерывания по данной линии IRQ. То есть действие влияет на отзывчивость системы — возможность обрабатывать другие действия. Следовательно, для этого придумали отложенные действия.

Softirq

Определение

Определены статически. Для этого в ядре определен массив из 32 экземпляров struct softirq_action (<linux/interrupts.h>)

```
struct softirq_action
{
    /*функция, которая должна выполняться*/
    void      (*action)(struct softirq_action *);
};
```

В файле <kernel/softirq.c> определен этот массив

```
static struct softirq_action softirq_vec[NR_SOFTIRQS]
```

NR_SOFTIRQS = 32, то есть в ядре имеется возможность статически зарегистрировать 32 soft irq

В настоящее время определено 10 типов soft irq

Изначально когда появились сети, появилась сетевая подсистема. Сетевая карта — device, на ней определено аппаратное прерывание

Типы soft irq

Индекс	Приоритет	Значения
HI_SOFTIRQ	0	Высокоприоритетные softirq
TIMER_SOFTIRQ	1	Таймеры
NET_TX_SOFTIRQ	2	Отправка сетевых пакетов
NET_RX_SOFTIRQ	3	Прием сетевых пакетов
BLOCK_SOFTIRQ	4	Блочные устройства
BLOCK_IOPOLL_SOFTIRQ	5	// новое
TASKLET_SOFTIRQ	6	Тасклеты
SCHED_SOFTIRQ	7	Планировщик

HRTIMER_SOFTIRQ	8	Не используется
RCU_SOFTIRQ	9	Сохранился для сохранения нумерации

```
const char * const softirq_to_name[NR_SOFTIRQS] = {
    "HI", "TIMER", "NET_TX", "NET_RX", "BLOCK", "IRQ_POLL",
    "TASKLET", "SCHED", "HRTIMER", "RCU"
};
```

Работающие soft irq — `/proc/softirqs`

Чтобы добавить свой soft irq, необходимо перекомпилировать ядро

Если создавать, то с приоритетом ниже тасклетов (разновидностью softirq)

Задача ksoftirq daemon — постановка softirq в очередь ⇒ постановка тасклетов в том числе

Обработчик отложенных прерываний

Прототип обработчика отложенного прерывания, action, выглядит следующим образом:

```
void softirq_handler(struct softirq_action *);
```

При выполнении в ядре этого обработчика отложенных прерываний запускается функция `action`, которой в качестве единственного аргумента передается указатель на соответствующую структуру типа `softirq_action`. Например, если переменная `my_softirq` содержит указатель на элемент массива `softirq_vec`, то ядро вызовет функцию-обработчик соответствующего отложенного прерывания в следующем виде:

```
my_softirq->action(my_softirq);
```

Включение в перечень softirq своего обработчика

Для регистрации обработчика softirq определены следующие функции: (для регистрации обработчика аппаратного прерывания)

На одной и той же линии irq может быть зарегистрировано несколько обработчиков.

```
<linux/interrupt.h>
typedef irqreturn_t (*irq_handler_t)(int, void *);
int request_irq(unsigned int irq, irq_handler_t handler, unsigned long flags, const char *name, void *dev);
```

Следующий этап состоит в регистрации обработчика отложенного прерывания во

время выполнения программы с помощью функции

`open_softirq()`. Ей передаются два параметра:

индекс отложенного прерывания и функция-обработчик.

```
void open_softirq(int nr, void (*action)(struct softirq_action *))
// set the handlers for softirq nr
```

1. Необходимо определить индекс - меньше индекса тасклета
2. Во время инициализации модуля необходимо зарегистрировать обработчик softirq (`open_softirq()`)

Пример:

```
void my_soft_handler(struct softirq_action *)
{...}

static int __init my_init(void)
{
    /* .... */
    request_irq(irq, mytop_interrupt, 0, "my", NULL); //регистрируем свой обработчик
```

```
//аппаратных прерываний
open_softirq(MY_SOFT_IRQ, my_soft_handler);
}
```

Таким образом, при загрузке модуля будет загружен обработчик

Запуск отложенных действий

Перед тем как зарегистрированное отложенное прерывание будет запущено, оно

должно быть промаркировано. Этот процесс называется генерацией отложенного прерывания (`raise_softirq`).

Зарегистрированный `softirq` запускается на выполнение функцией

`raise_softirq()` — генерация отложенного прерывания, вызывает обработчик аппаратного прерывания.

В результате будет сгенерировано отложенное прерывание с указанным.

```
static irqreturn_t mytop_interrupt(int irq, void *dev_id)
{
    ...
    /* mark softirq pending*/
    raise_softirq(MY_SOFT_IRQ); // созданный обработчик softi
    rq, его надо запланировать на выполнение
    return IRQ_HANDLED;
}
```

Несмотря на то, что приведен конкретный пример с `raise_softirq`, это — классический пример обработчика аппаратного прерывания

Правила выполнения `softirq`

1. `softirq`(обработчик) никогда не вытесняет другой `softirq`
 - а. Единственное событие, которое может вытеснить `softirq` — аппаратное прерывание

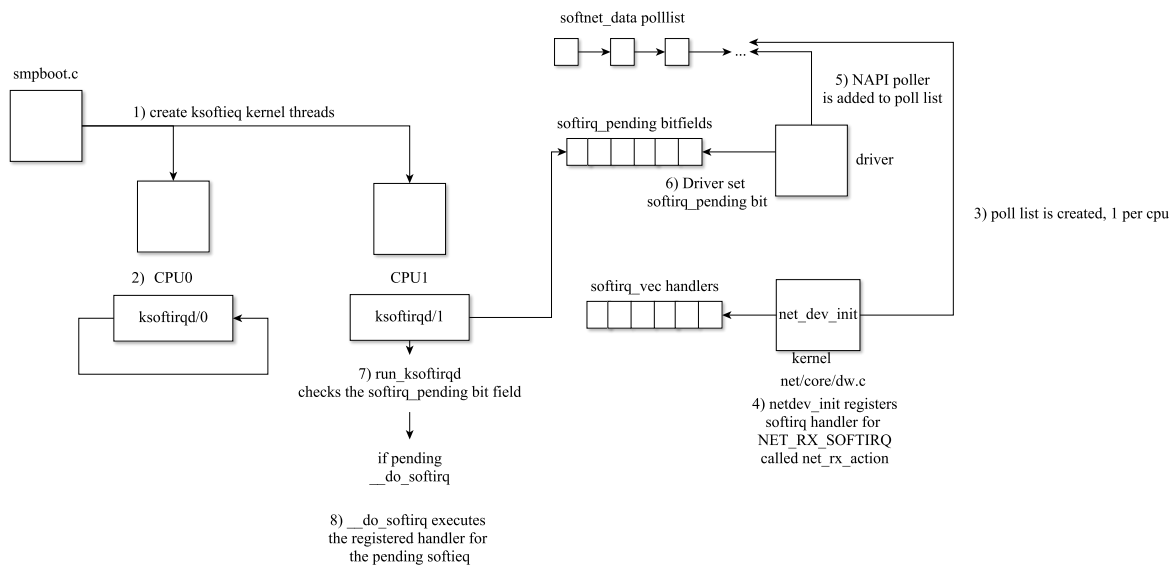
2. На другом процессоре может параллельно выполняться другой softirq / тот же softirq
3. softirq масштабируется $\Rightarrow$ функция, реализующая действие, должна быть реентерабельной, а критические участки должны выполняться монопольно (чаще всего используется spinlock)

NET_RX_SOFTIRQ

1. Схема для двудерной системы, для входящих пакетов.
2. Отражает многоуровневую структуру сетевой подсистемы.

Каждый локальный компьютер имеет сетевую подсистему. Важнейшая часть ОС.

3. 1) $\rightarrow$ 2) $\rightarrow$ 3) $\rightarrow$... — последовательность развертывания в системе.



1. На каждое ядро при загрузке системы создается и инициализируется `ksoftirq` daemon.
Его задача выполнять отложенные действия.
`ksoftirq` daemon работает с `softirq` — гибкие прерывания.
Тасклет — один из типов `softirq`.

▼ Код функции для понимания

```

static struct smp_hotplug_thread softirq_threads = {
    .store                = &ksoftirqd,
    .thread_should_run    = ksoftirqd_should_run,
    .thread_fn            = run_ksoftirqd,
    .thread_comm          = "ksoftirqd/%u",
};

static __init int spawn_ksoftirqd(void)
{
    cpuhp_setup_state_nocalls(CPUHP_SOFTIRQ_DEAD,
    "softirq:dead", NULL,
    takeover_tasklets);
    BUG_ON(smpboot_register_percpu_thread(&softirq_
threads));

    return 0;
}
early_initcall(spawn_ksoftirqd);

```

2. Для первого ядра ksoftirq daemon processed loop are started. То есть находится в петле выполнения.
Вся информация приводится для cpu1.
3. Создаются poll list, по одному на ядро

▼ Код для понимания

```

static int __init net_dev_init(void)
{
    struct softnet_data *sd = &per_cpu(softnet_data,
i);
    ...
    INIT_LIST_HEAD(&sd->poll_list);
    ...

    for_each_possible_cpu(i) {

```

```

        ...
        net_page_pool_create(i)
        ...
    }
    ...
}

```

Структура `softnet_data` имеет в ядре следующее описание:

```

/* * Incoming packets are placed on per-CPU queues */

```

4. `softirq_vec_handlers` — массив обработчиков входящих пакетов.
 (`netdev_init` регистрирует обработчик `softirq` для типа `NET_RX_SOFTIRQ`)
 (то есть с помощью функции `open_softirq` выполняется регистрация обработчиков в массиве `struct softirq_action softirq_vec[NR_SOFTIRQS]`)

▼ Код функции для понимания (сокращенный)

```

static int __init net_dev_init(void)
{
    ...

    open_softirq(NET_TX_SOFTIRQ, net_tx_action);
    open_softirq(NET_RX_SOFTIRQ, net_rx_action);

    ...
}

```

5. Самое интересное — наличие NAPI.
 New Api появилось в версии ядра 2.4.
 Прерывания от сетевой карты стало частым событием, а обработка прерываний — затратное действие.
 Несмотря на то, что действия аппаратных прерывания минимизируются (помещает входящий пакет в кольцевой буфер(для сетевой подсистемы) и планируется отложенное действие, которое закончит работу, но будет

выполнено на другом уровне приоритета.)

Обработчик аппаратного прерывания

регистрирует отложенное действие, и это отложенное действие запускается демоном softirq.

Но все равно очень затратно, так как в архитектуре x86 не реализовано вложенных прерываний

⇒ аппаратные прерывания выполняются при запрете всех прерываний на данном ядре, и запрете прерывания по данной линии irq на других процессорах.

Если аппаратное прерывание приходит на CPU0, то на нем будут запрещены все аппаратные прерывания, а на CPU1 будут запрещены аппаратные прерывания по данной линии IRQ.

Это серьезное ограничение быстродействия.

⇒ для сетевой подсистемы для входящих пакетов прерывания заменяются на опрос.

Основная цель NAPI — сократить количество прерываний, генерируемых при получении пакетов.

В NAPI механизм прерываний сочетается с опросом.

6. softirq_pending bitfields — битовое поле.

Демон ksoftirq обращается к битовому полю (крайне компактный способ отражения есть/нет). В данном случае в битовом поле отмечается информация, пришел ли пакет.

У сетевой карты есть драйвер. Он инициализирует битовую очередь softirq, в которую softirq помещаются при приходе пакетов.

Когда пакет приходит, драйвер заполняет один бит соответствующего поля, демон его анализирует и понимает, что надо обработать пакет.

Основная задача демона: обработка отложенных действий, в частности обработка сетевых пакетов.

Работа драйвера: установить бит.

7. `run_ksoftirq` — проверяет `bitfields`.

Если бит выставлен, то пакет пришел и его надо обработать ⇒ вызывается функция `do_softirq`.

▼ Код для понимания

```
static void run_ksoftirqd(unsigned int cpu)
{
    ksoftirqd_run_begin();
    if (local_softirq_pending())
    {
        handle_softirqs(true);
        ksoftirqd_run_end();
        cond_resched();
        return;
    }
    ksoftirqd_run_end();
}
```

8. Функция `do_softirq` выполняет зарегистрированный обработчик.

**Аппаратное прерывание выполняется на высоком уровне приоритета.
(отложенные действия выполняются на другом уровне приоритета)**

Тасклеты

Как и остальные механизмы ядра, тасклеты модернизируются и улучшаются. Тасклеты появились в ядре начиная с версии v2.3

Тасклеты являются типом `softirq`, но они написаны так, чтобы упростить написание собственных отложенных действий.

Не могут блокироваться

Почему возникла необходимость в тасклетах?

Кратко: tasklety не могут сериализоваться ⇒ не надо писать
взаимоисключение ⇒ их написание упрощается

Подробно:

softirq могут сериализоваться. Одно и тоже softirq, например, net_rx_softirq, могут выполняться одновременно на разных процессорах. Но это требует реализации в коде softirq взаимовыключения. Мало вызвать mutex — надо вызвать в нужном месте. Нужно гарантировать параллельное выполнение без потери данных.

Следовательно, tasklety созданы таким образом, что они не могут сериализоваться.

В один момент может выполнять только один tasklet, решающий конкретную задачу ⇒ в tasklete не нужно писать взаимовыключение.

То есть написание taskleta в силу ограничения их выполнения упрощается.

Структура taskleta

Если softirq могут регистрироваться только статически, то tasklety могут регистрироваться как статически, так и динамически.

Библиотека:

```
<linux/interrupts.h>
```

```
struct tasklet_struct {
    struct tasklet_struct *next; /* Указатель на следующий та
склет в списке */
    unsigned long state;
    atomic_t count;
    bool use_callback;
    void (*func)(unsigned long data); /* Функция-обработчик т
асклета */
    unsigned long data; /* Аргумент функции-обработчика таск
лета */
};
```

1. state — текущее состояние.

У tasklet может быть одно из следующих состояний:

```
enum {  
    TASKLET_STATE_SCHED, //запланирован  
    TASKLET_STATE_RUN //выполняется  
}
```

TASKLET_STATE_RUN имеет значение только в многопроцессорной системе, так как если tasklet выполняется, то такой tasklet на другом процессоре выполняться не может.

2. count — счетчик ссылок

Если счетчик ссылок равен нулю, то tasklet разрешен и может выполняться, если он отмечен как ждущий выполнения.

Если count > 0, то выполнение запрещено.

Для большей эффективности tasklet выполняется на том же процессоре, на котором был запланирован. То есть там, где выполнялся обработчик аппаратного прерывания.

Обычно после обработки аппаратного прерывания выполняется tasklet.

Но

приоритет отложенного действия ниже приоритета аппаратного прерывания. Перебить выполнение tasklet может пришедшее аппаратное прерывание.

Планирование tasklet на выполнение

Для планирования tasklet должна быть вызвана функция `tasklet_schedule`

Прототипы:

1. tasklet_vec (для обычных taskletов)

```
extern void __tasklet_schedule(struct tasklet_struct *);
```

```
static inline void tasklet_schedule(struct tasklet_struct *t)
{
    if(!test_and_set_bit(TASKLET_STATE_SCHED, &t->state))
        __tasklet_schedule(t);
}
```

2. Также есть функция для планирования высокоприоритетных тасклетов:

```
extern void __tasklet_hi_scheduler(struct tasklet_struct *t);
```

Регистрация тасклета

Тасклет может быть инициализирован статически и динамически. Для этого есть специальные средства.

1. Статические — через `declare_tasklet`

Макрос:

```
DECLARE_TASKLET(name, _callback)
```

2. Динамически через вызов функции `tasklet_init`, которой передается указатель на инициализируемую структуру `tasklet_struct`, и обработчик тасклета `handler`.

```
void tasklet_init(struct tasklet_struct *t,
                 void (*func)(unsigned long), unsigned long data)
{
    t->next = NULL;
    t->state = 0;
    atomic_set(&t->count, 0); //The default setting is 0
    and can be called
    t->func = func;
    t->data = data;
}
```

Удаление тасклета

Если есть функция `tasklet_init`, то должна быть функция, которая удаляет тасклеты из системы:

```
extern void tasklet_kill(struct tasklet_struct *t);
```

Написание собственной функции-обработчика тасклета

Функция-обработчик тасклета должна соответствовать корректному прототипу.

```
void tasklet_handler(unsigned long data);
```

Очереди работ

Понятие очереди работ

Есть еще один механизм реализации отложенных действий — очереди работ `workqueue`.

Видели кучу `worker`-ов — потоков ядра.

В отличие от тасклетов, которые являются типом `softirq`, **очереди работ отношения к `softirq` не имеют.**

Очереди работ — отдельный, более гибкий механизм реализации отложенных действий.

Но две составляющие. `Work`(работа) ставится в очередь. Создается очередь, ставится `work`, и пишутся обработчики.

Выполнением очереди работ заведует рабочий поток ядра.

Особенностью является то, что они могут блокироваться.

Тасклеты блокироваться не могут!

Данный тип нижних половин обработчиков всегда выполняется в контексте процесса и,

как следствие, пользуется всеми преимуществами контекста процесса.

Главное из них

состоит в том, что этим процессом управляет системный планировщик, поэтому выполняющийся код может переходить в состояние ожидания.

Структуры для управления очередями работ

Для управления отложенными действиями типа очередей работ в ядре имеется ряд структур, которые связаны между собой. Таким образом при работе с очередью работ инициализируется целый ряд объектов, чтобы управлять очередями работ.

1. Работа (work);
2. Очередь работ (workqueue) – коллекция work. Workqueue и work относятся как один-ко-многим;
3. Рабочий (worker). Worker соответствует потоку ядра worker_thread;
4. Пул рабочих потоков (worker_pool) это – набор рабочих (worker). Worker_pool и worker относятся как «один ко многим»;
5. Pwd (pool_workqueue) это – посредник, который отвечает за отношение workqueue и worker_pool: workqueue и pwd является отношением один-ко-многим, а pwd и worker_pool – отношение один-к-одному.

Каждое действие представлено с помощью структуры work_struct, определенной в файле <linux/workqueue.h>. Эта структура показана ниже

```
struct work_struct {
    atomic_long_t data;
    struct list_head entry;
    work_func_t func;
};
```

queue_worker считают за обычный процесс.

Соответственно его планируют как обычный процесс, то есть его планированием занимается стандартный планировщик ядра.

у

worker приоритет 120.

```

struct workqueue_struct {
    struct list_head    pwqs;
    struct list_head    list;

    struct worker        *rescuer;

    char                name[WQ_NAME_LEN];

    struct pool_workqueue __percpu *cpu_pwqs; // pool workque
ues на процессор.
    struct pool_workqueue __rcu *numa_pwq_tbl[]; // для новых
систем - numa
};

```

Видимая из вне очередь работ — она передает рабочие элементы worker_pool через его pool_workers.

Эта структура определена в файле kernel/workqueue.c. В ней содержится массив структур cpu_workqueue_struct, каждый элемент которого соответствует одному процессору в системе. Поскольку рабочие потоки создаются для каждого процессора в системе, для каждого рабочего потока, работающего на каждом процессоре машины, существует такая структура

В дополнение к приведенным структурам есть struct cpu_workqueue_struct. То есть для каждого cpu есть структура, описывающая очередь к этому ядру.

```

struct cpu_workqueue_struct {
    spinlock_t lock;
    long remove_sequence;
    long insert_sequence;
    struct list_head worklist;
    wait_queue_head_t more_work;
    wait_queue_head_t work_done;
    struct workqueue_struct *wq;
    struct task_struct *thread;
};

```

```

    int run_depth;
} ____cacheline_aligned;

```

pool_workqueue — очередь работ на pool. То есть в системе рассматривается набор очередей работ. Может быть, что это связано с работой планировщика системы, поскольку он планирует выполнение всех работ ⇒ это должно быть собрано воедино, чтобы эффективно планировать выполнение работы.

```

struct worker_pool {
    raw_spinlock_t      lock; // низкоуровневый spinlock
    int                 cpu;
    int                 node; //node id
    int                 id; // pool id
    int                 nr_running; // число выполняемых работ

    struct list_head     worklist;
    //list of pending works
    //список ожидающих работ, то есть список работ, которые о
жидают получения
    //процессорного времени, то есть речь идет об очереди к п
роцессору.

    int                 nr_workers; // общее число рабочих
    int                 nr_idle; // число неработающих worker

    struct list_head     idle_list; //список неработающих work
er
    struct timer_list     idle_timer; /* worker idle timeout */
    // то есть для worker может быть установлен timeout.
    // Это отложенное действие и отложенное выполнение через
какое-то время
    struct timer_list     mayday_timer;
};

```

```

struct pool_workqueue {
    struct worker_pool  *pool;
    struct workqueue_struct *wq;

    int          nr_active; // количество активных работ
    struct list_head  inactive_works;
    struct list_head  pending_node; // список ожидающих узлов
    struct list_head  pwqs_node; // список узлов workqueue
    в пуле
    struct list_head  mayday_node; // список узлов бедствия
};

```

`worker_pool` — на `cpu` ⇒ видим индекс ядра `cpu`.
 Можно предположить, что `pool` имеет `inode`.

Создание очереди работ

Для создания очереди работ используется функция `alloc` (в старых материалах использовался `create`).

```

struct workqueue_struct *alloc_workqueue(const char fmt,
                                         unsigned int flags, int max_activ
e);

```

1. `fmt` — имя формата `printf`. В отличие от старой реализации, не создаются потоки с использованием этого имени.
2. `flags` определяет, как будет выполняться очередь работ
 Флаги:
 - a. `WQ_HIGHPRI` — указывает, что поток, обслуживающий соответствующую работу, будет иметь повышенный приоритет.
 - b. `WQ_UNBOUND` — очередь работ не будет привязана к конкретному ядру
 - c. `WQ_CPU_INTENSIVE` — имеет смысл только для привязки очередей работ и означает отказ от дополнительной организации параллельного

выполнения

d. WQ_FREEZABLE — можно блокировать

Функция `alloc_workqueue` создает очередь работ, в нее можно поставить свои работы.

Способы создания работ

Работы могут создаваться статически и динамически.

1. Статически:

```
DECLARE_WORK(name, void (*func)(void*));
```

a. `name` — указатель на `struct work_struct`

b. `func` — функция, которая будет вызвана из `work_queue`

2. Во время выполнения программы отложенное действие создается путем передачи

указателя на структуру с использованием макроса

`INIT_WORK`, как показано ниже.

```
INIT_WORK(struct work_struct *work, void (*func)(void *),  
void *data);
```

В результате отложенное действие будет проинициализировано динамически с помощью структуры, на которую указывает указатель `work`, функции-обработчика `func` и аргумента `data`.

Обработчик отложенного действия

Функция-обработчик отложенного действия имеет следующий прототип:

```
void work_handler(void *data);
```

Эта функция будет выполняться в рабочем потоке и, следовательно, в контексте процесса

Планирование на выполнение

После создания отложенного действия системный планировщик должен поставить его в очередь на выполнение.

```
bool schedule_work(struct work_struct *work);
```

Сравнение softirq, tasklets, очередей работ

Softirq:

- Сериализуемы (могут параллельно выполняться на нескольких процессорах)
- Не могут блокироваться
- Регистрируются статически (нужно перекомпилировать ядро)

Тасклеты:

- Несериализуемы (выполняются строго на одном процессоре — на том же, на котором и был запланирован)
(два тасклета одного типа не могут выполняться одновременно)
- Не могут блокироваться
- Могут регистрироваться как статически, так и динамически

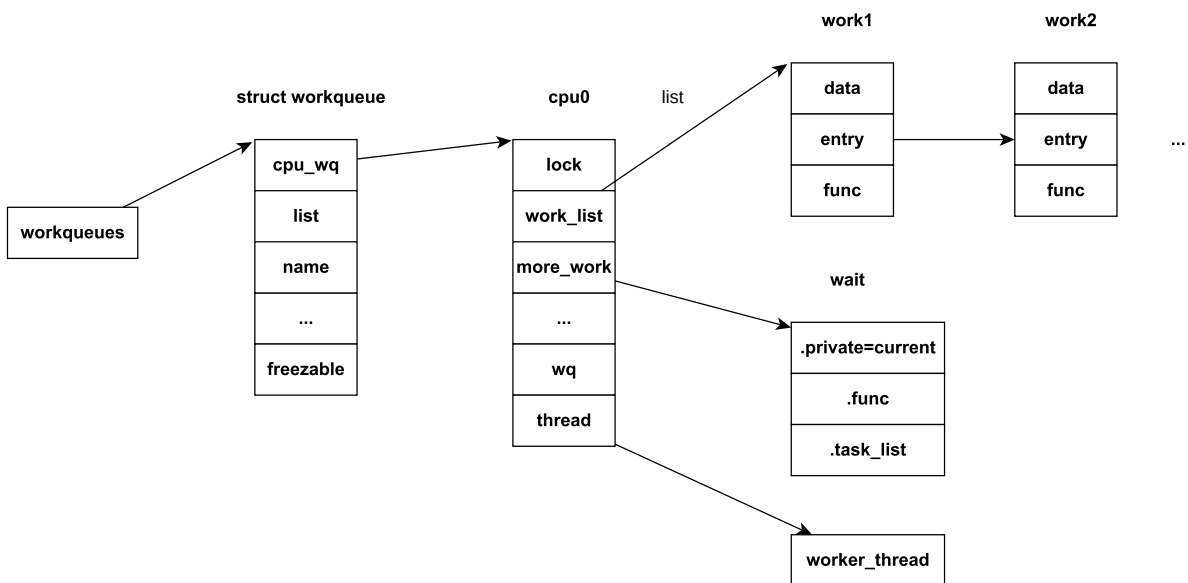
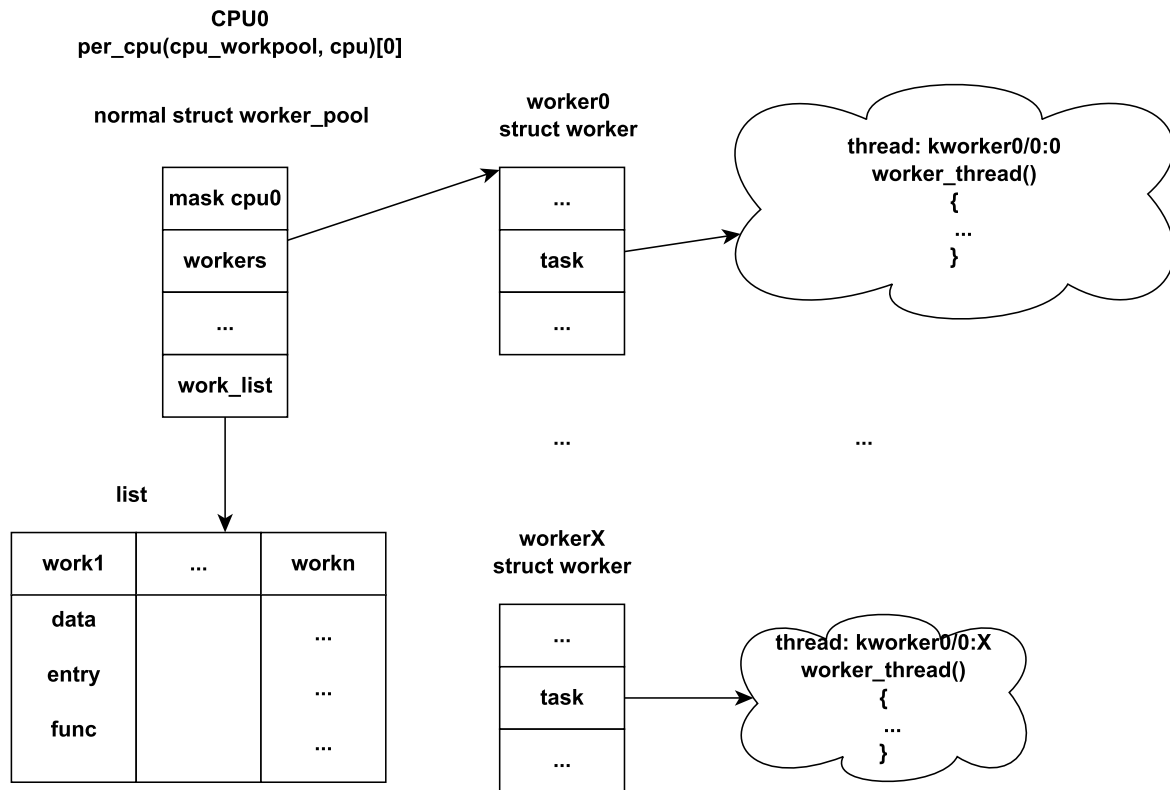
Очереди работ:

- Могут блокироваться
- Могут регистрироваться как статически, так и динамически

Связь структур для одного сри — сри0

normal — нормальный приоритет (есть повышенный — high)

По словам Рязановой эта схема не нужна на экзамене



Специальные файлы устройств

Самая большая часть кода ОС связана с управлением внешними устройствами, опосредовано или напрямую.

Напрямую — драйвера.

Типы драйверов в Linux

1. Драйверы нижнего уровня (функциональные).

Эти драйверы являются частью кода ОС, и загружаются при загрузке системы.

Эти драйверы пишут разработчики устройств, так как только они знают формат передаваемых драйверу данных.

Например, у многих видеокарт не существует драйверов под Linux. И такой драйвер не напишешь, так как надо знать форматы потоков данных, которые отправляет и получает устройство.

Эти драйвера должны быть частью ОС: они загружаются при старте системы и работают все время.

2. Смешанного уровня: состоит из двух частей:

а. одна часть загружается при загрузке системы

б. вторая часть выполняется в виде загружаемого модуля ядра и соответственно может быть выгружена

Чтобы управлять устройством, нужно выполнить соответствующие команды, чтобы загрузить модуль. Но часть кода загружается при старте ОС.

Такое разбиение сделано для экономии адресного пространства ядра: когда отпадает необходимость в устройстве, то загружаемый модуль может быть выгружен.

3. Драйвера верхнего уровня

Реализуются как загружаемые модули ядра. У них написаны специальные структуры и функции, которые способны перехватывать низкоуровневые обработчики.

Задача идентификации устройств в системе

Если исходить из парадигмы Unix — устройство тоже файл. Система должна как-то идентифицировать внешнее устройство.

В системе существует понятие специальных файлов устройств, которое обеспечивает унифицированный доступ к девайсу.

Эти файлы обеспечивают связь файловой системы и драйверов устройств.

Эти фактические драйверы можно увидеть в /dev корневой ФС.

Подкаталог /dev/fd содержит файлы с именами 0, 1 и 2.

В некоторых системах непосредственно указываются имена stdin, stdout, stderr.

Типы файловых устройств

В системе поддерживается только 2 типа файловых устройств:

1. Символьный небуферизуемый
2. Блочный буферизуемый

Связь имени специального файла с конкретным устройством обеспечивает **индексный дескриптор (inode)**.

Тип специального файла задает поле `i_mode`.

При этом для блок-ориентированного файла указывается значение: `i_mode = 060000`

Для байт-ориентированного (символьного): `i_mode = 020000`.

Идентификация файлов устройств

В Linux файлы устройств идентифицируются двумя положительными числами: `major` и `minor device number` (старший и младший номера устройств).

Старший номер

Старший номер устройства как правило идентифицирует тип устройства. Или чаще старший номер идентифицирует драйвер, ассоциированный с устройством.

Например, жесткий диск — разделен на разделы. Каждый раздел будет иметь свой младший номер.

Список старших номеров устройств можно найти в файле:

```
/usr/include/linux/major.h
```

В файле type.h — <linux/types.h> есть функция:

```
unsigned int major(dev_t dev);
```

При этом функция не определена в POSIX.

Наиболее часто используемые значения старших номеров

Старший номер	Тип устройства
1	ОЗУ
2	дисковод гибких дисков
3	первый контроллер жестких дисков
4	терминал
5	терминал
6	принтер (параллельный порт)
8	жесткие SCSI диски

Файлы устройств одного типа имеют одинаковые имена и отличаются только по номеру, который добавляется в конец имени.

Например, все файлы сетевых карт имеют имена eth: eth:0, eth:1, ...

Младший номер

Младший номер:

```
unsigned int minor(dev_t dev);
```

Младший номер идентифицирует конкретное устройство.

Младший номер диска ... partition, а на partition монтируется файловая система.

`#ls -l` — увидим 2 числа, которые разделены запятыми: это старший и младший номера.

Если выполнить команду `cat /proc/devices`, то можно увидеть старшие номера устройств, известные ядру.

dev_t

Определены два макроса:

```
MAJOR(dev_t dev);  
MINOR(dev_t dev);
```

Стандарт POSIX указывает на существование типа `dev_t`, которые определен в библиотеке `<linux/types.h>`

Но формат полей не оговаривается: POSIX признает существование типа, но не формализует формат.

Начиная с версии ядра 6.2.0 `dev_t` — 32 разрядное число.

12 разрядов для старшего номера, 20 разрядов для младшего номера.

С помощью макросов можно получить значения старшего и младшего номера устройства.

Если есть номер, то используя макрос `MKDEV(int major, int minor)` можно перейти к экземпляру типа.

Определение (выделение) старших и младших номеров

Старшие и младшие номера могут быть определены **статически в коде драйвера** или могут быть получены драйвером динамически.

1. Если старший и младший номера определены статически, то система будет пытаться их использовать. И если они уже используются, то будет ошибка.
2. При динамическом определении номеров, после того, как номера назначены, они резервируются и не могут быть использованы другими

модулями.

Одно из первых действий, которые должен выполнить разработчик драйвера (в качестве примера возьмем драйвер символьного устройства) — нужно получить 1 или более номеров устройств, чтобы работать с ним.

Для этого необходимо вызывать следующую функцию.

```
int register_chrdev_region(dev_t from, unsigned count,  
                           const char  
                           *name)
```

Функция объявлена и описана в файле <linux/fs.h>

1. Первый параметр — начальный номер диапазона номеров устройств, который вы хотите выделить. Причем часть `minor` часто ноль, но нет специальных требований.
2. `count` — количество номеров устройств, которые запрашиваются
3. `name` — имя устройства, которое будет связано с диапазоном.
Его можно увидеть, используя:

```
cat /proc/devices
```

, а также `sysfs`.

dev_t и подводка к USB (rdev в inode)

Идентификация устройств в системе — все файл, все имеет `inode`. Но внешние устройства, существенно отличающиеся от файлов, должны идентифицироваться особым образом ⇒ в Linux введен тип `dev_t`.

```
typedef u32 __kernel_dev_t;  
typedef __kernel_dev_t dev_t;
```

```
struct inode  
{  
    ...  
    dev_t i_rdev;
```

```
...  
}
```

Это поле связанного устройства.

Если это inode физического файла, то это устройство связано с этим физическим файлом, но в каждом отдельном случае это отдельный

Пример работы со старшими и младшими номерами устройств. Информация о драйверах является важной, так как управление устройствами осуществляют драйвера.

Изменение работы внешнего устройства может быть осуществлено написанием собственного драйвера.

```
int register_chrdev_region(dev_t from, unsigned count, const  
char *name);  
int alloc_chrdev_region(dev_t *dev, unsigned baseminor, unsig  
ned count, const char *name)
```

Эти две функции являются функциями, которые вызываются в драйверах символьных устройств.

Пример драйверов символьных устройств можно найти в интернете.

С помощью первого системного вызова число count регистрируется среди номеров файлов устройств (регистрирует диапазон N символьных устройств, которые начинаются с from с именем name).

Второй системный вызов динамически определяет свободный старший номер и регистрирует число count среди номеров файлов устройств.

```
if(alloc_chrdev_region(&first, 0, 3, "shweta") < 0)  
{  
    return -1;  
}  
printk(KERN_INFO "<major: minor>: <%d: %d>\n", MAJOR(first),  
MINOR(first));
```

USB

Организация работы с внешними устройствами — самая сложная задача операционной системы.

Большое количество структур и команд предназначены для работы с внешними устройствами.

Основанная на архитектуре фон Неймана архитектура компьютера меняется. Видели Numa — сильно поменялась материнская плата. Это также безусловно касается шин.

В компьютере постоянно передаются потоки данных.

Работа с сетью, мышью, принтером — потоки данных.

Если к системе подключить много устройств, то шина становится узким местом системы.

При этом если вернуться на архитектуру Pentium, то в его составе была шина PCI.

Для архитектуры x86 intel шина оставалась важной составляющей.

Pro PCI Express.

Структуры для работы с внешними устройствами общего вида

Существуют спецификации для USB. К USB подключаются внешние устройства

В ядре имеется целый ряд структур для работы с внешними устройствами общего вида

Если говорить о драйверах, то такой структурой является:

```
struct device_driver {
    const char      *name;
    const struct bus_type *bus;

    struct module    *owner;

    const struct of_device_id *of_match_table;
```

```

    int (*probe) (struct device *dev);
    int (*remove) (struct device *dev);
    void (*shutdown) (struct device *dev);
};

```

В отличие от `struct usb_driver`, где есть функция `disconnect`, здесь функция `remove` и `shutdown`

1. `of_match_table` — фактически это поле содержит важную информацию об устройстве. Видим, что рассматривается как девайс `id`.
2. `probe` — функция, в которой осуществляются действия по подключению устройства, которое подключается, инициализация структуры (функция проверки подключения устройства, контроль подключения устройства)

struct device — самая низкоуровневая структура ядра, определенная для работы с устройствами. "Базовая структура `device`". То есть на самом низком уровне каждое устройство в системе описано этой структурой. То есть существует ее экземпляр для каждого устройства, которое инициализируется системой.

Эта структура инициализируется автоматически самой системой на основании информации, которую предоставил разработчик драйвера.

```

struct device {
    struct kobject kobj;
    struct device      *parent;

    const char      *init_name; /* initial name of the device
*/
    const struct device_type *type;

    const struct bus_type  *bus; /* type of bus device is
on */
    struct device_driver *driver; /* which driver has alloc
ated this device */

```

```

    u64      *dma_mask; /* dma mask (if dma'able device) */
    u64      bus_dma_limit; /* upstream dma constraint */

#ifdef CONFIG_NUMA
    int      numa_node; /* NUMA node this device is close to
    */
#endif
    dev_t      devt; /* dev_t, creates the sysfs "dev"
    */
};

```

Опять о DMA

DMA присутствует тотально.

Это способ освобождения процессора от передачи данных через регистры. На первых этапах все данные передавались через регистры, то есть пока выполняется передача данных — процессор ничего не может делать — регистры заняты.

Появился контроллер DMA, который, как и контроллер прерываний, расположен на материнской плате.

Контроллер DMA получает от процессора начальный адрес области памяти, в которую надо записать данные, ее размер в байтах и область, из которой они могут быть прочитаны.

... — Потом DMA стал тормозить передачу данных, так как он существенно медленнее процессора ⇒ были публикации: если процессор свободен, то данные будут передаваться через регистры.

Была диаграмма — искали нужную эффективную передачу данных — 1 из способов — кража циклов.

Сейчас DMA очень активно используется: аппаратная составляющая ПК активно развивается и DMA не могут не использоваться ⇒ в структурах есть поля с DMA.

USB топология

Прежде всего говорят о шине USB и существуют спецификации USB, существует несколько версий, так как usb совершенствуется в плане увеличения скорости передачи информации.

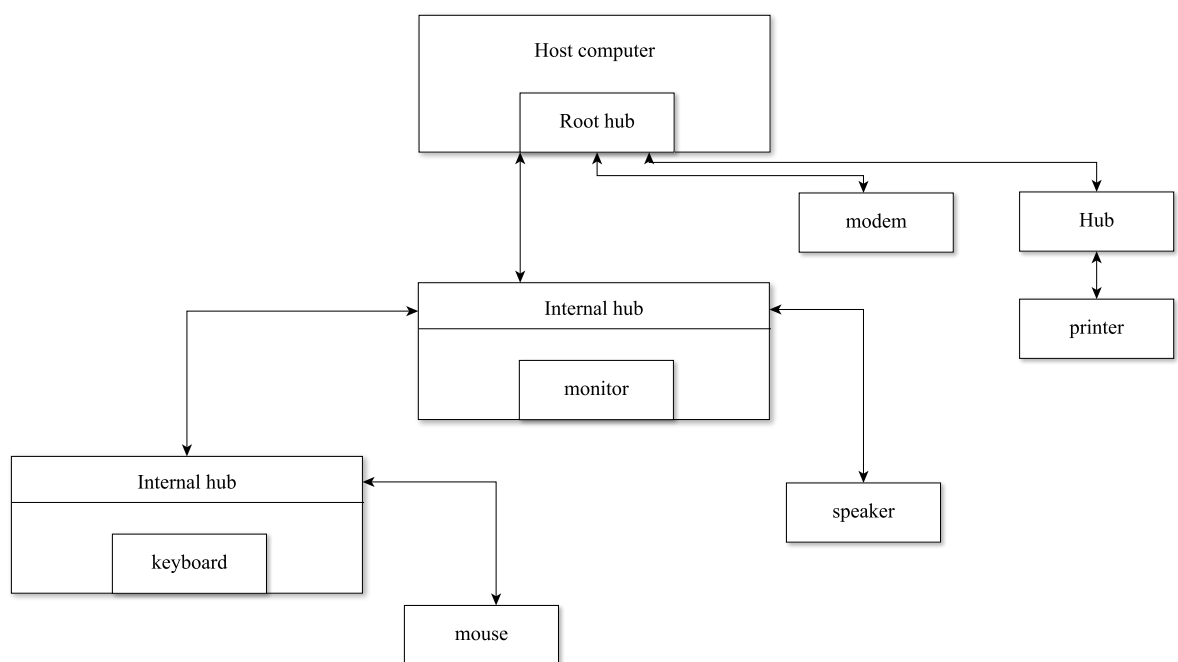
Все устройства, даже датчики, являются устройствами ввода-вывода

У ПК могут быть только устройства ввода/вывода (даже датчики). Других устройств у ПК быть не может ⇒ все спецификации изменяются в плане количества подключенных в usb устройств.

Спецификации определяют топологию USB: многоярусная звезда — tiered star.

Одновременно к 1 шине может быть подключено до 127 устройств.

Рассмотрим дерево usb — usb топологию (**многоярусная звезда, tiered star**)



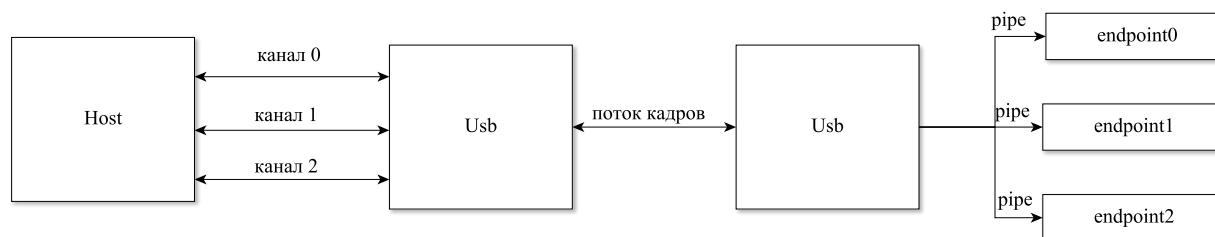
1. Host ПК — главный ПК
2. Hub — сетевой концентратор или повторитель, через который соединены узлы сети
3. Root hub — корневой повторитель

Если есть необходимость, добавляется internal hub, и это позволяет подключать до 127 устройств.

Многие подключенные устройства имеют интегрированные в себя хабы. Например, клавиатура может содержать hub, подключенный к компьютеру. Мышь и другие устройства могут быть воткнуты в клавиатуру. Многие мониторы так же имеют встроенный хаб.

Вводится понятие endpoint

Конечные точки могут быть описаны как источники или приемники данных(буфер). Поскольку шина является хост-ориентированной, конечные точки оказываются в конце канала связи, на функции USB. (из инета)



Каналы — пайпы — двунаправленная передача данных.

Поток/канал — логическое соединение между хостом и конечной точкой (не стоит путать с программными каналами).

У USB подсистемы существует несколько уровней, которые управляют соответствующим ПО.

USB поддерживает

т горячее подключение, то есть устройство, подключенное к USB порту, становится доступным без дополнительных действий.

Архитектура USB драйвера

Хостом может быть или компьютер, или usb контроллер.

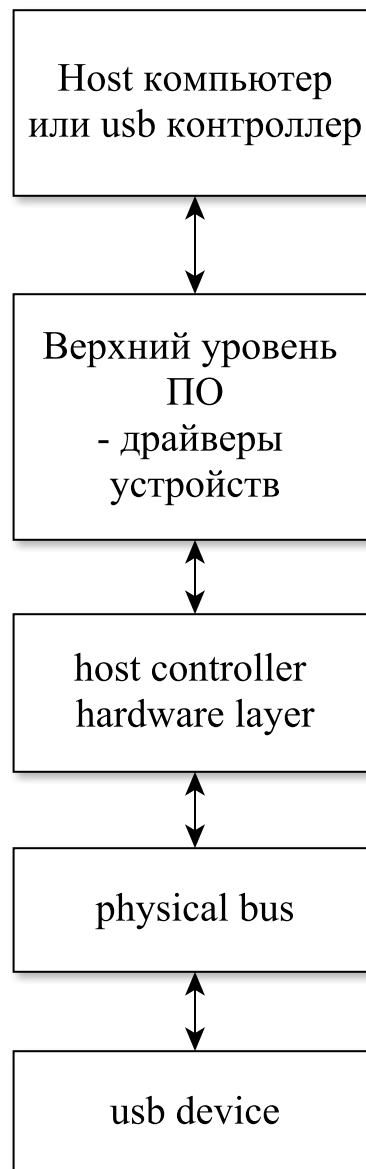
Основная задача, которую решает хост — контролировать данные, передаваемое от или к usb устройству.

root hub — корневой повторитель. Соответственно, хост компьютер имеет два уровня.

Верхний уровень программного обеспечения — драйвер устройства.

Второй уровень — хост-контроллер — уровень аппаратного обеспечения.

Хост-контроллер — это главный управляющий шиной USB. Именно он обеспечивает связь устройств, подключенных к шине, с компьютером (инета)



Физическая шина usb — usb кабель, который соединяется контроллер и периферию.

usb кабель состоит из 4-х медных проводников. Последовательная передача данных. Из 4 проводников: 2 — питание и 2 — витая пара в заземленной оплетке.

На тех частотах, на которых работает компьютер, никакая параллельная передача данных невозможна. Была бы возможна, если бы параллельные проводники были в заземленной оплетке, но тогда бы провода очень утолщились.

То есть на таких частотах никакой параллельной передачи данных осуществить невозможно.

Высокая скорость, так как шины работают на огромных частотах.

Шина — устройство, которое может иметь собственный тактовый генератор.

В процессе передачи данных выполняются следующие действия:

1. usb устройство инициирует передачу, используя функции интерфейса usb драйвера, выдавая запросы к модулю usb драйвера.
2. usb драйвер отсылает запросы к host controller driver module: HCD Module
3. HCD модуль делит запросы на отдельные транзакции, учитывая особенности шины и характеристики подключенного к шине usb устройства, и планирует транзакции по шине.
4. host controller выполняет и завершает транзакцию.

Транзакции определяются шиной: устройство полностью зависимо

Типы передачи данных

Существует 4 типа (режима) передачи данных:

1. control
2. interrupt
3. bulk
4. isochronus

Передача осуществляется между буфером компьютера и конечной точкой. Endpoint находится на usb девайсе. Шина является хост ориентированной — направленной к хосту ⇒ endpoint находится в конце канала связи. Активной стороной шины всегда является контроллер.

Передача пакета от устройства к контроллеру реализуется как короткий вопрос контроллера и длинный, содержащий данные, ответ устройства.

Многие контроллеры используют DMA со сложной DMA прогр. форм. др.

Размер пакета для endpoint — это забитая в таблицу конечных точек устройств константа, выбираемая разработчиками.

1. control — гарантированная доставка. Передача этого типа двунаправленная и предназначена для обмена с устройством пакетами типа вопрос/ответ.
Используется системным ПО USB для отправки опред. общих команд на usb устройство и позволяет программному обеспечению операционной системы прочитать информацию об устройстве, такую как коды производителя(vender) и модели(product).
Это важная информация, которая позволяет идентифицировать.
vender и product: один и тот же vendor может быть, но продукт другой.
+ есть доп поля
2. isochronous канал — канал непрерывной передачи данных, имеет гарантированную пропускную способность (N пакетов за 1 период шины).
Количество пакетов за один период шины — скорость доставки.
И определяются задержки — все вместе время передачи.
Передача осуществляется без подтверждения приема.

Нужна для приложений реального времени (аудио, видео)

3. interrupt — короткие, до 64 байт на полной скорости, и до 8 байт на низкой скорости. Как правило это или ввод символов или координаты.
Прерывания имеют спонтанный характер и должны обслуживаться не медленнее, чем требует устройство. Но это не аппаратные прерывания, а специальный тип передачи данных.

4. bulk — потоковая передача данных, используется устройствами, которые принимают и отправляют большое количество данных на высокой скорости за короткое время, но не гарантируется определенная пропускную способность или минимальная задержка: как получится так и передадим.

При этом bulk передается в последнюю очередь, то есть имеет низкий приоритет и занимает всю полосу пропускания шины.

Форматы передачи данных знают разработчики устройств. По аналогии с Windows, где многоуровневые драйверы, загружаемые модули ядра являются драйверами верхнего уровня, которые перехватывают данные от функциональных драйверов — драйверов нижнего уровня.

struct usb_driver vs struct usb_device_driver

```
struct usb_driver {
    const char *name;

    int (*probe) (struct usb_interface *intf,
                  const struct usb_
device_id *id);
    void (*disconnect) (struct usb_interface *intf);

    const struct usb_device_id *id_table;
};
```

В id_table записывается product_id и vender_id.

```
struct usb_device_driver {
    const char *name;

    bool (*match) (struct usb_device *udev);
    int (*probe) (struct usb_device *udev);
    void (*disconnect) (struct usb_device *udev);
};
```

```

    struct device_driver driver;
    const struct usb_device_id *id_table;
};

```

В `usb_driver` работаем с интерфейсом и есть `table`, а здесь работаем с устройством: `struct usb_device`

Важнейшей структурой при работе с пакетами, передаваемых в виде транзакций (неделимая передача, пока не закончится — прервать нельзя) является `struct urb`.

Эта структура содержит всю соответствующую информацию для выполнения любой usb транзакции и для доставки данных и статуса. То есть когда речь идет о способах передачи данных — передаются URB!

```

struct urb {
    /* public: documented fields in the urb that can be used
    by drivers */
    struct list_head urb_list; /* list head for use by the u
rb's current owner */
    struct usb_device *dev; /* (in) pointer to associated
device */
    struct usb_host_endpoint *ep; /* (internal) pointer to
endpoint */
    unsigned int pipe; /* (in) pipe information */
}

```

Данная структура определяет запросы на передачу данных по usb.

Пакеты должны выделять функциями `usb_alloc_urb()` и `usb_free_urb()`

В `init` вызывается функция `usb_register`

```

static int __init xbox_module_init(void)
{
    return usb_register(&xbox_drivers);
}

```

```

static void __exit xbox_module_exit(void)
{
    usb_deregister(&xbox_driver);
}

static struct usb_driver xbox_driver =
{
    .name = DEV_NAME,
    .probe = xbox_probe,
    .disconnect = xbox_remove,
    .id_table = xbox_table
};

static const struct usb_device_id xbox_table[] = {{USB_DEVICE
(VID, PID)}, {}};

static int xbox_probe(struct usb_interface *interface, const
struct usb_device_id *id)
{
    struct usb_endpoint_descriptor *int_in;
    ....
    device_urb_interrupt = usb_alloc_urb(0, GFP_KERNEL);
    int res = usb_register_dev(interface, &xbox_class);
    ...
    usb_fill_int_urb(device.urb_interrupt, device.udev, u
sb_revintpipe(device_ndev, device_endpoint_address), device.d
ata, device.buffer_size, proccess_key_press, NULL, device.tim
e);
    ...
}

```

file_operations

Функция, которая регистрируется в file_operations, вызывает copy_to_user

При работе с usb шиной нет аппаратных прерываний. Передача данных осуществляется хостом.

Передача: `usb_fill_int` — это как раз **interrupt** — это тип передачи.

И в usb драйвере можно планировать на выполнение tasklety или создавать очереди работ, но будут задействованы совсем другие функции, не обрабатывающие аппаратные прерывания.

То есть отложенные действия могут использоваться, но в других функциях, характерных для usb шины.

```
static struct file_operations xbox_ops =
{
    .read = xbox_read,
    .owner = THIS_MODULE,
    .open = xopen,
    .release = xrelease
};
```

`xopen` return 0;

`release` return 0;

Только в `read` `copy_to_user`:

```
static ssize_t xbox_read(struct file *file, char *buf, size_t
count, loff_t *pos)
{
    copy_to_user(buf, button, 10);
    return 10;
}
```

Plug and play

Любое устройство для того, чтобы срабатывала опция Plug And Play, при подключении, например, к USB разъему, посылается системе `report`.

Он позволяет системе распознать устройство.

Можно поменять `report` и тем самым изменить функциональность устройства.

Сокеты

Две значимые Unix: System V, BSD

Задача: разработка универсального средства взаимодействия параллельных процессов на отдельно стоящей машине и в распределенной системе

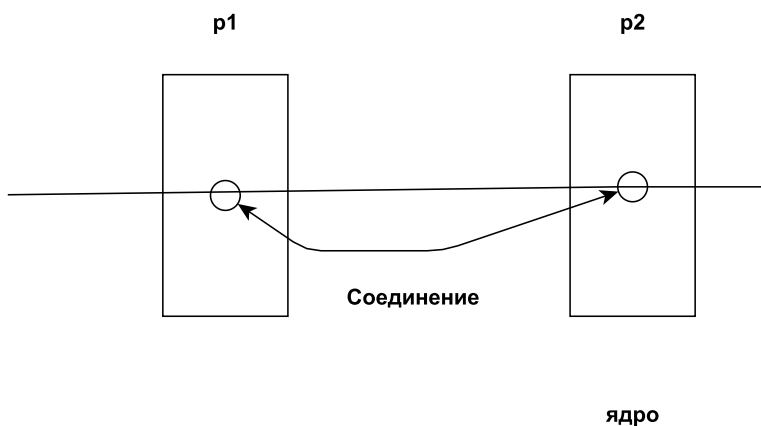
Сокет — конечная точка взаимодействия (предложены разработчиками BSD)

Модель клиент-сервер - 2 стороны взаимодействия

Комплексный адрес - номер порта и IP-адрес (абстракция)

Сокеты - важнейшее средство взаимодействия параллельных процессов

Системный вызов — функция API, определенная для режима пользователя, в результате которой происходит переход в режим ядра (а там функции ядра)



Парные сокеты

Создание парных сокеты: `socketpair()`

```
#include <sys/socket.h>
```

```
int socketpair(int domain, int type, int protocol, int sv  
[2]);
```

sv — массив из двух переменных, в которых возвращаются дескрипторы сокетов (эти два сокета неразличимы)

Особенности

- дуплексная связь
- написаны для замены программных каналов

Сходства с pipe

- Определен массив из двух файловых дескрипторов, не имеющих имен
- Так же, как и в случае неименованного канала, один из дескрипторов используется одним процессом, другой – другим.
- Как и каналы (pipe), сокеты используют простой интерфейс, основанный на «файловых» функциях read(2) и write(2) (открывая сокет, программа Unix получает дескриптор файла, благодаря которому можно работать с сокетами, используя файловые функции)

Отличия от pipe:

- Двусторонняя (дуплексная связь) как в синхронном, так и в асинхронном режиме (у пайпов симплексная связь)
- В пайпах реализовано взаимное исключение: нельзя писать, когда кто-то читает и наоборот

Код из лабораторной

```
#include <sys/types.h>
#include <sys/socket.h>
#include <stdlib.h>
#include <stdio.h>
#include <errno.h>
#include <unistd.h>

#define N 2
int main(int argc, char **argv)
{
```

```

int sockets[2];
char buf[32];
char message[14];
int pid;
if (socketpair(AF_UNIX, SOCK_DGRAM, 0, sockets) < 0)
{
    perror("socketpair() failed");
    exit(1);
}
for (int i = 0; i < N; ++i)
{
    pid = fork();
    if (pid == -1)
    {
        printf("Can't fork");
        exit(1);
    }
    else if (pid == 0)
    {
        sprintf(message, " %d", getpid());
        close(sockets[1]);
        printf("child %d write: %s\n", i, message);
        write(sockets[0], message, sizeof(message));
        sleep(1);
        read(sockets[0], buf, sizeof(buf));
        printf("child %d read: %s\n", i, buf);
        close(sockets[0]);
        exit(0);
    }
}
close(sockets[0]);
for (int i = 0; i < N; i++)
{
    read(sockets[1], buf, sizeof(buf));
    printf("parent read: %s\n", buf);
    sleep(1);
}

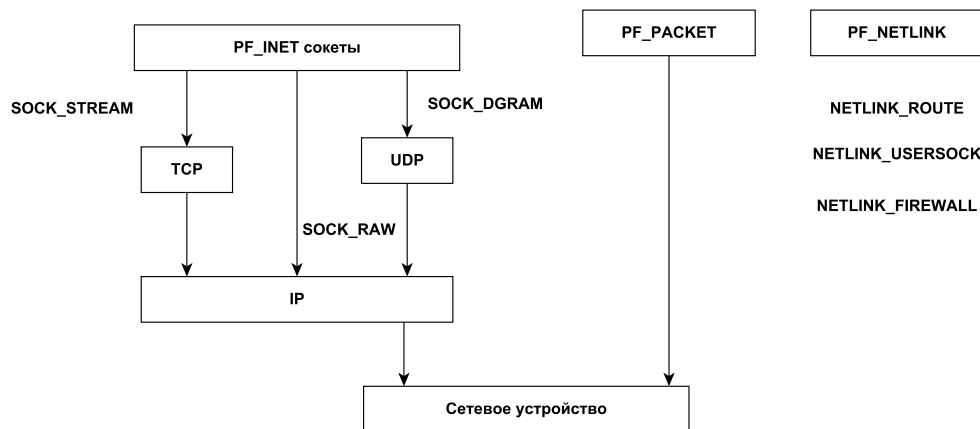
```

```

    sprintf(message, " %d", getpid());
    printf("parent write: %s\n", message);
    write(sockets[1], message, sizeof(message));
}
close(sockets[1]);
exit(0);
}

```

Сокеты BSD/Беркли



Создание сокетов: системный вызов socket()

```
int socket(int family, int type, int protocol);
```

- **family** — пространство имен (домен), AF - address family
 - **AF_UNIX** — сокеты для взаимодействия на отдельно стоящей машине в пространстве файловых имен
 - **AF_INET** — сокеты семейства протоколов TCP/IP, основанные на взаимодействии процессов в сети интернет версии 4 (IP v4)
 - **AF_INET6** — IP v6
 - **AF_IPX** — протокол IPX

- **AF_UNSPEC** — неспецифицированные
- **AF_NETLINK**
- **type** — тип сокета
 - **SOCK_STREAM** — потоковые сокеты, ориентированные на потоки. Надежное, упорядоченное, полнодуплексное логическое соединение между двумя сокетами
 - **SOCK_DGRAM** — определяет ненадежную службу без установки надежного логического соединения, где пакеты могут передаваться без сохранения порядка
 - **SOCK_RAW** — низкоуровневый сокет, обеспечивает прямой доступ к сетевому протоколу
 - **SOCK_SEQPACKET** — обеспечивает работу последовательного двустороннего канала для передачи дейтаграмм на основе соединений; дейтаграммы имеют постоянный размер; от получателя требуется за один раз прочитать целый пакет.
- **protocol** — протокол, по которому осуществляется взаимодействие между точками
 - Как правило 0 (протокол по умолчанию)
 - Для SOCK_STREAM — TCP
 - Для SOCK_DGRAM — UDP
 - Можно указать: IPPROTO_* (например, IPPROTO_TCP)
- Возвращает файловый дескриптор

▼ Как реализовано в ядре:

```
<net/socket.c>
asmlinkage long sys_socketcall(int call, unsigned long *args)
{
    unsigned long a[6];
```

```

    unsigned long a0,a1;
    int err;

    if(call<1||call>SYS_RECVMSG)
        return -EINVAL;

    /* copy_from_user should be SMP safe. */
    if (copy_from_user(a, args, nargs[call]))
        return -EFAULT;

    a0=a[0];
    a1=a[1];

    switch(call)
    {
        case SYS_SOCKET:
            err = sys_socket(a0,a1,a[2]);
            break;
        case SYS_BIND:
            err = sys_bind(a0, (struct sockaddr *)a1, a
[2]);
            break;
        case SYS_CONNECT:
            err = sys_connect(a0, (struct sockaddr *)a
1,
a[2]);
            break
        .....
        default:
            err = -EINVAL;
            break;
    }
    return err;
}

```

Сокеты адресуются (`struct sockaddr *`)

```
asm linkage long sys_socket(int family, int type, int protocol)
{
    int retval;
    struct socket *sock; // структура ядра
    ...
    retval = sock_create(family, type, protocol, &sock);
    ...
    return retval;
}
```

Структура socket

```
/**
 * struct socket - general BSD socket
 * @state: socket state (%SS_CONNECTED, etc)
 * @type: socket type (%SOCK_STREAM, etc)
 * @flags: socket flags (%SOCK_NOSPACE, etc)
 * @ops: protocol specific socket operations
 * @file: File back pointer for gc
 * @sk: internal networking protocol agnostic socket representation
 * @wq: wait queue for several uses
 */
struct socket {
    socket_state      state;

    short             type;

    unsigned long     flags;

    struct file       *file;
    struct sock       *sk;
```

```

const struct proto_ops *ops;
struct fasync_struct *fasync_list;

wait_queue_head_t wait;
};

typedef enum {
    SS_FREE = 0,           /* not allocated */
    SS_UNCONNECTED,       /* unconnected to any socket */
    /*
    SS_CONNECTING,        /* in process of connecting */
    SS_CONNECTED,         /* connected to socket */
    SS_DISCONNECTING      /* in process of disconnecting */
    */
} socket_state;

```

Сокет как файл

Дизайн сокетов Беркли следует парадигме UNIX: в идеале отобразить все объекты, к которым осуществляется доступ на чтение или запись, на файлы, чтобы работать с ними как с файлами (свести к read и write)

`struct file` в системе описывает открытый файл — дескриптор открытого файла, для которого есть запись в таблице открытых файлов

Так как сокеты рассматриваются как специальные файлы, то его копии на диске не существует, он рассматривается как открытый файл

Адресация сокетов

Определена структура `sockaddr` в самом общем виде (базовая структура)

```

struct sockaddr
{
    sa_family_t sa_family;
    char sa_data[14];
}

```

Для взаимодействия в интернете эта структура не подходит, поэтому определена структура `sockaddr_in`

```
/* Internet address. */
struct in_addr {
    __be32  s_addr;
};

#define __SOCK_SIZE__ 16      /* sizeof(struct sockaddr)
*/
/* Structure describing an Internet (IP) socket address. */
struct sockaddr_in {
    __kernel_sa_family_t  sin_family;
    __be16                sin_port;
    struct in_addr         sin_addr;
    /* padding */
    /* Pad to size of `struct sockaddr'. */
    unsigned char          __pad[__SOCK_SIZE__ - sizeof(short int) -
                                sizeof(unsigned short int) - sizeof(struct in_add
r)];
};
```

Номера порта, адреса — должны быть указаны в сетевом порядке байт

Порядок байт:

- Прямой (little endian) -- host byte order (аппаратный порядок байт) (старшие ← младшие)
- Обратный (big endian) — networking byte order (сетевой порядок байт) (младшие → старшие)

Функции для конвертации адреса:

- `htons` (host to network short - short(16))
- `htonl` (long(32))
- `ntohs`

- ntohs

Если порядок байт совпадает, то эти функции не будут выполняться

Структура для IPv6

```
struct sockaddr_in6 {  
    unsigned short int  sin6_family;  
    __be16              sin6_port;  
    __be32              sin6_flowinfo;  
    struct in6_addr      sin6_addr;  
    __u32               sin6_scope_id;  
};
```

Сетевой стек

Название стек только потому, что функции вызываются в определенном порядке.

На стороне сервера одни, на стороне клиента другие.

Взаимодействие с использованием сокетов выполняется как клиент-сервер
⇒ две стороны взаимодействия: сторона клиента и сторона сервера.

И там, и там вызывается системный вызов `socket`, чтобы создать так называемую точку соединения.

Затем сервер надо связать с адресом с помощью системного вызова `bind`, так как клиенты должны обращаться к серверу по адресу.

На стороне клиента эта функция может как вызываться, так и не вызываться.

Клиенты пытаются установить с сервером соединение. Соединение можно установить только с работающим сервером, следовательно код сервера должен выполняться.

И чтобы принимать соединение от клиентов сервер вызывает системный вызов `listen`.

Тем самым сообщает ядру, что переходит в режим пассивного ожидания соединений ⇒ основную работу по ожиданию соединений выполняет ядро.

На стороне клиента вызывается системный вызов `connect` .

Если удастся соединиться, то есть сервер принимает соединение. Работает протокол и если соединение установлено, то на стороне сервера вызывается системный вызов

`accept` .

После этого клиент посылает серверу сообщения.

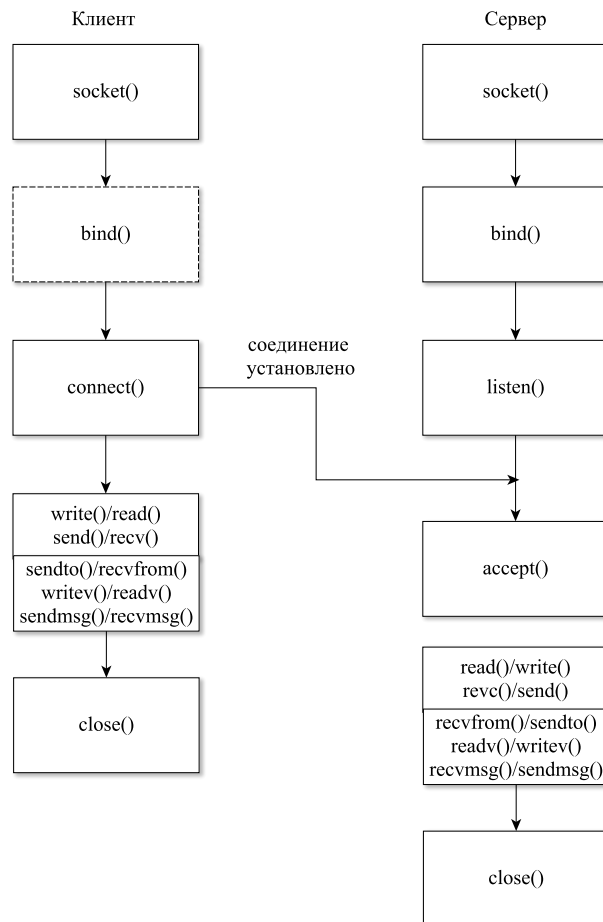
Для этого существует набор системных вызовов:

1. `read/write` — базовые системные вызовы для работы с файлами.
Реализуется парадигма UNIX — все файл, безотносительно того, какие сокеты — AF Socket или ...
2. Чтобы лучше понимать, что взаимодействие в сети идет с использованием сообщений, лучше использовать `sendto()/recvfrom()`

Когда взаимодействие заканчивается, вызывается `close()` .

Сервер может не только принимать сообщения, но и отвечать. Клиенты могут запросить у сервера самую разную информацию — вычисления, ...

Если клиент выполняет `write`, то сервер выполняет `read`, и наоборот.



СИСТЕМНЫЕ ВЫЗОВЫ

1. socket()

Создает новый сокет, передается три параметра, в частности протокол UDP или TCP.

2. `int bind(int sockfd, struct sockaddr *addr, int addrlen);`

Созданный сокет связывается с заданным адресом с помощью системного вызова `bind()`

Связывание сокета с адресом нужно в приложении сервера. Для сокетов семейства AF — это адрес, который состоит из IP адреса локальной сетевой системы и номера порта.

Клиенты могут не вызывать `bind()`, поскольку их точный адрес зачастую роли не играет.

В таком случае адрес им назначается автоматически.

```
struct sockaddr {
    sa_family_t sa_family;
    char sa_data[14];
};
```

Структура `sockaddr` реализована в самом общем виде — так сделано для универсализации взаимодействия при использовании сокетов.

3. `int listen(int sockfd, int backlog);`

Системный вызов `listen` используется сервером для информирования ядра о том, что сокет должен принять соединение. Фактически этим системным вызовом сервер информирует ядро, что он переходит в режим пассивного ожидания.

Организовывать бесконечную очередь нельзя $\Rightarrow$ `backlog`.

4. `int connect(int sockfd, struct sockaddr *serv_addr, int addrlen);`

Системный вызов `connect` выполняется на стороне клиента для установления активного соединения.

`struct sockaddr *serv_addr` — адрес сервера!

5. `int accept(int sockfd, void *addr, int *addrlen);`

Системный вызов `accept` выполняется на стороне сервера для того, что принять соединение. Сначала по протоколу соединения должно быть подтверждено, что соединение действительно установлено, и только после этого вызывается `accept`.

Системный вызов `accept` создает копию исходного сокета. При этом исходный сокет остается в состоянии `listen`, а созданная копия находится в состоянии соединения.

До этого момента приложение сервера находится в состоянии блокировки.

Такая огромная работа ядра позволяет серверу ожидать новых соединений, обеспечивает возможность работы с большим числом клиентов.

ЛР сервер только получает сообщения

Сервер:

```
#include <sys/types.h>
#include <sys/socket.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>

#define SOCK_NAME "sock.srv"

int sock;

void sig_handler(int signal)
{
    close(sock);
    unlink(SOCK_NAME);
    printf("Stop server\n");
    exit(0);
}

int main()
{
    struct sockaddr srvr_name;
```

```

char buf[16];

sock = socket(AF_UNIX, SOCK_DGRAM, 0);

if (sock < 0)
{
    perror("socket failed");
    return EXIT_FAILURE;
}

srvr_name.sa_family = AF_UNIX;
strcpy(srvr_name.sa_data, SOCK_NAME);

if (bind(sock, &srvr_name, strlen(srvr_name.sa_data) + si
zeof(srvr_name.sa_family)) < 0)
{
    perror("bind");
    return EXIT_FAILURE;
}

if (signal(SIGINT, sig_handler) == (void *)-1)
{
    perror("signal");
    return EXIT_FAILURE;
}

while (1)
{
    int bytes = recvfrom(sock, buf, sizeof(buf), 0, NULL,
NULL);

    if (bytes == -1)
    {
        printf("%s", buf);
        perror("recvfrom");
        exit(1);
    }
}

```

```

    }

    buf[bytes] = '\0';
    printf("%s", buf);
}
}

```

Клиент:

```

#include <sys/types.h>
#include <sys/socket.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

#define SOCK_SRVR "sock.srv"

int main()
{
    int sock;
    struct sockaddr srvr_name;
    char buf[16];

    sock = socket(AF_UNIX, SOCK_DGRAM, 0);

    if (sock < 0)
    {
        perror("socket failed");
        return EXIT_FAILURE;
    }

    srvr_name.sa_family = AF_UNIX;
    strcpy(srvr_name.sa_data, SOCK_SRVR);
    sprintf(buf, "pid: %d\n", getpid());

```

```

    if (sendto(sock, buf, strlen(buf), 0, &srvr_name, strlen
(srvr_name.sa_data) + sizeof(srvr_name.sa_family)) == -1)
    {
        perror("sendto");
        return EXIT_FAILURE;
    }

    close(sock);

    return EXIT_SUCCESS;
}

```

ЛР сервер получает и посылает сообщения

Сервер:

```

#include <sys/types.h>
#include <sys/socket.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>

#define SOCK_NAME "sock.srv"

int sock;

void sig_handler(int signal)
{
    close(sock);
    unlink(SOCK_NAME);
    printf("Stop server\n");
    exit(0);
}

```

```

int main()
{
    struct sockaddr srvr_name, clnt_name;
    int clnt_size = sizeof(clnt_name);
    char buf[64];
    char copybuf[128];

    sock = socket(AF_UNIX, SOCK_DGRAM, 0);

    if (sock < 0)
    {
        perror("socket failed");
        return EXIT_FAILURE;
    }

    srvr_name.sa_family = AF_UNIX;
    strcpy(srvr_name.sa_data, SOCK_NAME);

    if (bind(sock, &srvr_name, sizeof(srvr_name)) < 0)
    {
        perror("bind");
        return EXIT_FAILURE;
    }

    if (signal(SIGINT, sig_handler) == (void *)-1)
    {
        perror("signal");
        close(sock);
        unlink(SOCK_NAME);
        return EXIT_FAILURE;
    }

    while (1)
    {
        sleep(3);
        int bytes = recvfrom(sock, buf, sizeof(buf), 0, &clnt

```

```

_name, &clnt_size);

    if (bytes == -1)
    {
        printf("%s", buf);
        perror("recvfrom");
        close(sock);
        unlink(SOCK_NAME);
        return EXIT_FAILURE;
    }

    buf[bytes] = '\0';
    printf("%s", buf);
    sprintf(copybuf, "answer to client with %s\n", buf);

    if (sendto(sock, copybuf, strlen(copybuf) + 1, 0, &clnt_name, clnt_size) == -1)
    {
        perror("sendto");
        close(sock);
        unlink(SOCK_NAME);
        return EXIT_FAILURE;
    }
}
}

```

Клиент:

```

#include <sys/types.h>
#include <sys/socket.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

```

```

#define SOCK_SRVR "sock.srv"
#define SOCK_CLNT "sock.clnt"

int main()
{
    int sock;
    struct sockaddr srvr_name, clnt_name;
    int srvr_size = sizeof(srvr_name);
    char buf[64];

    sock = socket(AF_UNIX, SOCK_DGRAM, 0);

    if (sock < 0)
    {
        perror("socket failed");
        return EXIT_FAILURE;
    }

    srvr_name.sa_family = AF_UNIX;
    strcpy(srvr_name.sa_data, SOCK_SRVR);
    sprintf(buf, "message from client with pid: %d\n", getpid
());

    clnt_name.sa_family = AF_UNIX;
    sprintf(clnt_name.sa_data, "%d.clnt", getpid());

    if (bind(sock, &clnt_name, sizeof(clnt_name)) < 0)
    {
        perror("bind");
        return EXIT_FAILURE;
    }

    if (sendto(sock, buf, strlen(buf), 0, &srvr_name, sizeof
(srvr_name)) == -1)
    {
        perror("sendto");
    }
}

```

```

        close(sock);
        unlink(clnt_name.sa_data);
        return EXIT_FAILURE;
    }

    int bytes = recvfrom(sock, buf, sizeof(buf), 0, NULL, NULL);

    if (bytes == -1)
    {
        perror("recvfrom");
        close(sock);
        unlink(clnt_name.sa_data);
        return EXIT_FAILURE;
    }

    buf[bytes] = '\0';
    printf("%s", buf);

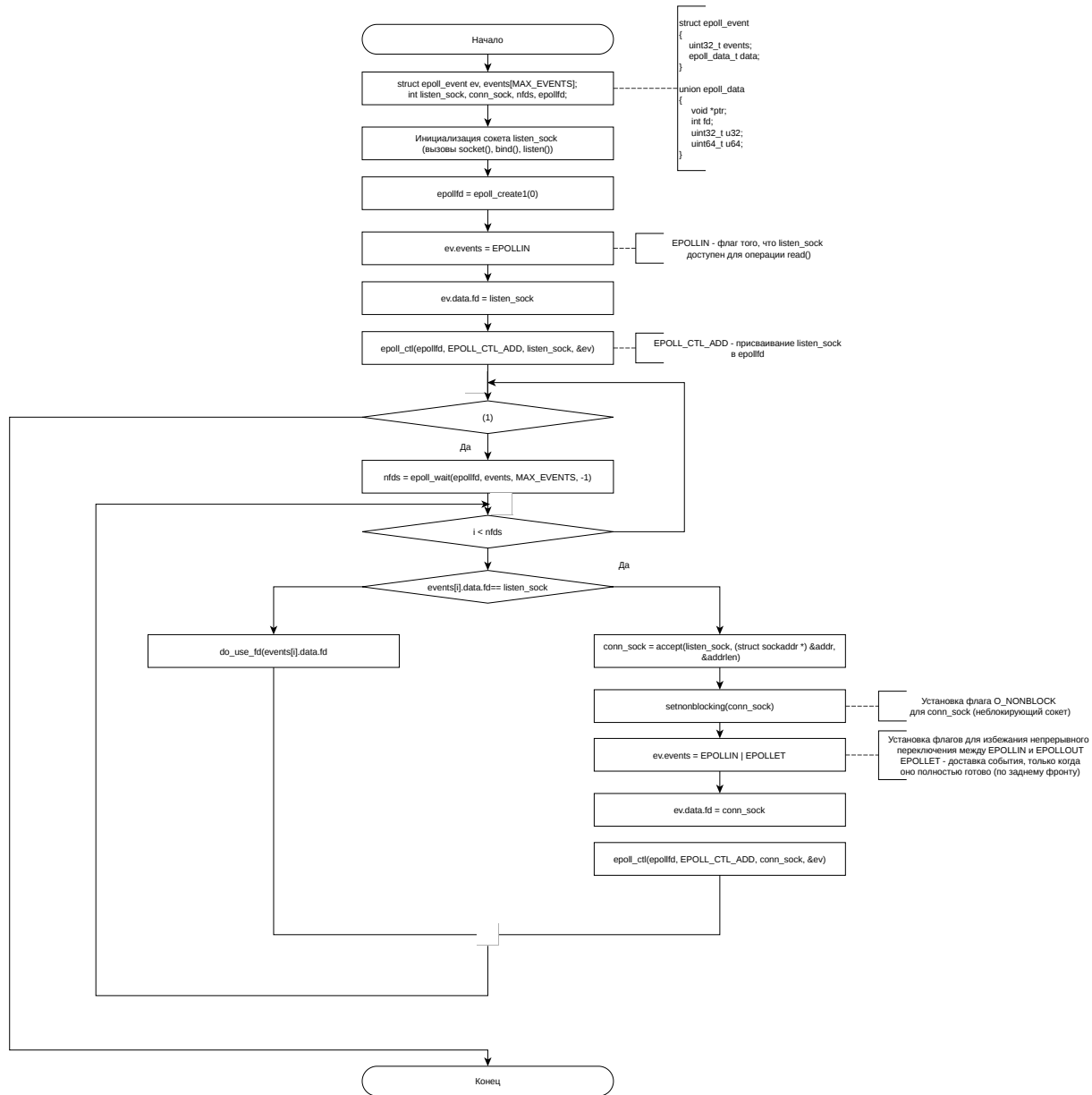
    close(sock);
    unlink(clnt_name.sa_data);

    return EXIT_SUCCESS;
}

```

Процессы взаимодействуют через пространство файловых имен

ЛР еpoll, сокеты AF_INET



Сервер:

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <signal.h>
#include <errno.h>
  
```

```

#include <fcntl.h>
#include <netinet/in.h>
#include <sys/epoll.h>
#include <netdb.h>
#include <string.h>

#define PORT 6423
#define MAX_EVENTS 10
#define BUF_SIZE 64
#define SNT_BUF 16

int listen_sock;

void sig_handler(int signal)
{
    close(listen_sock);
    printf("Stop server\n");
    exit(EXIT_SUCCESS);
}

int main()
{
    struct epoll_event ev, events[MAX_EVENTS];
    int conn_sock, nfds, epollfd;
    struct sockaddr_in serv_addr;
    char sent_msg[BUF_SIZE], rcv_msg[SNT_BUF];
    struct hostent *server;
    struct sigaction sa;

    listen_sock = socket(AF_INET, SOCK_STREAM, 0);

    if (listen_sock < 0)
    {
        perror("socket() failed\n");
        exit(EXIT_FAILURE);
    }

```

```

if (fcntl(listen_sock, F_SETFL, O_NONBLOCK) < 0)
{
    perror("fcntl() failed\n");
    close(listen_sock);
    exit(EXIT_FAILURE);
}

sa.sa_handler = &sig_handler;

if (sigaction(SIGINT, &sa, NULL) == -1)
{
    perror("signal");
    close(listen_sock);
    exit(EXIT_FAILURE);
}

server = gethostbyname("127.0.0.1");
if (server == NULL)
{
    printf("Host not found\n");
    return EXIT_FAILURE;
}

serv_addr.sin_family = AF_INET;
strncpy((char *)&serv_addr.sin_addr.s_addr, (char *)server->h_addr_list[0], server->h_length);

serv_addr.sin_port = htons(PORT);

if (bind(listen_sock, (struct sockaddr *) &serv_addr, sizeof(serv_addr)) < 0)
{
    perror("bind() failed\n");
    close(listen_sock);
}

```

```

        exit(EXIT_FAILURE);
    }

    if (listen(listen_sock, MAX_EVENTS) < 0)
    {
        perror("listen() failed\n");
        close(listen_sock);
        exit(EXIT_FAILURE);
    }

    epollfd = epoll_create1(0);

    if (epollfd == -1)
    {
        perror("epoll_create1");
        close(listen_sock);
        exit(EXIT_FAILURE);
    }

    ev.events = EPOLLIN;
    ev.data.fd = listen_sock;

    if (epoll_ctl(epollfd, EPOLL_CTL_ADD, listen_sock, &ev) =
= -1)
    {
        perror("epoll_ctl: listen_sock\n");
        close(listen_sock);
        exit(EXIT_FAILURE);
    }
    for (;;)
    {
        nfds = epoll_wait(epollfd, events, MAX_EVENTS, -1);
        if (nfds == -1) {
            perror("epoll_wait\n");
            close(listen_sock);
            exit(EXIT_FAILURE);
        }
    }

```

```

    }
    for (int i = 0; i < nfd; i++) {
        if (events[i].data.fd == listen_sock) {
            conn_sock = accept(listen_sock, NULL, NULL);
            if (conn_sock == -1)
            {
                perror("accept() failed\n");
                close(listen_sock);
                exit(EXIT_FAILURE);
            }
            ev.events = EPOLLIN | EPOLLET;
            ev.data.fd = conn_sock;
            if (epoll_ctl(epollfd, EPOLL_CTL_ADD, conn_sock, &ev) == -1) {
                perror("epoll_ctl: conn_sock\n");
                close(listen_sock);
                exit(EXIT_FAILURE);
            }
        }
        else
        {
            int conn = events[i].data.fd;
            sent_msg[BUF_SIZE - 1] = 0;
            rcv_msg[SNT_BUF - 1] = 0;

            int rc = read(conn, rcv_msg, sizeof(rcv_msg));
            if (rc == -1)
            {
                perror("read() failed\n");
                close(conn);
                exit(EXIT_FAILURE);
            }
            if (rc > 0)
            {
                printf("received client pid: %s\n", rcv_m

```

```

sg);
        sprintf(sent_msg, "server %d answer to client with pid %s\n", getpid(), rcv_msg);

        if (send(conn, sent_msg, sizeof(sent_msg), 0) == -1)
        {
            perror("send() failed\n");
            exit(EXIT_FAILURE);
        }

        printf("send to client: %s\n", sent_msg);
    }

    close(conn);
}
}
close(listen_sock);
}

```

Клиент:

```

#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <unistd.h>
#include <netdb.h>
#include <string.h>

#define PORT 6423
#define BUF_SIZE 64

int main()
{

```

```

int sock;
struct sockaddr_in serv_addr;
char sent_msg[BUF_SIZE], rcv_msg[BUF_SIZE];
struct hostent *server;

if ((sock = socket(AF_INET, SOCK_STREAM, 0)) == -1)
{
    perror("socket() failed\n");
    exit(EXIT_FAILURE);
}

server = gethostbyname("127.0.0.1");
if (server == NULL)
{
    printf("Host not found\n");
    return EXIT_FAILURE;
}

serv_addr.sin_family = AF_INET;
strncpy((char *)&serv_addr.sin_addr.s_addr, (char *) server->h_addr_list[0], server->h_length);
serv_addr.sin_port = htons(PORT);

if (connect(sock, (const struct sockaddr *) &serv_addr, sizeof(serv_addr)) == -1)
{
    perror("connect() failed\n");
    close(sock);
    exit(EXIT_FAILURE);
}

sent_msg[BUF_SIZE - 1] = 0;
rcv_msg[BUF_SIZE - 1] = 0;

sprintf(sent_msg, "%d", getpid());

```

```
if (send(sock, sent_msg, sizeof(sent_msg), 0) == -1)
{
    perror("send() failed\n");
    close(sock);
    exit(EXIT_FAILURE);
}

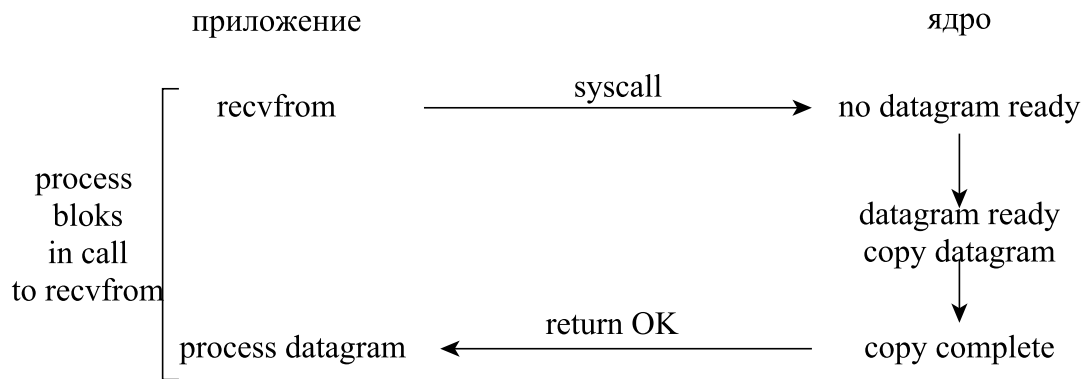
printf("client sent message to server: %s\n", sent_msg);

if (read(sock, rcv_msg, sizeof(rcv_msg)) == -1)
{
    perror("read() failed\n");
    close(sock);
    exit(EXIT_FAILURE);
}

printf("%s\n", rcv_msg);
close(sock);
exit(EXIT_SUCCESS);
}
```

Модели ввода/вывода

Блокирующий [синхронный] ввод/вывод приложения



Приложение вызывает запрос на ввод данных.

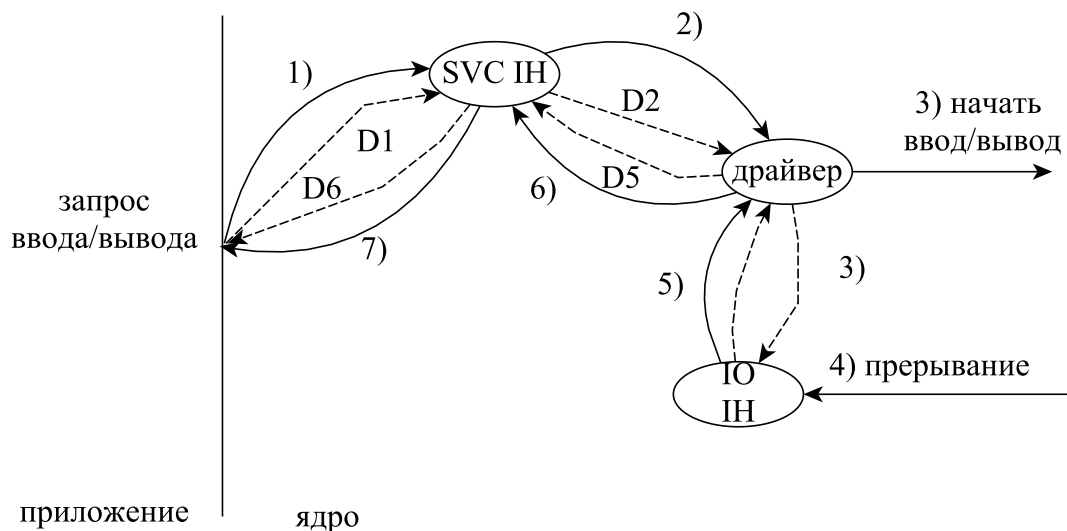
Для общности используется `recvfrom`.

В результате такого системного вызова процесс будет блокирован (на диаграмме — `no datagram ready`). Так как обращаемся за данными, например, к файлу — он на внешнем устройстве — получить сразу по запросу приложение данные не может. Обращение к внешним устройствам — длительное действие.

После того как данные получены — необходимо ... полный объем полученной информации.

Когда устройство готово отдать данные, вызывается обработчик аппаратных прерываний, основной задачей которого является скопировать данные из буфера устройства в буфер ядра.

Затем из буфера ядра они должны быть скопированы в буфер приложения. Все это время процесс блокирован.



SVC — supervisor call

IH — interrupt handler

Dn — данные, которые передаются при вызове

Любой системный вызов передает ядру какие-то данные. Система переходит в режим ядра, в результате выполнения функций ядра будет выполнен драйвер внешнего устройства. Драйвер — многовходовая программа, будет вызвана соответствующая функция драйвера, которая вызовет контроллер внешнего устройства, который переведет данные в формат, понятный устройству.

3) — начать ввод/вывод — процесс отключен от управления внешним устройством. Переходит на выполнение другой работы. Когда I/O завершится, возникает **аппаратное прерывание — interrupt**.

Прерывание от внешнего устройства формируется внешним устройством по завершению операции I/O, и будет вызван **interrupt handler**.

Разрыв обусловлен архитектурой ЭВМ — распараллеливание функций. В 3 поколении ЭВМ распараллелили функции — функция управления внешними

устройства была передана каналам (канальная архитектура, шинная архитектура).

Каналы, шины — программное управление устройства.

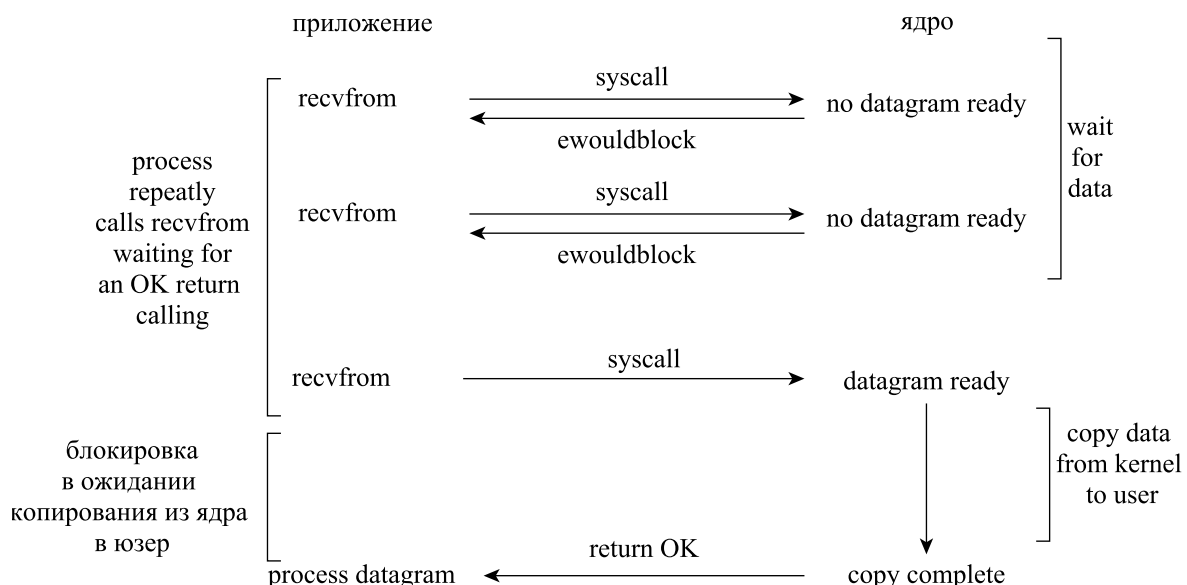
Контроллер в результате работы драйвера получает команду. На этом завершается управление процессором. Выполняет команду контроллер, управляющий внешним устройством.

Процессор освобожден от задачи управления внешним устройством.

Polling — опрос. Неблокирующий синхронный ВВОД/ВЫВОД.

Исторически первым появился именно опрос.

В первых системах процессор должен был опрашивать внешние устройства на предмет готовности.



В результате опроса действия должны выполняться в цикле.

Все модели рассматриваются с точки зрения сокетов.

При работе с сокетами можно установить неблокирующий режим, следовательно можно его установить и приложение будет опрашивать ядро, пока оно не вернет данные для считывания.

Multiplexing — уплотнение. Мультиплексирование ввода/вывода — асинхронный блокирующий

Мультиплексор и коммутатор — устройства, принимающие информацию по нескольким каналам для выдачи ее по одному выходному каналу.

Определение из оксфордского словаря.

Мультиплексирование, уплотнение — процесс совмещения нескольких сообщений, передаваемых одновременно в одной логической или физической среде.

Мультиплексирование бывает двух видов: временное и частотное.

На сокетах — временное.

Мультиплексирование является альтернативой многопоточности/многопроцессорности.

В Unix ничего не стоит создать процесс. Процессы сопоставляются с потоками.

В Windows потоки затратны: в общем то цена создания потока неизвестна.

Мультиплексирование позволяет избежать затрат ресурсов на создание потоков/процессов.

Потоки разделяют код/данные, но для каждого потока создается собственный стек и анонимное отображение. В адресном пространстве процесса создаются регионы, и это большое количество страниц.

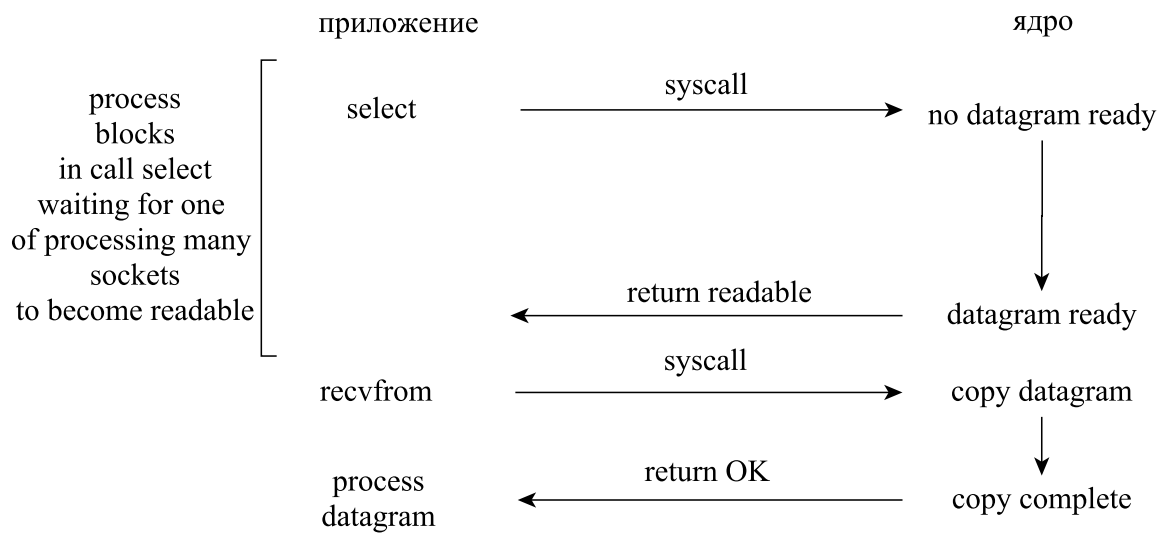
Мультиплексоры:

`select poll`

`pselect epoll`

`epoll` — современная версия `poll`.

Мультиплексирование выполняет сервер, так как сервер принимает большое количество соединений.



Важно принять соединение наиболее эффективным образом.

Когда данные готовы — возвращает информация, что данные готовы.

Все это время на

`select` процесс блокирован. Ожидая какого-либо соединения сокета с клиентом.

Когда соединение установлено — дальше должен быть вызван системный вызов

`accept`.

В этой модели это не показано.

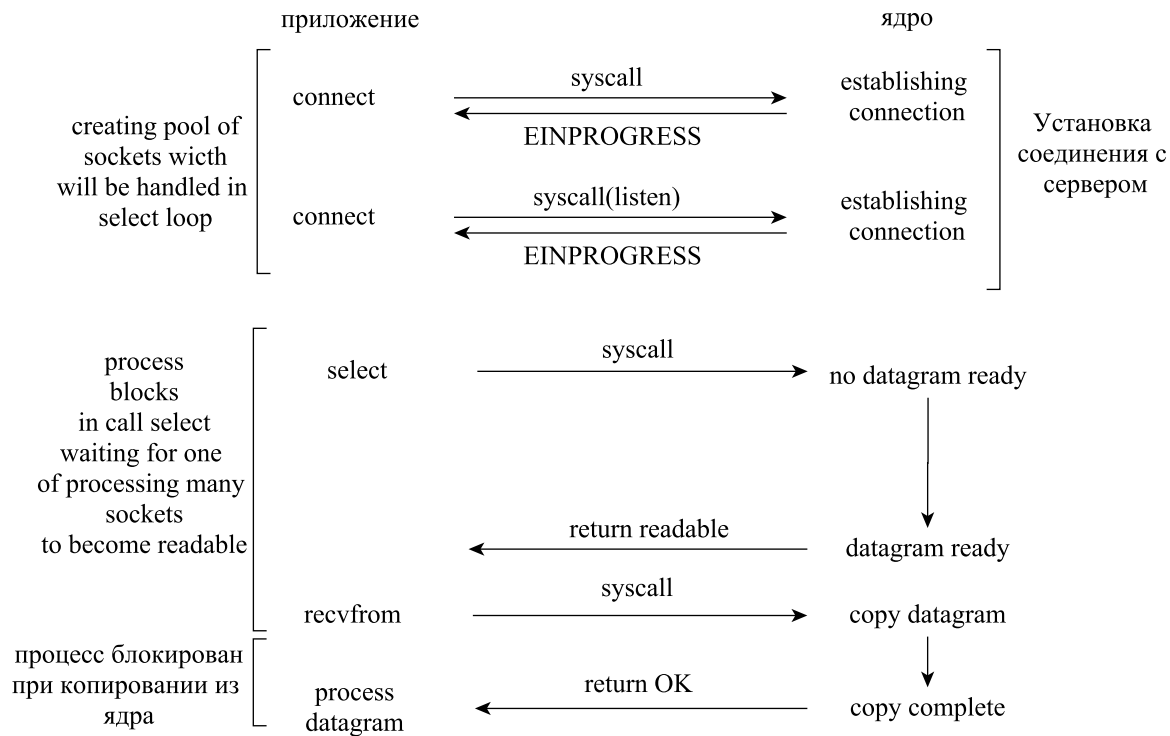
Фактически

`select` опрашивает очередь соединений. Если какой-то клиент (любой из возможных) — с этим клиентов дальше устанавливается соединение системным вызовом `accept`.

Затем процесс блокирован при копировании в буфер приложения.

Допустим к серверу выстроилось в очередь 10 клиентов. Все клиенты асинхронные, выполняются с собственной скоростью. В результате может получиться так, что 4 соединения готовы, но сервер ждет первого. Пока первый не обслужен, не будет обслужен второй.

А мультиплексирование — как возникает первое доступное, обслуживает его.



Клиенты ждут установления соединения — establishing connection.

Клиентов много — каждый выполняет connect.

Дви... — соединение, ориентированное на tcp — соединение требует времени, так как tcp был написан, чтобы обеспечить надежное соединение — тройное рукопожатие (клиент посылает сообщение запрос, сервер посылает ответ, на который сервер отправляет сообщение подтверждение, после чего сервер выполняет ассепт ⇒ в EINPROGRESS).

На диаграмме не разделены стороны клиента и сервера. В начале показано клиентское приложение, серверное вызывает select.

select, для которого на сервере определен массив дескрипторов сокетов, которые будут опрашивать этот select.

В ядре эти соединения формализуются как сокеты raw, и их может опрашивать select, это делает ядро.

После того, как ядро проинформировало приложение, что соединение принято, вызывается ассепт.

То есть, когда возникает хотя бы одно соединение, вызывается ассепт.

Почему datagram?

Авторы скорее всего подчеркивают, что это завершенное, complete сообщение, имеющее определенный формат.

Альтернатива мультиплексированию

Если нет мультиплексора, то чтобы обеспечить несколько подключения для обслуживания каждого нового — нужно создавать или новый процесс или новый поток, что для Unix практически одно и то же.

```
int main()
{
    int listenfd, connfd;
    pid_t childpid;
    socklen_t clilen;
    struct sockaddr_in cliaddr, servaddr;
    .... // вызов listen
    for(;;)
    {
        clilen = sizeof(cliaddr);
        connfd = accept(listenfd, (struct sockaddr *)
&cliaddr, &clilen);
        if((childpid = fork()) == 0)
        {
            // child process
            close(listenfd);
            str_echo(connfd); // функция обработк
и
            exit(0);
        }
        close(connfd);
    }
    return 0;
}
```

Вызывается системный вызов `socket`, инициализируются поля структуры `sock_addr`, ..., вызывается `bind` для связи сокета `listenfd` — исходного, с адресом сервера.

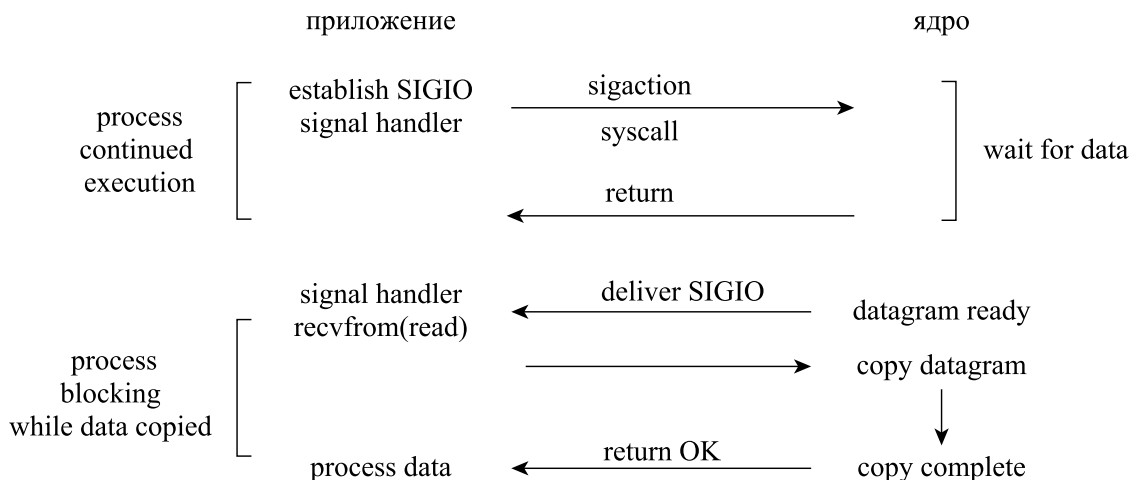
Сервер переходит в пассивное ожидание соединений.

После установки соединения создается `child`, в ней закрываем `listenfd` и вызываем функцию обработчик.

4 модель. Ввод/вывод, управляемый сигналом. Асинхронный/блокирующий. Signal driven IO

В системе должен быть специальный сигнал — `SIGIO`.

Этот сигнал должен быть установлен.



`sigaction` — установка сигнала `SIGIO`

Этот сигнал должен нам сообщать о каком-то событии. Нас интересует I/O.

Асинхронно — процесс не блокируется, процесс продолжает выполнение.

Он блокируется только для того, чтобы дождаться получения данных.

Похоже на прерывание, но смысл другой: приложение не блокируется, продолжает выполняться. Когда сигнал получен, то есть доставлен — вызывается `recvfrom`, данные готовы и копируются в юзер.

Но: основная проблем асинхронного ввода/вывода: нужны данные, чтобы их обрабатывать.

То есть если удастся в процессе выделить действия, которые можем выполнить до получения данных — то не блокируется, продолжается выполнение, и получим сигнал: ... — главное совместить эти действия.

В результате устраняется блокировка.

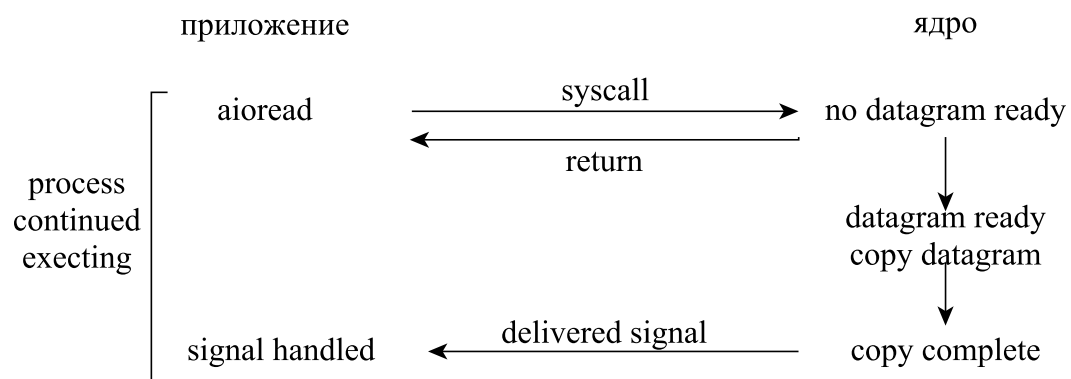
Блокировки — зло — замедляет выполнение программы.

В первую очередь блокировки взаимного исключения, но и I/O тоже.

Асинхронный ввод/вывод

Для асинхронного ввода вывода в системе должны существовать специальные системные вызовы:

`aioread()`, `aiowriter()` — специальные асинхронные вызовы в Unix.



Сразу осуществляется возврат, если не удалось установить — ошибка, иначе успех и выполняется асинхронный ввод/вывод.

В конце: сигнал будет обрабатываться signal handler — определенный для асинхронного ввода вывода.

Нет блокировки вообще. Процесс продолжает работать до тех пор, пока не получит сигнал для обработки.

Проблема та же (что и в 4 модели):

главное уметь найти возможность выполнения действий, ожидая данных.

Время идет, но процесс не блокируется ядром, а продолжает выполнение.

Ядро процесс не блокирует в отличие от 1 модели.

Классификация

	blocking	non_blocking
синхронный	read/write Блокирующий ввод вывод	read/write (O_NONBLOCK) опрос(polling)
асинхронный	Мультиплексирование I/O multiplexing + SIGIO	AIO

Загружаемый модуль ядра

Unix/Linux имеют монолитное ядро, предоставляют возможность включать расширения функциональности — для этого загружаемые модули ядра

Инициализация модуля

Макрос `MODULE_INIT` служит для регистрации функции инициализации модуля. Функция, переданная в качестве фактического параметра, будет вызываться при загрузке модуля в ядро.

Передаваемая функция должна соответствовать прототипу:

```
int func_init(void);
```

Функция возвращает значение типа `int`. Если функция инициализации завершилась успешно, то возвращается значение ноль. В случае ошибки возвращается ненулевое значение.

Как правило, функция инициализации предназначена для запроса ресурсов и выделения памяти под структуры данных и т.п.. Так как функция инициализации редко вызывается за пределами модуля, ее обычно не нужно экспортировать и можно объявить с ключевым словом `static`.

Выгрузка модуля

Макрос `MODULE_EXIT` служит для регистрации функции, которая вызывается при удалении модуля из ядра. Обычно эта функция выполняет задачу освобождения ресурсов. После завершения функции модуль выгружается.

Функция должна соответствовать прототипу:

```
void func_exit(void);
```

Функцию завершения, как и инициализации, можно объявить как `static`.

Пример загружаемого модуля

Структура загружаемого модуля ядра:

```
static int __init md_init(void)
{
    printk("module md loaded\n");
    return 0;
}

static void __exit md_exit(void)
{
    printk("module unloaded\n");
}

MODULE_INIT(md_init);
MODULE_EXIT(md_exit);
```

Двойное подчеркивание указывает ядру на необходимость пристального внимания к данному коду: то есть это предупреждение ядра.

Макрос `__init` используются для обозначения функций, встраиваемых в ядро, или инициализированных данных, которые ядро воспринимает как указание о том, что функция используется только на этапе инициализации и освобождает использованные ресурсы после. Это позволяет экономить ресурсы системы.

После определения `md_init` и `md_exit` передаются макросам `MODULE_INIT` и `MODULE_EXIT` соответственно.

Работа с загружаемым модулем

Модули ядра должны компилироваться специальным образом.

После этого модуль загружается в ядро командой

```
insmod .
```

`insmod` загружает модуль и переходит на `init` — начинает выполнять.

`insmod` загружает модуль в ядро из конкретного файла, если модуль зависит от других модулей, которые не загружены в ядро, то выдает ошибку и не загружает модуль. Только суперпользователь может загрузить модуль в ядро.

Пример использования: `sudo insmod ./md.ko`

Выгружается командой

```
rmmod .
```

Посмотреть список загруженных модулей можно командой `lsmod`.

Также можно посмотреть в `/proc/modules`

Для загрузки модуля должны быть права `superuser`.

`printk`, `k` — `kernel` — эта функция пишет в `syslog`.

Обычно в начале указывается константа, определяющая уровень приоритета сообщения: `KERN_INFO`, `KERN_DEBUG`, `KERN_ERR`,

После константы не ставится запятая!

```
printk(KERN_INFO "var=%d\n", var);
```

В ядре не могут использоваться функции стандартных библиотек Си. В ядре могут использоваться только ядреные функции, то есть функции, определенные в ядре.

Нельзя непосредственно использовать стандартные функции в ядре и функции ядра в юзере.

Например, нельзя read/write информацию в ядро: это сделано для защиты ядра.

Для передачи информации из ядра и в ядро используются специальные библиотеки и функции.

В начале каждого загружаемого модуля обычно указываются:

```
#include <linux/init.h>
#include <linux/fs_struct.h>

MODULE_AUTHOR("Name");
MODULE_LICENSE("GPL");
```

Макрос `MODULE_LICENSE` является обязательным!

Макрос `MODULE_LICENSE` используется для того, чтобы сообщить ядру, под какой лицензией распространяется исходный код модуля, что влияет на то, к функциям и переменным (символы ядра) он может получить доступ в ядре.

Лр

```
struct task_struct {
    struct list_head    tasks;

    struct fs_struct *fs;
    struct files_struct *files;
```

```
};
```

Кольцевой список $\Rightarrow$ есть ссылка на начало.

`&init_task` — указатель на первый `task_struct` в кольцевом списке

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/proc_fs.h>
#include <linux/fs_struct.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Aleksey Tolmachev");

static int __init md_init(void)
{
    struct task_struct *task = &init_task;
    do
    {
        printk(KERN_INFO "Task: pid: %d, comm: %-15s, ppid: %d, parent comm: %-15s, state: %ld, flags: %x, prio: %d, policy: %d, root: %s, pwd: %s\n",
            task->pid,
            task->comm,
            task->parent->pid,
            task->parent->comm,
            task->state,
            task->flags,
            task->prio,
            task->policy,
            task->fs->root.dentry->d_iname,
            task->fs->pwd.dentry->d_name.name
        );
    } while ((task = next_task(task)) != &init_task);
}
```

```

    printk(KERN_INFO "Current: pid: %d, comm: %-15s, ppid: %d, parent comm: %-15s, state: %ld, flags: %x, prio: %d, policy: %d, root: %s, pwd: %s\n",
        current->pid,
        current->comm,
        current->parent->pid,
        current->parent->comm,
        current->state,
        current->flags,
        current->prio,
        current->policy,
        current->fs->root.dentry->d_iname,
        current->fs->pwd.dentry->d_name.name
    );

    return 0;
}

static void __exit md_exit(void)
{
    printk(KERN_INFO "Module completed\n");
}

module_init(md_init);
module_exit(md_exit);

```

policy: 1, 2, 0 — алгоритм планирования

1 — FIFO (нет переключения, выполняется пока не закончится)

2 — RR (переключение по кванту)

0 — по умолчанию

По умолчанию самый справедливый.

Справедливость: чтобы в итоге все процессы получили процессорное время и по возможности в равных количествах.

Выйдя из блокировки в ожидании ввода/вывода процесс должен получить запрошенные данные, для этого они должны быть скопированы из ядра ⇒ процесс должен получить процессорное время ⇒ его приоритет повышается.

Учитывается время простоя в очереди готовых процессов и полученное процессорное время.

Любой процесс относится к определенной файловой системе, у нее есть root. У root есть имя. Его можно получить: root→fs→root.dentry→d_name.name

Поле fs содержит информацию о файловой системе, которая в Linux содержит 3 части

- root — каталог и точка монтирования
- альтернативный root и точка монтирования
- текущий рабочий каталог

Приставка current

Используем приставку current, чтобы вывести ту же информацию для текущего процесса.

Задача взаимодействия загружаемых модулей

Рассмотрим эту возможность на примере 3-х модулей ядра: md1, md2 и md3.

```
extern char *md1_data;

extern char *md1_proc(void);

extern char *md1_noexport(void);
```

```
#include <linux/init.h>
#include <linux/module.h>

#include "md.h"
```

```

MODULE_LICENSE( "GPL" );
MODULE_AUTHOR( "Oleg Tsiliuric <olej@front.ru>" );

char *md1_data = "aaaa";

extern char *md1_proc(void) {
    return md1_data;
}

static char *md1_local(void) {
    return md1_data;
}

extern char *md1_noexport(void) {
    return md1_data;
}

EXPORT_SYMBOL( md1_data );
EXPORT_SYMBOL( md1_proc );

static int __init md_init(void) {
    printk("+ module md1 start!\n");
    return 0;
}

static void __exit md_exit(void) {
    printk("+ module md1 unloaded!\n");
}

module_init( md_init );
module_exit( md_exit );

```

```

#include <linux/init.h>
#include <linux/module.h>

```

```

#include "md.h"

MODULE_LICENSE( "GPL" );
MODULE_AUTHOR( "Oleg Tsiliuric <olej@front.ru>" );

static int __init md_init(void) {
    printk("+ module md2 start!\n");
    printk("+ data string exported from md1 : %s\n", md1_data);
    printk("+ string returned md1_proc() is : %s\n", md1_proc());
    // printk("+ string returned md1_proc() is : %s\n", md1_local()); (1)
    // printk("+ string returned md1_proc() is : %s\n", md1_noexport()); (2)
    return 0;
}

static void __exit md_exit(void) {
    printk("+ module md2 unloaded!\n");
}

module_init( md_init );
module_exit( md_exit );

```

md1.c и md2.c — в них нет ошибок.

Показано, что модули могут взаимодействовать.

Возникает вопрос: что лежит в основе взаимодействия модулей?

Взаимодействие модулей заключается в том, что один модуль может использовать данные, объявленные и описанные в другом модуле.

Для этого в модуле, который экспортирует данные, они должны быть переданы в макрос `EXPORT_SYMBOL`. В другом модуле подключается `md.h`, в котором они обозначены как `extern`.

Экспортируемыми символами называются данные и функции, которыми могут пользоваться другие модули ядра. При загрузке модули динамически компонуются с ядром, в коде модулей могут вызываться только те функции ядра, которые явно экспортируются для использования.

В ядре экспортирование осуществляется с помощью специального макроса `EXPORT_SYMBOL()`. Функции, которые экспортируются, доступны для использования модулям, остальные функции не могут быть вызваны из модулей. Для экспортируемых данных правило аналогично.

```
#include <linux/init.h>
#include <linux/module.h>

#include "md.h"

MODULE_LICENSE( "GPL" );
MODULE_AUTHOR( "Oleg Tsiliuric <olej@front.ru>" );

static int __init md_init(void) {
    printk("+ module md3 start!\n");
    printk("+ data string exported from md1 : %s\n", md1_data);
    printk("+ string returned md1_proc() is : %s\n", md1_proc());
    return -1;
}

module_init( md_init );
```

Видим, что функция `init` возвращает `-1`, то есть ошибку.

Такой модуль в ядро загружен не будет — будет выдано соответствующее сообщение об ошибке.

Общие точки входа ядра:

Системные вызовы, аппаратные прерывания, исключения (все ошибки обрабатываются ядром).

Любой загружаемый модуль ядра — многовходовая программа.

Система предоставляет возможность писать драйверы в виде загружаемых модулей ядра.

То есть драйверы — многовходовые программы.

В модулях выше две точки входа:

1. `init` — вызывается, когда в командной строке вводим `insmod`
2. `exit` — вызывается, когда в командной строке вводим `rmmod`

Важен порядок загрузки модулей

Абсолютный адрес — физический адрес в ОЗУ, не преобразуется.

Порядок важен, так как `md2` должен получить абсолютный адрес переменных/функций из `md1`.

А если `md1` не загружен, то ему не откуда взять адрес.

Модуль может использовать только те имена, которые явно экспортированы .

Если раскомментировать в `md2.c` строку (1) с `md1_local`, то возникнет ошибка на этапе компиляции (несоответствие `md1_local` в `md1.c` и `md.h`).

Если раскомментировать в `md2.c` строку (2) с `md1_noexport`, то возникнет ошибка на этапе компоновки, так как `md1_noexport` не была передана в макрос `EXPORT_SYMBOL` и нет абсолютного адреса.

Из Цирюлика про ошибки порядка загрузки и выгрузки модулей:

Попробуем установить (загрузить) один из новых модулей в системе:

1	<code>\$ sudo insmod md2.ko</code>
2	<code>insmod: error inserting 'md2.ko': -1 Unknown symbol in module</code>

Возникла ошибка, потому что модуль **содержит** ссылки на неизвестные ядру имена (хотя известно, что эти недостающие имена определены в другом модуле `md1`). В системном журнале об этой ошибке будет выведена следующая информация:

1	\$ dmesg tail -n30 grep md
2	md2: Unknown symbol md1_data
3	md2: Unknown symbol md1_proc

После успешного создания и загрузки модулей можно их выгрузить с помощью команды `rmmod`:

1	\$ sudo rmmod md1
2	ERROR: Module md1 is in use by md2

Однако на этом шаге снова возникает ошибка. Рассмотрим листинг выполнения команды `lsmod`, расположенный несколькими абзацами выше:

- на модуль **md1** ссылаются некоторые другие модули или объекты ядра: цифра **1** — это число таких ссылающихся модулей, которое называется **счётчиком ссылок**;
- дальше за счётчиком ссылок указывается **список** тех модулей, откуда исходят такие ссылки, в данном случае, это один модуль **md2**;
- до тех пор, пока число ссылок на любой модуль в системе **не станет нулевым**, модуль **не может быть выгружен**;
- другими словами, модуль может быть выгружен только после того, как будут выгружены все ссылающиеся на него другие модули (загруженные **после** него — здесь не может возникнуть цикличности или перекрёстности ссылок);
- бывают случаи (как будет показано дальше), когда модуль вообще не может быть выгружен, в частности, когда счётчик ссылок для модуля не может быть сделан нулевым в силу каких-то причин.

Виртуальная файловая система proc

Для чего нужна?

Сама файловая система `proc` написана, чтобы программист мог получить в режиме пользователя важную для него информацию.

Если программиста не устроит эта информация, то используя файл `proc` в

ядре можно получить расширенную информацию.
Для этого нужно написать загружаемый модуль ядра.

В первом случае в режим пользователя, в другом — в режиме ядра.

Исторически: ФС `proc` написана, чтобы предоставить разработчикам ПО расширенную информацию о его программном продукте. Часть могли получить командами `ls`, `ps`, ...

Что из себя представляет:

`/proc` — виртуальная файловая система `proc`, так как она немонтируемая. Создается налету, когда к ней производится обращение.

Папки (каталоги) и файлы виртуальной файловой системы `/proc` не хранятся на диске. Они создаются динамически при обращении к ним. Большинство файлов в ней доступны только для чтения, однако некоторые из них доступны для записи, что позволяет изменять переменные ядра.

Отступление: код ядра — структуры и функции. Для разных объектов ядра создаются соответствующие экземпляры. Различаем объявление (определение) и инициализацию **объекта ядра** для конкретных элементов, с которыми работает программа.

Для каждого процесса в виртуальной файловой системе `proc` имеется директория: `proc/<pid>`, определенная идентификатором процесса. `Proc` — виртуальная файловая система $\Rightarrow$ информация, полученная о конкретном процессе с использованием `proc`, будет взята из дескриптора процесса и из подчиненных структур, на которые в дескрипторе есть указатели. Помимо указания идентификатора конкретного процесса можно использовать ссылку `self`.

В каждой такой директории большое количество поддиректорий, которые позволяют анализировать ресурсы, потребляемые разработанным ПО.

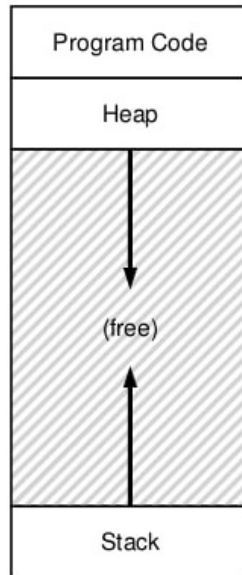
В `/proc/<pid>` есть следующие файлы, символичные ссылки и подкаталоги:

Элемент	Тип	Содержание
<code>cmdline</code>	файл	указывает на директорию процесса (прежде чем запустить файл на выполнение)

		он лежал в какой-то директории) (то есть речь о исполняемом файле)
cwd	символическая ссылка	указывает на директорию процесса
environ	файл	список окружения процесса
exe	символическая ссылка	указывает на образ процесса (то есть на его файл)
fd	директория	ссылка на открытые процессом файлы
root	soft link (символическая ссылка)	корень файловой системы процесса. То есть той файловой системы, где хранится исполняемый файл.
stat	файл	информация о процесса: например vsize — размер виртуальной памяти в байтах
maps	файл	содержит список регионов (участков памяти) выделенных процессу. Речь о виртуальном адресном пространстве.
pagemap	файл	Таблица страниц

Что видели в лабе (есть в билете на экзе!)

1. tasks — информация коррелирует с той, что видели в struct task_struct
Однако есть и дополнительная информация — stat.
В stat есть vsize — размер виртуального адресного пространства, есть
start_brk — адрес начала кучи, и т.д.
2. maps — отображение регионов адресного пространства
На экзамене нужно нарисовать то, что видели в лабе: по типу той, что
приведена ниже, но не free, а анонимные отображения



3. pagermap — таблицы страниц.

Важные биты: present, swapped, dirty, ... (те, что были в лабе по процу. Эта инфа требуется на экзамене, причем важны именно названия битов)

Программа, которая выводит на экран содержимое файла `environ`

```
#include <stdio.h>
#define BUF_SIZE 0x100
int main(int argc, char **argv)
{
    char buf[BUF_SIZE];
    int len, i;
    FILE *f;
    f = fopen("/proc/self/environ", "r");
    while((len=fread(buf, 1, BUF_SIZE, f)) > 0)
    {
        for(i = 0; i < len; ++i)
        {
            if(buf[i]==0)
            {
                buf[i] = 10;
            }
        }
    }
}
```

```
        buf[len] = 0;
        printf("%s", buf);
    }
}
close(f);
}
```

Файл `environ` не разделен на строки, все идет последовательно ⇒ добавляем перевод каретки.

Какую информацию еще предоставляет

VFS `proc` предоставляет информацию о системе в целом: `/proc/interrupts`.

В этом файле получим все прерывания, то есть те прерывания, что установлены в системе.

Также есть информация:

1. `/proc/cpuinfo`
2. `/proc/vmstat`
3. `/proc/pci`
4. `/proc/apm` — состояние батареи
5. `/proc/drivers` — информация об активных драйверах устройств

Прос: структуры

Важнейшая задача VFS `proc` — выдавать информацию о процессах в режиме пользователя: время, информацию об адресном пространстве процесса и ресурсах, которые этот процесс получает. Ресурсы разнообразные: сокеты, семафоры, разделяемая память.

Основная информация находится в ядре. Прос формирует информацию, которую используют пользователи, налету на основании соответствующих объектов ядра, в частности дескриптора процесса — структуры, описывающей виртуальное адресное пространство.

Как передать информацию из режима ядра в режим пользователя?

Напрямую туда обратно нельзя.

Для этого имеются специальные функции ядра: и нужно писать загружаемый модуль, в нем использовать структуры `vfs proc`.

Иерархия версий ядра: структуры ядра не стандартизированы, стандартизировано API

Под Linux есть большое число драйверов, и есть критические моменты, которые надо учитывать. Система должна поддерживать старые драйвера ⇒ структуры, связанные с драйверами просто выбросить нельзя.

Яркий пример добавления функций — VFS proc: `<linux/proc_fs.h>`

Proc dir entry

На proc определена структура ядра: `struct proc_dir_entry`. Эта структура — базовая структура ВФС определяет файл proc.

```
struct proc_dir_entry {
    atomic_t in_use;
    const struct inode_operations *proc_iops;
    union {
        const struct proc_ops *proc_ops; // нововведение в ве
псии 5.12
        const struct file_operations *proc_dir_ops;
    };
    const struct dentry_operations *proc_dops;
    union {
        const struct seq_operations *seq_ops;
        int (*single_show)(struct seq_file *, void *);
    };
    proc_write_t write;
    void *data;
    unsigned int low_ino;
    nlink_t nlink;
    kuid_t uid;
    kgid_t gid;
    loff_t size;
    struct proc_dir_entry *parent;
```

```
    char *name;  
    u8 flags;  
};
```

Раньше в `proc_dir_entry` был только `file_operations`: эта структура используется для регистрации собственных функций.

`struct proc_ops` — были определены операции над файлами `proc`, нововведение.

Зачем?

Функции `read/write` в `struct file_operations` используются очень часто ⇒ использовать их для нужд VFS, основная задача которых — передача данных в user — лишнее.

Следовательно, освободили от действий, связанных с файлами `proc`.

Также нововведением является второй `union`: второй возможный интерфейс передачи данных: `sequence` файлы.

Этот механизм предназначен только для передачи данных из ядра в user, то есть только `read`,

```
struct proc_ops {  
    unsigned int proc_flags;  
    int (*proc_open)(struct inode *, struct file *);  
    ssize_t (*proc_read)(struct file *, char __user *, size_  
t, loff_t *);  
    ssize_t (*proc_write)(struct file *, const char __user *,  
size_t, loff_t *);  
    int (*proc_release)(struct inode *, struct file *);  
};
```

Ядро всегда предоставляет возможность написания собственных точек входа.

`read/write` как раз являются точками входа, их мы можем переписать.

То есть для того, чтобы передать данные между user и kernel нам нужны функции передачи: `read`(в юзер)/`write`(в ядро).

1. `open` — функция, которая открывает файл. В зависимости от флагов для `open`, `open` либо открывает существующий файл, либо создает новый. Параметры `open` соответствуют параметрам `open` в ...
2. Ни в `struct proc_ops`, ни в ..., нет функции `close`. Она есть только на уровне пользователя в стандартной библиотеке. Как системный вызов существует только `release`.

Создание `proc`

Для создания файла `proc` используется системный вызов `proc_create`.
`proc_create` — обертка `proc_create_data`.

```
struct proc_dir_entry* proc_create_data(const char* name,
                                         umode_t mode, struct proc_dir_entry* parent,
                                         const struct file_operations* proc_fops,
                                         void* data);
```

```
static inline struct proc_dir_entry* proc_create(
    const char* name,
    umode_t mode,
    struct proc_dir_entry* parent,
    const struct file_operations* proc_fops)
{
    return proc_create_data(name, mode, parent, proc_fops, NULL);
}
```

Данные сигнатуры соответствуют версии ядра 4.10, поэтому видим `struct file_operations`.

С версии 5.10 в сигнатурах будет `proc_ops`.

```
struct proc_dir_entry* proc_create_data(const char* name,
                                         umode_t mode, struct proc_dir_entry* parent,
```

```
const struct proc_ops* proc_fops,  
void* data);
```

Чтобы сделать программу менее зависимой от версии ядра, нужно использовать условную компиляцию.

Формальные параметры у функций в этих структурах одни и те же!

```
#ifdef HAVE_PROC_OPS  
static struct proc_ops file_ops =  
{  
    .proc_read = procfs_read,  
    .proc_write = procfs_write,  
    .proc_open = procfs_open,  
    .proc_release = procsfs_close  
};  
#else  
static const struct file_operations file_ops =  
{  
    .read = procfs_read,  
    .write = procfs_write,  
    .open = procfs_open,  
    .release = procfs_close  
};  
#endif
```

Передача данных из ядра в юзер и обратно

Для передачи данных из ядра в юзер и из юзера в ядро нужно использовать специальные функции: `copy_to_user()` и `copy_from_user()`.

Ядро находится в физической памяти (замечим, что md2 получает абсолютные адреса — адреса физической памяти).

В любой операционной системе:

1. резидентная часть — постоянно в памяти, NON_PAGING
2. транзитивная часть — подгружается в память по мере надобности (например, библиотеки ядра) — PAGING

Ядро оперирует физическими адресами. Для обращения к памяти нужен физический адрес.

Процессы же имеют виртуальное адресное пространство.

Поэтому невозможно функциями `stdlib.h` передать данные.

Нужны специальные функции ядра.

Если у процесса виртуальное адресное пространство, и процессу надо передать данные в kernel (например запрос определенных данных, которые выдает ядро), для этого страница где выполняется действие должна быть загружена в ОЗУ.

Хотим передать из user в kernel. (`echo/cat` — выполнили в пользователе)

Записали данные в буфер ядра.

Прочитали в ядре строку и хотим вывести `cat` ⇒ из буфера ядра надо передать в буфер пользователя.

Но страница оказалась выгружена (не будет физического адреса, куда загружать).

⇒ нужны специальные функции.

1.

```
unsigned long __copy_to_user(void __user* to, const void *
from, unsigned long n);
```

1. `to` — куда записываем

2. `from` — источник в ядре

3. `n` — количество байтов

В случае успеха функция возвращает 0.

2.

```
unsigned long __copy_from_user(void *to, const void __user
*from, unsigned long n);
```

1. `to` — адрес буфера ядра

2. from — адрес буфера пользователя

3. n — количество байт

В случае успеха функция возвращает 0.

Эти функции могут перевести процесс в состояние блокировки, если PAGE_FAULT.

Лр фортулки

```
static struct proc_ops file_ops =
{
    .proc_read = proc_read,
    .proc_write = proc_write,
    .proc_open = proc_open,
    .proc_release = procs_close
};

static int __init md_init(void)
{
    ...
    proc_file = proc_create(FILENAME, 0640, NULL, &fops) ==
    ...
    return 0;
}

static ssize_t proc_read(struct file* file, char __user *buf, s:
{
    char *data;
    ...
    copy_to_user(buf, tmp_buf, read_len);
    ...
    return read_len;
}
```

```
static ssize_t proc_write(struct file* file, const char __user
{
    ...
    copy_from_user(&cookie_buf[write_index], buf, len);
    ...
}
```

1. Инициализация полей структуры `file_operations` — данный нам способ регистрации точек входа, так как `proc_read` и `proc_write` являются для нас точками входа.
До регистрации эти функции должны быть объявлены и описаны.
2. `proc_create` — создает файл `proc`, то есть файл с которым будут работать наши функции.
В `init` создаем директорию и символьную ссылку, а перед этим выделяем память для буфера, чтобы было куда записывать данные.
3. В функциях `read/write` в качестве первого параметра идет указатель на `struct file` — описывает открытый файл. Файл, который создали функцией `proc_create`.
4. `loff_t` — смещение в файловых системах Unix может иметь очень большие значения, так как Unix/Linux поддерживают очень большие файлы.

Полный код:

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/proc_fs.h>
#include <linux/vmalloc.h>
#include <linux/version.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Lebedev Vladimir");

#if LINUX_VERSION_CODE >= KERNEL_VERSION(5,6,0)
#define HAVE_PROC_OPS
#endif
```

```

#define COOKIE_POT_SIZE 4096
#define FILENAME "fortune_file"
#define DIRNAME "fortune_dir"
#define SYMLINK "fortune_link"

static struct proc_dir_entry *fortune_file;
static struct proc_dir_entry *fortune_dir;
static struct proc_dir_entry *fortune_link;

static char *cookie_pot = NULL;
char buffer[COOKIE_POT_SIZE];

static int read_index = 0;
static int write_index = 0;

static int fortune_open(struct inode *inode, struct file *file)
{
    printk(KERN_INFO "+fortune: open\n");
    return 0;
}

static int fortune_release(struct inode *inode, struct file *file)
{
    printk(KERN_INFO "+fortune: release\n");
    return 0;
}

static ssize_t fortune_write(struct file *file, const char __user *buf, size_t len, loff_t *fpos)
{
    printk(KERN_INFO "+fortune: write\n");

    if (len > COOKIE_POT_SIZE - write_index + 1)

```

```

    {
        printk(KERN_ERR "+fortune: buffer overflow\n");
        return -ENOSPC;
    }

    if (copy_from_user(&cookie_pot[write_index], buf, len) !=
0)
    {
        printk(KERN_ERR "+fortune: copy_from_user error\n");
        return -EFAULT;
    }

    write_index += len;
    cookie_pot[write_index - 1] = '\0';

    return len;
}

static ssize_t fortune_read(struct file *file, char __user *b
uf, size_t len, loff_t *fpos)
{
    int read_len;

    printk(KERN_INFO "+fortune: read\n");

    if ((*fpos > 0) || (write_index == 0))
        return 0;

    if (read_index >= write_index)
        read_index = 0;

    read_len = snprintf(buffer, COOKIE_POT_SIZE, "%s\n", &cooki
e_pot[read_index]);
    if (copy_to_user(buf, buffer, read_len) != 0)
    {
        printk(KERN_ERR "+fortune: copy_to_user error\n");
    }

```

```

    return -EFAULT;
}

read_index += read_len;
*fpos += read_len;

return read_len;
}

#ifdef HAVE_PROC_OPS
static const struct proc_ops fops =
{
    .proc_open = fortune_open,
    .proc_release = fortune_release,
    .proc_write = fortune_write,
    .proc_read = fortune_read
};
#else
static const struct file_operations fops =
{
    .open = fortune_open,
    .release = fortune_release,
    .write = fortune_write,
    .read = fortune_read
};
#endif

static int __init md_init(void)
{
    printk(KERN_INFO "+fortune: init\n");

    if ((cookie_pot = vmalloc(COOKIE_POT_SIZE)) == NULL)
    {
        printk(KERN_ERR "+fortune: memory error\n");
        return -ENOMEM;
    }
}

```

```

memset(cookie_pot, 0, COOKIE_POT_SIZE);

if ((fortune_file = proc_create(FILENAME, 0640, NULL, &fops)) == NULL)
{
    printk(KERN_ERR "+fortune: create file error\n");
    vfree(cookie_pot);
    return -ENOMEM;
}

if ((fortune_dir = proc_mkdir(DIRNAME, NULL)) == NULL)
{
    printk(KERN_ERR "+fortune: create dir error\n");
    remove_proc_entry(FILENAME, NULL);
    vfree(cookie_pot);
    return -ENOMEM;
}

if ((fortune_link = proc_symlink(SYMLINK, NULL, FILENAME)) == NULL)
{
    printk(KERN_ERR "+fortune: create link error\n");
    remove_proc_entry(FILENAME, NULL);
    remove_proc_entry(DIRNAME, NULL);
    vfree(cookie_pot);
    return -ENOMEM;
}

read_index = 0;
write_index = 0;

printk(KERN_INFO "+fortune: loaded\n");

return 0;

```

```

}

static void __exit md_exit(void)
{
    printk(KERN_INFO "+fortune: exit\n");
    remove_proc_entry(FILENAME, NULL);
    remove_proc_entry(DIRNAME, NULL);
    remove_proc_entry(SYMLINK, NULL);
    vfree(cookie_pot);
}

module_init(md_init);
module_exit(md_exit);

```

Sequence файлы

Второй интерфейс. Для сиквенсов определена структура `struct seq_file` и структура `struct seq_operations`.

В union два поля: `seq_ops` и `single open`.

Для seq файлов определены два способа работы: полный и упрощенный.

Полный — итератор, есть функция `iter`.

Упрощенный — `single_open`, разработчики освобождаются от необходимости писать полный итератор.

Почему ввели? В seq файлах более строго реализован переход по указателям и гарантируется, что данные будут переданы правильно.

```

struct seq_file {
    char *buf;
    size_t size;
    size_t from;
    size_t count;
    size_t pad_until;
    loff_t index;

```

```

loff_t read_pos;
struct mutex lock;
const struct seq_operations *op;
int poll_event;
const struct file *file;
void *private;
};

```

```

struct seq_operations {
    void * (*start) (struct seq_file *m, loff_t *pos);
    void (*stop) (struct seq_file *m, void *v);
    void * (*next) (struct seq_file *m, void *v, loff_t *pos);
    int (*show) (struct seq_file *m, void *v);
};

```

show — ядро интерфейса.

Seq файлы предназначены только для передачи данных из ядра в режим пользователя.

В отличие от proc_ops, где определены базовые:

1. read — из ядра в юзер
2. write — передача данных и режима пользователя в режим ядра

1.

```
int seq_open(struct file * file, const struct seq_operations * op);
```

2.

```
ssize_t seq_read(struct file* file,
                  char __user* buf, //буфер уровня пользователя, в него пишем из file
                  size_t size,
                  loff_t* ppos);
```

```
3. int seq_release ( struct inode * inode, struct file * fi
le);
```

4. Упрощенный интерфейс работы с seq_file

```
int sigle_open(struct file*, int (*)(struct seq_file *, vo
id *), void *);
```

В качестве аргумента передается функция, и это функция show.

Single Open

```
int sigle_open(struct file *file, int (*show)(struct seq_file
*, void*), void *data)
{
    struct seq_operations *op = kmalloc(sizeof(*op), GFP_KERN
EL_ACCOUNT);
    int res = -ENOMEM;
    if(op)
    {
        op->start = single_start;
        op->next = single_next;
        op->stop = single_stop;
        op->show = show;
        res = seq_open(file, op);
        if(!res)
        {
            ((struct seq_file*)file->private_data)->private =
data;
        }
        else
        {
            kfree(op);
        }
    }
}
```

```

        return res;
    }

EXPORT_SYMBOL(single_open);

```

Функции определены в самой функции `single_open` — непосредственно в теле они зарегистрированы. **Это освобождает нас от необходимости писать полный итератор.**

Пример: helloworld из ядра в user. Single File

```

#include <linux/module.h>
#include <linux/proc_fs.h>
#include <linux/seq_file.h>
#define PROC_FILE_NAME "Hello"
static struct proc_dir_entry *proc_file;
static char *out_str;

static int proc_hello_show(struct seq_file *m, void v)
{
    int error = seq_printf(m, "%s\n", out_str);
    return error;
}

static int proc_hello_open(struct inode *inode, struct file *
file)
{
    return single_open(file, proc_hello_show, NULL);
}

static const struct file_operations proc_fops =
{
    .owner = THIS_MODULE,
    .open = proc_hello_open,
    .release = single_release,

```

```

        .read = seq_read
};

static int __init proc_hello_init(void)
{
    out_str = "Hello";
    proc_file = proc_create_data(PROC_FILE_NAME, S_IRUGO, NULL,
    &proc_fops, NULL);
    if(!proc_file)
        return -ENOMEM;
    return 0;
}

static void __exit proc_hello_exit(void)
{
    if(proc_file)
        remove_proc_entry(PROC_FILE_NAME, NULL);
}

```

Можно использовать для seq файлов struct file_operations, регистрируя свои функции.

В данном случае single_read и single_release;

Таким образом мы регистрируем 3 точки входа в модуль: open, release и read (+init, exit).

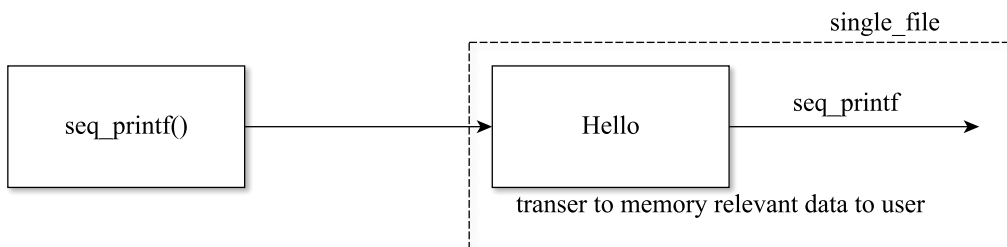
В init вызываем proc_create_data — эта функция принимает имя файла в proc и устанавливает права: read user group others; Третий параметр NULL - это parent. Соответственно указатель, где данный файл будет находиться в дереве каталогов.

Null ⇒ будет находиться в корневом каталоге proc.

Также видим, что для доступа к файлу нужны три функции: open, read и release, однако в данном случае нет необходимости и пишем только open. Можно определить read/release, но в этом нет необходимости и это полезно в случае итератора.

В `proc_hello_show` вызывается `seq_printf` — она и выполняет все действия. Но точкой входа является `open`, которую мы регистрируем.

Разработчики ядра построили промежуточный слой отказоустойчивости, которым считается слой обмена данным до чего-то вроде `print`. Не нужно ничего дополнительно писать — все написано за нас. Базовая идея промежуточного слоя в том, что разработчики модуля записывают все данные в достаточно большую область памяти. Для `single_open` размер передаваемых данных ограничен 64 КБ (16 страниц).



На рисунке показано использование функции `seq_printf` для записи данных в ОЗУ. Подсистема `single_file` копирует данные в пространство пользователя. Как мы видим раскрыв тело функции `single_open`, эта функция передает более сложная, чем наша, и существует опасность, что `user` может прочитать не все данные, а только их подмножество.

В `single_open` вызывается `show`, которая передает адрес страницы памяти, из которой данные выводятся функцией `seq_printf`.

Функция `seq_printf` в `show` может быть вызвана произвольное число раз.

В `single file` не определена функция `write`. Для `write` надо использовать....

Код лабораторной с итератором

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/proc_fs.h>
#include <linux/vmalloc.h>
#include <linux/seq_file.h>
```

```

#include <linux/version.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Lebedev Vladimir");

#if LINUX_VERSION_CODE >= KERNEL_VERSION(5,6,0)
    #define HAVE_PROC_OPS
#endif

#define FILENAME "seq_file"

static struct proc_dir_entry *seq_file;

static void *seq_start(struct seq_file *m, loff_t *pos)
{
    printk(KERN_INFO "+seq: start\n");
    static unsigned long counter = 0;
    if (!*pos)
    {
        return &counter;
    }
    else
    {
        *pos = 0;
        return NULL;
    }
}

static void seq_stop(struct seq_file*m, void *v)
{
    printk(KERN_INFO "+seq: stop\n");
}

static int seq_show(struct seq_file *m, void *v)
{
    printk(KERN_INFO "+seq_file: show\n");
}

```

```

    seq_printf(m, "PID: %d, PPID: %d, COMM: %s\n", current->pid, current->parent->pid, current->comm);
    seq_printf(m, "Prio: %d, Policy: %d\n", current->prio, current->policy);
    return 0;
}

static void *seq_next(struct seq_file *m, void *v, loff_t *pos)
{
    printk(KERN_INFO "+seq: next\n");
    unsigned long *tmp = (unsigned long *)v;
    (*tmp)++;
    (*pos)++;
    return NULL;
}

static struct seq_operations seq_ops = {
    .start = seq_start,
    .stop = seq_stop,
    .show = seq_show,
    .next = seq_next
};

static int seq_file_open(struct inode *i, struct file *f)
{
    printk(KERN_INFO "+seq: open seq_file\n");
    return seq_open(f, &seq_ops);
}

static ssize_t seq_file_read(struct file *file, char __user *buf, size_t size, loff_t *off)
{
    printk(KERN_INFO "+seq: read\n");
    return seq_read(file, buf, size, off);
}

```

```

static int seq_file_release (struct inode * inode, struct file * file)
{
    printk(KERN_INFO "+seq: release seq_file\n");
    return seq_release(inode, file);
}

static ssize_t seq_file_write(struct file *file, const char __user *buf, size_t len, loff_t *fpos)
{
    printk(KERN_INFO "+seq: write\n");
    return 0;
}

#ifdef HAVE_PROC_OPS
    static const struct proc_ops fops =
    {
        .proc_open = seq_file_open,
        .proc_release = seq_file_release,
        .proc_write = seq_file_write,
        .proc_read = seq_file_read
    };
#else
    static const struct file_operations fops =
    {
        .open = seq_file_open,
        .release = seq_file_release,
        .write = seq_file_write,
        .read = seq_file_read
    };
#endif

static int __init md_init(void)
{
    printk(KERN_INFO "+seq: init\n");

```

```

    if ((seq_file = proc_create(FILENAME, S_IRUGO, NULL, &fop
s)) == NULL)
    {
        printk(KERN_ERR "+seq: create file error\n");
        return -ENOMEM;
    }

    printk(KERN_INFO "+seq: loaded\n");

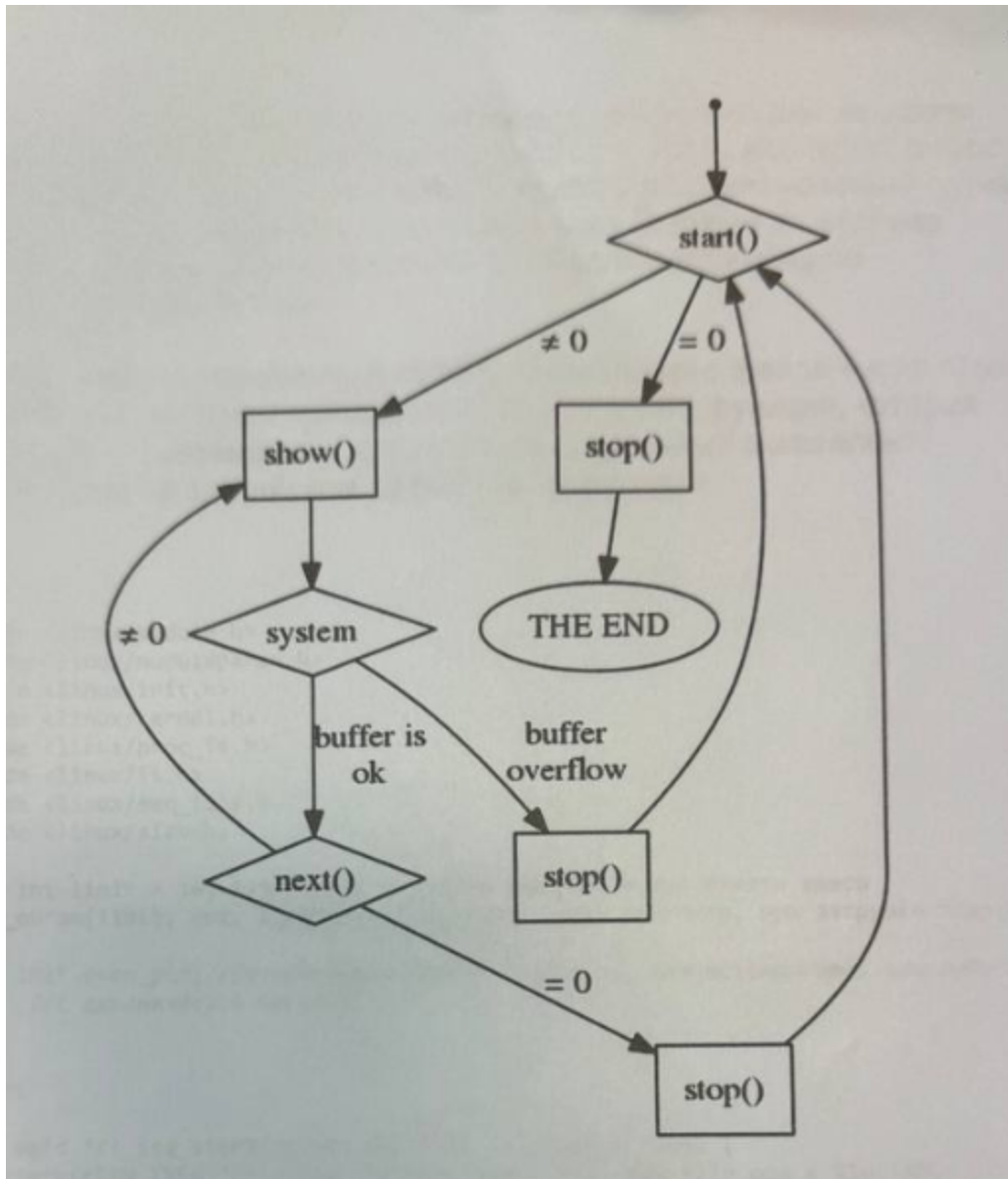
    return 0;
}

static void __exit md_exit(void)
{
    printk(KERN_INFO "+seq: exit\n");
    remove_proc_entry(FILENAME, NULL);
}

module_init(md_init);
module_exit(md_exit);

```

Итератор



Для того, чтобы передать данные, используя полный интерфейс seq_file, нужны 4 функции: start, stop, show, next.

Функция stop вызывается в трех случаях:

1. Вызов stop сразу после start означает окончание передачи данных
2. Stop после show означает, что после последней передачи данных буфер, в который передаются данные, заполнен $\Rightarrow$ передача данных завершается.

3. Вызов после next выполняется, если нужно закончить передачу в буфер данных и завершить работу.

Код лабораторной с single_open

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/proc_fs.h>
#include <linux/vmalloc.h>
#include <linux/seq_file.h>
#include <linux/version.h>
#include <linux/fs_struct.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Lebedev Vladimir");

#if LINUX_VERSION_CODE >= KERNEL_VERSION(5,6,0)
#define HAVE_PROC_OPS
#endif

#define COOKIE_POT_SIZE 4096
#define FILENAME "single_seq_file"

static struct proc_dir_entry *seq_file;

static char *cookie_pot = NULL;
char buffer[COOKIE_POT_SIZE];

static int read_index = 0;
static int write_index = 0;

static int seq_show(struct seq_file *m, void *v)
{
    printk(KERN_INFO "+seq: show\n");

    if (!write_index)
```

```

    return 0;

    if (read_index >= write_index)
        read_index = 0;

    int read_len = snprintf(buffer, COOKIE_POT_SIZE, "%s\n", &cookie_pot[read_index]);
    struct task_struct *task = &init_task;
    int flag = 0;
    long pid = 0;
    read_index += read_len;
    if (kstrtoul(buffer, 10, &pid) != 0)
    {
        seq_printf(m, "converting string to number error\n");
    }
    else {
        do {
            if (pid == (long int) task->pid)
            {
                flag = 1;
                seq_printf(m, "%s, pid: %d, parent: %s, ppid: %d, policy: %d\n",
                           task->comm, task->pid, task->parent->comm, task->parent->pid, task->policy);
            }
        } while (!flag && (task = next_task(task)) != &init_task);

        if (flag == 0)
        {
            printk(KERN_ERR "+seq: no such pid, %ld\n", pid);
            seq_printf(m, "no such pid, %ld\n", pid);
        }
    }

    return 0;

```

```

}

static int seq_file_open(struct inode *inode, struct file *file)
{
    printk(KERN_INFO "+seq: open\n");
    return single_open(file, seq_show, NULL);
}

static ssize_t seq_file_read(struct file *file, char __user *
buf, size_t size, loff_t *off)
{
    printk(KERN_INFO "+seq: read\n");
    return seq_read(file, buf, size, off);
}

static int seq_file_release(struct inode *inode, struct file
*file)
{
    printk(KERN_INFO "+seq: release\n");
    return single_release(inode, file);
}

static ssize_t seq_file_write(struct file *file, const char _
_user *buf, size_t len, loff_t *fpos)
{
    printk(KERN_INFO "+seq: write\n");

    if (len > COOKIE_POT_SIZE - write_index + 1)
    {
        printk(KERN_ERR "+seq: buffer overflow\n");
        return -ENOSPC;
    }

    if (copy_from_user(&cookie_pot[write_index], buf, len) !=
0)

```

```

{
    printk(KERN_ERR "+seq: copy_from_user error\n");
    return -EFAULT;
}

write_index += len;
cookie_pot[write_index - 1] = '\0';

return len;
}

#ifdef HAVE_PROC_OPS
static const struct proc_ops fops =
{
    .proc_open = seq_file_open,
    .proc_release = seq_file_release,
    .proc_write = seq_file_write,
    .proc_read = seq_file_read
};
#else
static const struct file_operations fops =
{
    .open = seq_file_open,
    .release = seq_file_release,
    .write = seq_file_write,
    .read = seq_file_read
};
#endif

static int __init md_init(void)
{
    printk(KERN_INFO "+seq: init\n");

    if ((cookie_pot = vmalloc(COOKIE_POT_SIZE)) == NULL)
    {
        printk(KERN_ERR "+seq: memory error\n");
    }
}

```

```

        return -ENOMEM;
    }
    memset(cookie_pot, 0, COOKIE_POT_SIZE);

    if ((seq_file = proc_create(FILENAME, 0640, NULL, &fops)) =
= NULL)
    {
        printk(KERN_ERR "+seq: create file error\n");
        vfree(cookie_pot);
        return -ENOMEM;
    }

    read_index = 0;
    write_index = 0;

    printk(KERN_INFO "+seq: loaded\n");

    return 0;
}

static void __exit md_exit(void)
{
    printk(KERN_INFO "+seq: exit\n");
    remove_proc_entry(FILENAME, NULL);
    vfree(cookie_pot);
}

module_init(md_init);
module_exit(md_exit);

```

▼ Numa

NON_UNIFORM_MEMORY_ACCESS

Неравномерный доступ к памяти.

Схема реализована и используется в многоядерных системах.

Время доступа к памяти определяется ее положением по отношению к процессору.

Каждый процессор может "напрямую" обратиться к определенным слотам памяти, а другим опосредственно.

То есть каждый может обратиться к любым слотам, но у него есть его "личные" слоты.

+ задержки io controller, что плохо с точки зрения доступа к памяти

Также каждый процессор имеет собственные кеши L1, L2, L3.

SMP — symmetric multi processor.

Все разбивается на ячейки.

Ячейка состоит из процессора, локальной памяти, шины ввода/вывода.

Каждая ячейка может рассматриваться как SMP.

Поднабор, необходимый для одиночной SMP...

Ячейки Numa связаны друг с другом с помощью некоторого вида системной взаимосвязи, а именно crossbar или point to point.

В Linux: cache numa

Буферизованный/небуферизованный ввод/вывод

<stdio.h> — библиотека буферизованного ввода/вывода.

Все ее функции работают с буфером.

В это стандартной библиотеке Си определена структура `_IO_FILE`.

Название `FILE` определяется define-ом на основе этой структуры, в ней все поля определяют работу с буфером.

```
struct _IO_FILE
{
    int _flags;
    char *_IO_read_ptr;
```

```

char *_IO_read_end;
char *_IO_read_base;
char *_IO_write_base;
char *_IO_write_ptr;
char *_IO_write_end;
char *_IO_buf_base;
char *_IO_buf_end;

int _fileno; // файловый дескриптор в таблице открытых фа
йлов процесса
};

typedef struct _IO_FILE FILE;

```

При буферизованном вводе/выводе вся информация записывается сначала в буфер, и затем обрабатывается из буфера.

`fileno` — файловый дескриптор в таблице открытых файлов процесса.

Любой дескриптор процесса содержит информацию о файловой системе и о файлах, открытых процессом.

`FILE_SYSTEM_INFO` — нужен, так как до того как стать процессом, это программа. Программа это файл (речь, конечно же, о регулярных файлах). Любой процесс имеет собственную таблицу открытых файлов.

Структура `struct files_struct` описывает файлы, открытые процессом.

```

struct files_struct {
    atomic_t count;

    unsigned int next_fd;
    unsigned long close_on_exec_init[1];
    unsigned long open_fds_init[1];
    unsigned long full_fds_bits_init[1];
    struct file __rcu *fd_array[NR_OPEN_DEFAULT];
};

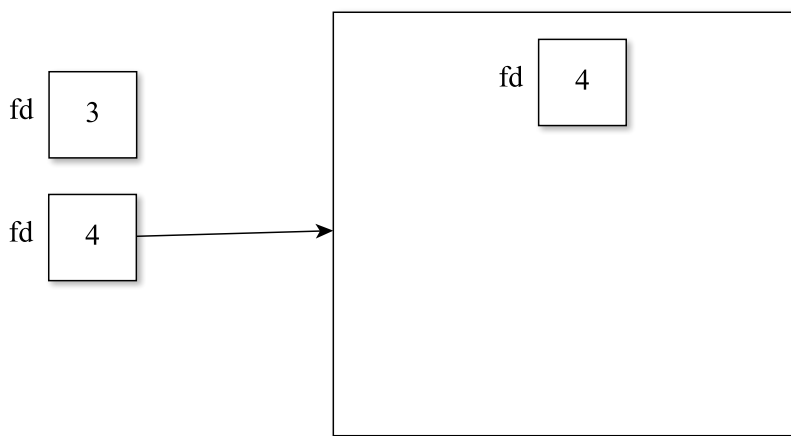
```

Индекс открытого файла в массиве открытых файлов процесса возвращает `open`.

Для любого процесса `stdin`, `stdout`, `stderr` создаются автоматически $\Rightarrow$ далее возможны индексы 3,4,5....

Процесс может создать только 512 открытых файлов. Базовое значение в ядре: 256.

Если надо еще — создается дополнительная таблица еще на 256.



PFT — process file table

SFT — system file table

3 и 4 ссылаются на дескриптор открытых файлов в системной таблице открытых файлов.

Будет создано два дескриптора: один библиотечной функцией `open`, другой системным вызовом `forkopen`.

SFT $\rightarrow$ offset — в Linux `f_pos`.

`struct file` предполагает логическую структуру файла.

Как существует логическая структура программы — любой файл начинается с 0 адреса, так и любой файл считает, что его смещение начинается с 0.

Любая операция чтения/записи смещает указатель.

Чтение текстового файла — смещение на байт.

В дескрипторе файла находится указатель на `inode` открытого файла.

`inode`, в свою очередь, имеет указатель на суперблок.

struct file содержит struct path.

Через path обращается к структуре dentry. У struct dentry тоже есть inode.

Path — путь к файлу. Его разбирает ехес. Разбирает с использованием этой структуры.

Программа 1

```
#include <stdio.h>
#include <fcntl.h>

int main()
{
    int fd = open("alphabet.txt", O_RDONLY);
    FILE *fs1 = fdopen(fd, "r");
    char buff1[1];
    setvbuf(fs1, buff1, _IOFBF, 1);
    FILE *fs2 = fdopen(fd, "r");
    char buff2[1];
    setvbuf(fs2, buff2, _IOFBF, 1);
    int flag1 = 1, flag2 = 2;
    while(flag1 == 1 || flag2 == 1)
    {
        char c;
        flag1 = fscanf(fs1, "%c", &c);
        if (flag1 == 1)
        {
            fprintf(stdout, "%c", c);
        }
        flag2 = fscanf(fs2, "%c", &c);
        if (flag2 == 1)
        {
            fprintf(stdout, "%c", c);
        }
    }
    printf("\n");
}
```

```
    return 0;
}
```

```
swanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sem/lab_io$ gcc -Wall -Werror -o ./testCIO.out ./testCIO.c
swanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sem/lab_io$ ./testCIO.out
Aubvcwdxyfzghijklmnopqrst
swanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sem/lab_io$
```

Многопоточная версия:

```
#include <stdio.h>
#include <fcntl.h>
#include <pthread.h>

struct thread_args_struct
{
    FILE* fs;
};

void *thread_read(void *args)
{
    struct thread_args_struct *cur_args = (struct thread_args_struct *) args;
    FILE *fs = cur_args->fs;
    int flag = 1;

    while (flag == 1)
    {
        char c;
        if ((flag = fscanf(fs, "%c", &c)) == 1)
            fprintf(stdout, "%c", c);
    }
    return NULL;
}

int main(void)
{
```

```

int fd = open("alphabet.txt", O_RDONLY);

FILE *fs[2] = {fdopen(fd, "r"),
               fdopen(fd, "r")};
char buff1[20];
setvbuf(fs[0], buff1, _IOFBF, 20);
char buff2[20];
setvbuf(fs[1], buff2, _IOFBF, 20);
pthread_t thr[2];
struct thread_args_struct args[2] = {{ .fs = fs[0] }, { .
fs = fs[1] }};
for (int i = 0; i < 2; i++)
{
    if (pthread_create(&thr[i], NULL, thread_read, &args
[i]))
    {
        perror("pthread_create");
        return 1;
    }
}
//pthread_join(thr[0], NULL);
//pthread_join(thr[1], NULL);
puts("");
return 0;
}

```

```

swanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sem/lab_1c$ gcc -Wall -Werror -o ./thread_testC10.out ./thread_testC10.c
swanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sem/lab_1c$ ./thread_testC10.out
Abcdefghijklmnopqrstuvwxyz
swanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sem/lab_1c$

```

В первой программе файл «alphabet.txt» открывается только для чтения (установлен флаг O_RDONLY). Системный вызов open() создает дескриптор открытого файла и возвращает индекс элемента в таблице дескрипторов файлов, открытых процессом. Индекс равен 3, так как элементы массива fd_array с индексами 0, 1, 2 инициализированы стандартными потоками (stdin,

stdout, stderr), а другие файлы процессом не открывались. Поле fd_array[3] указывает на struct file, связанную со структурой struct inode, соответствующую файлу «alphabet.txt».

Функция fdopen() стандартной библиотеки создает структуры FILE (fs1, fs2), поле _fileno которых инициализируется дескриптором открытого файла.

Создаются два буфера buff1, buff2 размером 20 байт. Вызов функции setvbuf() задает буферы размером 20 байт для fs1, fs2, устанавливая полную буферизацию (_IOFBF).

В цикле вызывается функция стандартной библиотеки fscanf() для fs1 и fs2. Так как была установлена буферизация, при первом вызове fscanf() в буфер структуры fs1 будут записаны первые 20 символов (буквы 'A' – 't'). Значение f_pos в структуре struct file установится на следующий символ ('u'), а в переменную с запишется символ 'A', который будет выведен на экран. При первом вызове fscanf() для fs2, в буфер buff2 будут записаны оставшиеся символы (от 'u' до 'z'), так как fs2 ссылается на тот же дескриптор открытого файла, что и структура fs1, а поле f_pos соответствующей структуры struct file было изменено. В переменную с запишется символ 'u'. Далее в цикле fscanf() переменной с будут присваиваться значения символов из буферов, и попеременно выводиться значения из каждого буфера. Когда символы в одном из буферов кончатся, продолжится вывод символов только из одного буфера.

Функция fdopen возвращает fs1* и fs2* FILE.

Для этих указателей вызывается функция setbuf.

Создается буфер 20 байт.

В файле находятся буквы лат. алфавита.

Далее читаем то из одного, то из другого файлового дескриптора.

Первый fscanf работает таким образом, что будет заполнен весь 1 буфер размером 20 символов.

Оставшиеся 6 — попадут во 2 буфер.

fdopen не создает дескриптор.

_IO_FILE — библиотечная, буфера будут созданы в пользователе.

Программа 2: 2 раза вызывается системный вызов open (чтение и запись)

То есть I/O не буферизованный, никакие буфера созданы не будут, но будут созданы два дескриптора и ссылаться они будут на два дескриптора открытых файлов в системной таблице открытых файлов.

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
struct stat statbuf;

void print_stat(char *s)
{
    stat("q.txt", &statbuf);
    fprintf(stdout, "%s: inode = %ld, size = %ld bytes\n", s,
statbuf.st_ino, statbuf.st_size);
}

int main()
{
    int fd1 = open("q.txt", O_RDWR);
    print_stat("open fd1 ");
    int fd2 = open("q.txt", O_RDWR);
    print_stat("open fd2");
    for (char c = 'a'; c <= 'z'; c++)
    {
        if (c % 2)
            write(fd1, &c, 1);
        else
            write(fd2, &c, 1);
        print_stat("write    ");
    }
    close(fd1);
    print_stat("close fd1");
    close(fd2);
    print_stat("close fd2");
}
```

```
    return 0;
}
```

```
swanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sem/lab_io$ gcc -Wall -Werror -o ./test_openIO.c  
swanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sem/lab_io$ ./test_openIO.out  
open fd1 : inode = 1077563, size = 13 bytes  
open fd2: inode = 1077563, size = 13 bytes  
write   : inode = 1077563, size = 13 bytes  
write   : inode = 1077563, size = 13 bytes  
write   : inode = 1077563, size = 13 bytes  
write   : inode = 1077563, size = 13 bytes  
write   : inode = 1077563, size = 13 bytes  
write   : inode = 1077563, size = 13 bytes  
write   : inode = 1077563, size = 13 bytes  
write   : inode = 1077563, size = 13 bytes  
write   : inode = 1077563, size = 13 bytes  
write   : inode = 1077563, size = 13 bytes  
write   : inode = 1077563, size = 13 bytes  
write   : inode = 1077563, size = 13 bytes  
write   : inode = 1077563, size = 13 bytes  
write   : inode = 1077563, size = 13 bytes  
write   : inode = 1077563, size = 13 bytes  
write   : inode = 1077563, size = 13 bytes  
write   : inode = 1077563, size = 13 bytes  
write   : inode = 1077563, size = 13 bytes  
write   : inode = 1077563, size = 13 bytes  
close fd1: inode = 1077563, size = 13 bytes  
close fd2: inode = 1077563, size = 13 bytes  
swanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sem/lab_io$ cat q.txt  
bdfhjInprtvmxswanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sem/lab_io$
```

Многопоточный вариант

```
#include <fcntl.h>
#include <unistd.h>
#include <pthread.h>
#include <stdio.h>
#include <sys/stat.h>
```

```
struct stat statbuf;
```

```
struct thread_arg {
    int fd;
    int i;
};
```

```

void print_stat(char *s, int i)
{
    stat("s.txt", &statbuf);
    fprintf(stdout, "%s %d: inode = %ld, size = %ld bytes\n",
s, i, statbuf.st_ino, statbuf.st_size);
}

void *thread_start(void *arg)
{
    struct thread_arg *targ = arg;
    for (char c = 'a'; c <= 'z'; c++)
        if (c % 2 == targ->i)
        {
            write(targ->fd, &c, 1);
            print_stat("write", targ->i);
        }
    return NULL;
}

int main()
{
    int fd[2] = {open("s.txt", O_RDWR),
open("s.txt", O_RDWR)};
    pthread_t thr[2];
    struct thread_arg targ[2];
    for (int i = 0; i < 2; i++)
    {
        targ[i].fd = fd[i];
        targ[i].i = i;
        if (pthread_create(&thr[i], NULL, thread_start, &targ
[i]))
        {
            perror("pthread_create");
            return 1;
        }
    }
    //pthread_join(thr[0], NULL);
}

```

```

    //pthread_join(thr[1], NULL);
    close(fd[0]);
    close(fd[1]);
    return 0;
}

```

```

swanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sen/lab_io$ gcc -Wall -Werror -o ./thread_openIO.out ./thread_openIO.c
swanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sen/lab_io$ ./thread_openIO.out
write 0: inode = 1077965, size = 1 bytes
write 0: inode = 1077965, size = 2 bytes
write 0: inode = 1077965, size = 3 bytes
write 0: inode = 1077965, size = 4 bytes
write 0: inode = 1077965, size = 5 bytes
write 0: inode = 1077965, size = 6 bytes
write 0: inode = 1077965, size = 7 bytes
write 0: inode = 1077965, size = 8 bytes
write 0: inode = 1077965, size = 9 bytes
write 0: inode = 1077965, size = 10 bytes
write 0: inode = 1077965, size = 11 bytes
write 0: inode = 1077965, size = 12 bytes
write 0: inode = 1077965, size = 13 bytes
write 1: inode = 1077965, size = 1 bytes
write 1: inode = 1077965, size = 13 bytes
write 1: inode = 1077965, size = 13 bytes
write 1: inode = 1077965, size = 13 bytes
write 1: inode = 1077965, size = 13 bytes
write 1: inode = 1077965, size = 13 bytes
write 1: inode = 1077965, size = 13 bytes
write 1: inode = 1077965, size = 13 bytes
write 1: inode = 1077965, size = 13 bytes
write 1: inode = 1077965, size = 13 bytes
write 1: inode = 1077965, size = 13 bytes
write 1: inode = 1077965, size = 13 bytes
swanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sen/lab_io$ cat s.txt
acegikmoqsuwswanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sen/lab_io$

```

Во втором варианте второй программы файл («q.txt») дважды открывается для чтения и записи (O_RDWR). open() создает два дескриптора открытого файла (им присваиваются значения 3 и 4). Так как файл открывается дважды, то в системной таблице создается два дескриптора struct file, каждый из которых имеет собственный указатель f_pos. При первом вызове write() для fd1 символ 'a' записывается в файл на 0 позицию и соответствующий указатель f_pos увеличивается на 1. При первом вызове write() для fd2 символ 'b' также записывается в файл на 0 позицию и соответствующий указатель f_pos увеличивается на 1. Таким образом, при поочередной записи символов в файл в нем будут находиться только символы, которые записывались через fd2. При выполнении программы с двумя потоками в файл также не будут записаны все символы алфавита. Произошла потеря данных.

Для предотвращения потери данных файл следует дважды открыть для чтения, записи и добавления записи в конец (O_RDWR | O_APPEND).

Тогда при каждом вызове write() соответствующий символ будет записан на последнюю позицию. При первом вызове write() для fd1 символ 'a' записывается в файл на последнюю позицию (0). При первом вызове write() для fd2 символ 'b' записывается в файл на текущую последнюю позицию (1). Таким образом, при записи символов по очереди файл будет содержать все символы алфавита.

Программа 2: 2 раза open (чтение)

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
int main()
{
    char c;
    int fd1 = open("alphabet.txt", O_RDONLY);
    int fd2 = open("alphabet.txt", O_RDONLY);
    int flag1 = 1, flag2 = 1;
    int pos = 0;
    while ((flag1 == 1) && (flag2 == 1))
    {
        if (pos % 2 == 0)
            if (1 == (flag1 = read(fd1, &c, 1)))
            {
                if (pos % 2 == 0)
                    write(1, &c, 1);
                if (1 == (flag2 = read(fd2, &c, 1)))
                    if (pos % 2 == 1)
                        write(1, &c, 1);
            }
        pos++;
    }
    puts("");
}
```

```

return 0;
}

```

```

swanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sem/lab_io$ gcc -Wall -Werror -o ./testKernelIO.out ./testKernelIO.c
swanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sem/lab_io$ ./testKernelIO.out
AAbbccddeeffgghhiijjkkllmmnnooppqrrssttuuvvwwxxyyzz
swanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sem/lab_io$ █

```

Многопоточный вариант

```

#include <stdio.h>
#include <pthread.h>
#include <fcntl.h>
#include <unistd.h>
void *thread_read(void *arg)
{
    int *fd = arg;
    char c;
    while (read(*fd, &c, 1))
        write(1, &c, 1);
    return NULL;
}
int main()
{
    int fd[2] = {open("alphabet.txt", O_RDONLY),
                 open("alphabet.txt", O_RDONLY)};
    pthread_t thr[2];
    for (int i = 0; i < 2; i++)
    {
        if (pthread_create(&thr[i], NULL, thread_read, &fd
[i]))
        {
            perror("pthread_create");
            return 1;
        }
    }
    //pthread_join(thr[0], NULL);
}

```

```

    //pthread_join(thr[1], NULL);
    puts("");
    close(fd[0]);
    close(fd[1]);
    return 0;
}

```

```

swanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sem/lab_io$ ./thread_testKernelIO.out
AbcdefghijklmnopqrstuvwxyzAbcdefghijklmnopqrstuvwxyz
swanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sem/lab_io$ ./thread_testKernelIO.out
AbcdefghijklmnAobpcqdresftguhviwxkylmnopqrstuvwxyz
swanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sem/lab_io$

```

В первом варианте второй программы файл «alphabet.txt» дважды открывается только для чтения (O_RDONLY). Системный вызов open() создает два дескриптора открытого файла (им присваиваются значения 3 и 4). Также создаются две различные структуры struct file, ссылающиеся на одну и ту же структуру struct inode. Так как структуры struct file разные и их поля f_pos изменяются независимо, то для каждого файлового дескриптора произойдет полное чтение файла и каждый символ будет выведен по два раза

Программа 3:

```

#include <stdio.h>
#include <fcntl.h>
#include <sys/stat.h>

struct stat statbuf;

void print_stat(FILE *fs, char *s)
{
    stat("res.txt", &statbuf);
    fprintf(stdout, "%s: inode = %ld, size = %ld bytes, pos = %ld\n", s, statbuf.st_ino, statbuf.st_size, ftell(fs));
}

```

```

int main()
{
    FILE *fs1 = fopen("res.txt", "w");
    print_stat(fs1, "open fs1");
    FILE *fs2 = fopen("res.txt", "w");
    print_stat(fs2, "open fs2");
    for (char c = 'a'; c <= 'z'; c++)
    {
        if (c % 2)
        {
            fprintf(fs1, "%c", c);
            print_stat(fs1, "fprintf");
        }
        else
        {
            fprintf(fs2, "%c", c);
            print_stat(fs2, "fprintf");
        }
    }
    fclose(fs1);
    print_stat(fs1, "fclose fs1");
    fclose(fs2);
    print_stat(fs2, "fclose fs2");
    return 0;
}

```

```

swanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sem/lab_io$ gcc -Wall -Werror -o ./3.out ./3.c
swanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sem/lab_io$ ./3.out
open fs1: inode = 1077982, size = 0 bytes, pos = 0
open fs2: inode = 1077982, size = 0 bytes, pos = 0
fprintf: inode = 1077982, size = 0 bytes, pos = 1
fprintf: inode = 1077982, size = 0 bytes, pos = 1
fprintf: inode = 1077982, size = 0 bytes, pos = 2
fprintf: inode = 1077982, size = 0 bytes, pos = 2
fprintf: inode = 1077982, size = 0 bytes, pos = 3
fprintf: inode = 1077982, size = 0 bytes, pos = 3
fprintf: inode = 1077982, size = 0 bytes, pos = 4
fprintf: inode = 1077982, size = 0 bytes, pos = 4
fprintf: inode = 1077982, size = 0 bytes, pos = 5
fprintf: inode = 1077982, size = 0 bytes, pos = 5
fprintf: inode = 1077982, size = 0 bytes, pos = 6
fprintf: inode = 1077982, size = 0 bytes, pos = 6
fprintf: inode = 1077982, size = 0 bytes, pos = 7
fprintf: inode = 1077982, size = 0 bytes, pos = 7
fprintf: inode = 1077982, size = 0 bytes, pos = 8
fprintf: inode = 1077982, size = 0 bytes, pos = 8
fprintf: inode = 1077982, size = 0 bytes, pos = 9
fprintf: inode = 1077982, size = 0 bytes, pos = 9
fprintf: inode = 1077982, size = 0 bytes, pos = 10
fprintf: inode = 1077982, size = 0 bytes, pos = 10
fprintf: inode = 1077982, size = 0 bytes, pos = 11
fprintf: inode = 1077982, size = 0 bytes, pos = 11
fprintf: inode = 1077982, size = 0 bytes, pos = 12
fprintf: inode = 1077982, size = 0 bytes, pos = 12
fprintf: inode = 1077982, size = 0 bytes, pos = 13
fprintf: inode = 1077982, size = 0 bytes, pos = 13
fclose fs1: inode = 1077982, size = 13 bytes, pos = -1
fclose fs2: inode = 1077982, size = 13 bytes, pos = -1
swanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sem/lab_io$ cat res.txt
bdfhjlnprtvxzw
swanpoi@swanpoi-VirtualBox:~/Documents/BMSTU/OS/6_sem/lab_io$

```

В третьей программе файл дважды открывается на запись функцией `fopen()` из библиотеки буферизованного ввода-вывода `stdio.h`. В результате выполнения `fopen()` в системной таблице открытых файлов создаются два дескриптора `struct file`, каждый из которых имеет собственный указатель `f_pos`, при этом оба дескриптора ссылаются на один и тот же `inode`. Функция стандартной библиотеки `fprintf()` выполняет буферизованный вывод. При этом буфер создается неявно.

В данном случае запись в файл происходит в результате вызова функции `fclose()`. При вызове `fclose()` для `fs1` буфер для `fs1` записывается в файл.

При вызове `fclose()` для `fs2` содержимое файла очищается и буфер для `fs2` записывается в файл. Таким образом, в файле находится только содержимое буфера для `fs2`, то есть происходит потеря данных.

Для предотвращения потери данных при втором открытии файла можно указать режим записи в конец файла ("a"). В таком случае при вызове `fclose()`

для fs2 данные из файла не будут очищены, содержимое буфера для fs2 будет добавлено в конец файла.

struct stat — структура режима пользователя, работали с ней в лабе по директориям.

В этой структуре есть 2 поля: поле inode и поле size.

То есть в user можем получить номер inode и размер файла.

Информация из этой структуры выводится: stat, statl, statf.

```
struct stat {
    unsigned int    st_dev;
    unsigned int    st_ino;
    unsigned int    st_mode;
    unsigned int    st_nlink;
    unsigned int    st_uid;
    unsigned int    st_gid;
    unsigned int    st_rdev;
    long            st_size;
    unsigned long    st_atime;
    unsigned long    st_mtime;
    unsigned long    st_ctime;
    unsigned int     st_blksize;
    unsigned int     st_blocks;
    unsigned int     st_flags;
    unsigned int     st_gen;
};
```

Существует три причины записи информации из буфера в файл:

1. Буфер заполнен и следующая операция приведет к записи данных из буфера в файл: overflow.
2. Когда вызывается функция fflush — принудительная запись из буфера в файл.

3. Когда вызывается `fclose()`.

Устранение гонок — средства взаимoisключения.

Код VFS

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/init_task.h>
#include <linux/fs.h>
#include <linux/slab.h>
#include <linux/time.h>
#include <linux/statfs.h>

# define MY_MAGIC_NUMBER 0x13131313

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Lebedev Vladimir");

static void myfs_put_super(struct super_block * sb)
{
    printk(KERN_DEBUG "+: myfs super block destroyed") ;
}

static struct dentry *my_simple_lookup(struct inode *dir, struct dentry *dentry, unsigned int flags)
{
    if (dentry->d_name.len > NAME_MAX)
        return ERR_PTR(-ENAMETOOLONG);
    if (!dentry->d_sb->s_d_op)
        d_set_d_op(dentry, &simple_dentry_operations);
    d_add(dentry, NULL);
    return NULL;
}
```

```

static int my_simple_statfs(struct dentry *dentry, struct kstatfs *buf)
{
    u64 id = huge_encode_dev(dentry->d_sb->s_dev);

    buf->f_fsid = u64_to_fsid(id);
    buf->f_type = dentry->d_sb->s_magic;
    buf->f_bsize = PAGE_SIZE;
    buf->f_namelen = NAME_MAX;
    return 0;
}

static struct super_operations const myfs_super_ops =
{
    .put_super = myfs_put_super,
    .statfs = my_simple_statfs,
    .drop_inode = generic_delete_inode
};

static struct file_operations my_simple_dir_operations = {
    .open      = dcache_dir_open,
    .release   = dcache_dir_close,
    .llseek    = dcache_dir_llseek,
    .read      = generic_read_dir,
    .iterate_shared = dcache_readdir,
    .fsync     = noop_fsync,
};

static struct inode_operations my_simple_dir_inode_operations
= {
    .lookup    = my_simple_lookup,
};

static int my_simple_fill_super(struct super_block *s, void *
data, int silent)
{

```

```

    printk(KERN_DEBUG "+: my_simple_fill_super") ;
    struct inode *inode;
    struct dentry *root;
    struct dentry *dentry;
    unsigned long magic = MY_MAGIC_NUMBER;

    s->s_blocksize = PAGE_SIZE;
    s->s_blocksize_bits = PAGE_SHIFT;
    s->s_magic = magic;
    s->s_op = &myfs_super_ops;
    s->s_time_gran = 1;

    inode = new_inode(s);
    if (!inode)
        return -ENOMEM;
    /*
     * because the root inode is 1, the files array must not
    contain an
     * entry at index 1
     */
    inode->i_ino = 1;
    inode->i_mode = S_IFDIR | 0755;
    inode->i_atime = inode->i_mtime = inode->i_ctime = curren
t_time(inode);
    inode->i_op = &my_simple_dir_inode_operations;
    inode->i_fop = &my_simple_dir_operations;
    set_nlink(inode, 2);
    root = d_make_root(inode);
    if (!root)
        return -ENOMEM;
    s->s_root = root;
    return 0;
}

static struct dentry *myfs_mount(struct file_system_type *typ
e, int flags, char const *dev, void *data)

```

```

{
    struct dentry* const root_dentry = mount_nodev(type, flag
s, data, my_simple_fill_super);
    if (IS_ERR(root_dentry))
        printk(KERN_ERR "+: myfs mounting failed \n");
    else
        printk(KERN_DEBUG "+: myfs mounted\n");
    return root_dentry;
}

void my_kill_anon_super(struct super_block *sb)
{
    dev_t dev = sb->s_dev;
    generic_shutdown_super(sb);
    free_anon_bdev(dev);
}

static void my_kill_block_super(struct super_block *sb)
{
    printk(KERN_DEBUG "+: my_kill sb");
    my_kill_anon_super(sb);
}

static struct file_system_type myfs_type =
{
    .owner = THIS_MODULE,
    .name = "my_fs",
    .mount = myfs_mount,
    .kill_sb = my_kill_block_super,
};

int number = 10;
static struct kmem_cache *mycache = NULL;
static void* *line = NULL;
static int __init my_init(void)
{

```

```

    if(register_filesystem(& myfs_type))
    {
        printk(KERN_ERR "+: register filesystem err");
        return -1;
    }

    if (!(line = kmalloc(sizeof(void *) * number, GFP_KERNEL)))
    {
        kfree(line);
        printk(KERN_ERR "+: allocate memory for SLAB cache error");
        return -1;
    }
    for(int i = 0; i < number; i++ )
        line[ i ] = NULL;

    if (!(mycache = kmem_cache_create("my_slab", sizeof(struct tree_descr), 0, SLAB_HWCACHE_ALIGN, NULL))
    {
        printk(KERN_ERR "+: kmem_cache_create err");
        kmem_cache_destroy(mycache);
        kfree(line);
        return -1;
    }
    for (int i = 0; i < number; i++)
        if (!(line[i] = kmem_cache_alloc(mycache, GFP_KERNEL)))
        {
            printk(KERN_ERR "+: kmem_cache_alloc err");
            for(int i = 0; i < number; i++ )
                kmem_cache_free( mycache, line[ i ] );
            kmem_cache_destroy(mycache);
            kfree(line);
            return -1;
        }

```

```

        printk(KERN_DEBUG "+: init");
        return 0;
    }

static void __exit my_exit(void)
{
    int ret = unregister_filesystem(&myfs_type);
    printk(KERN_DEBUG "+: exit");
    if (ret != 0)
        printk(KERN_ERR "+: unregister filesystem err");
    for(int i = 0; i < number; i++ )
        kmem_cache_free( mycache, line[ i ] );
    kmem_cache_destroy(mycache);
    kfree(line);
}

module_init(my_init);
module_exit(my_exit);

```

Код тасклеты

```

#include <linux/interrupt.h>
#include <linux/module.h>
#include <linux/slab.h>
#include <linux/init.h>
#include <linux/sched.h>
#include <linux/time.h>
#include <asm/io.h>
#define IRQ_NUM 1

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Lebedev Vladimir");

char * upper_codes[84] = { " ", "Esc", "1", "2", "3", "4",
    "5", "6", "7", "8", "9", "0", "-", "+", "Backspace",

```

```

        "Tab", "Q", "W", "E", "R", "T", "Y",
        "U", "I", "O", "P", "[", "]", "Enter", "Ctrl",
        "A", "S", "D", "F", "G", "H", "J", "K",
        "L", ";", "\"", "'", "Shift (left)", "|",
        "Z", "X", "C", "V", "B", "N", "M", "<",
        ">", "?", "Shift (right)",
        "*", "Alt", "Space", "CapsLock",
        "F1", "F2", "F3", "F4", "F5", "F6", "F
7", "F8", "F9", "F10",
        "NumLock", "ScrollLock", "Home", "Up",
        "Page-Up", "-", "Left",
        " ", "Right", "+", "End", "Down", "Page
-Down", "Insert", "Delete"};

```

```

char * lower_codes[84] = { " ", "Esc", "1", "2", "3", "4",
        "5", "6", "7", "8", "9", "0", "-", "+", "Backspace",
        "Tab", "q", "w", "e", "r", "t", "y",
        "u", "i", "o", "p", "[", "]", "Enter", "Ctrl",
        "a", "s", "d", "f", "g", "h", "j", "k",
        "l", ";", "\"", "'", "Shift (left)", "|",
        "z", "x", "c", "v", "b", "n", "m", "<",
        ">", "?", "Shift (right)",
        "*", "Alt", "Space", "CapsLock",
        "F1", "F2", "F3", "F4", "F5", "F6", "F
7", "F8", "F9", "F10",
        "NumLock", "ScrollLock", "Home", "Up",
        "Page-Up", "-", "Left",
        " ", "Right", "+", "End", "Down", "Page
-Down", "Insert", "Delete"};

```

```

int flag = false;
int prev_code = 0;
struct tasklet_struct* interrupt_1;

```

```

void my_tasklet_function(unsigned long data)

```

```

{
    printk( ">> interrupt_1: Bottom-half handled time=%llu\n", ktime_get());
    int code = interrupt_1->data;
    if (interrupt_1->data < 84)
    {
        if ((flag && (prev_code == 42)) || (!flag && (prev_code != 42)))
        {
            printk(">> interrupt_1: Key is %s\n", lower_codes[code]);
        }
        else
        {
            printk(">> interrupt_1: Key is %s\n", upper_codes[code]);
        }
        prev_code = code;
    }
    printk(">> interrupt_1: -----\n");
}

irqreturn_t my_handler(int irq, void *dev)
{
    printk(">> interrupt_1: Top-half start time=%llu\n", ktime_get());
    int code;
    if (irq == IRQ_NUM)
    {
        code = inb(0x60);

        if (code == 58)
        {
            flag = !flag;
        }
    }
}

```

```

        interrupt_1->data = code;
        printk(">> interrupt_1: Key code is %d\n", code);
        tasklet_schedule(interrupt_1);
        printk(">> interrupt_1: Bottom-half sheduled time=%llu\n", ktime_get());
        return IRQ_HANDLED;
    }
    printk(">> interrupt_1: irq wasn't handled\n");
    return IRQ_NONE;
}

static int __init my_init(void)
{
    if (request_irq(IRQ_NUM, my_handler, IRQF_SHARED, "interrupt_1", &my_handler))
    {
        printk(">> interrupt_1: ERROR request_irq\n");
        return -1;
    }
    interrupt_1 = kmalloc(sizeof(struct tasklet_struct), GFP_KERNEL);
    if (!interrupt_1)
    {
        printk(">> interrupt_1: ERROR kmalloc!\n");
        return -1;
    }
    tasklet_init(interrupt_1, my_tasklet_function, 0);

    printk(">> interrupt_1: init\n");
    return 0;
}

static void __exit my_exit(void)
{
    tasklet_kill(interrupt_1);
}

```

```

    kfree(interrupt_1);
    free_irq(IRQ_NUM, &my_handler);
    printk(">> interrupt_1: exit\n");
}

module_init(my_init)
module_exit(my_exit)

```

Код очереди работ

```

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/interrupt.h>
#include <linux/slab.h>
#include <asm/io.h>
#include <linux/stddef.h>
#include <linux/workqueue.h>
#include <linux/delay.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Lebedev Vladimir");

char * upper_codes[84] = { " ", "Esc", "1", "2", "3", "4",
    "5", "6", "7", "8", "9", "0", "-", "+", "Backspace",
    "Tab", "Q", "W", "E", "R", "T", "Y",
    "U", "I", "O", "P", "[", "]", "Enter", "Ctrl",
    "A", "S", "D", "F", "G", "H", "J", "K",
    "L", ";", "\"", "'", "Shift (left)", "|",
    "Z", "X", "C", "V", "B", "N", "M", "<",
    ">", "?", "Shift (right)",
    "*", "Alt", "Space", "CapsLock",
    "F1", "F2", "F3", "F4", "F5", "F6", "F
7", "F8", "F9", "F10",
    "NumLock", "ScrollLock", "Home", "Up",
    "Page-Up", "-", "Left",

```

```

        " ", "Right", "+", "End", "Down", "Page
-Down", "Insert", "Delete"};

char * lower_codes[84] = { " ", "Esc", "1", "2", "3", "4",
"5", "6", "7", "8", "9", "0", "-", "+", "Backspace",
        "Tab", "q", "w", "e", "r", "t", "y",
"u", "i", "o", "p", "[", "]", "Enter", "Ctrl",
        "a", "s", "d", "f", "g", "h", "j", "k",
"l", ";", "\"", "'", "Shift (left)", "|",
        "z", "x", "c", "v", "b", "n", "m", "<",
">", "?", "Shift (right)",
        "*", "Alt", "Space", "CapsLock",
        "F1", "F2", "F3", "F4", "F5", "F6", "F
7", "F8", "F9", "F10",
        "NumLock", "ScrollLock", "Home", "Up",
"Page-Up", "-", "Left",
        " ", "Right", "+", "End", "Down", "Page
-Down", "Insert", "Delete"};

int flag = false;

typedef struct
{
    struct work_struct work;
    int code;
} my_work_struct_t;

static struct workqueue_struct *my_wq;

static my_work_struct_t *work1;
static my_work_struct_t *work2;

int keyboard_irq = 1;
int flag1 = false;
int flag2 = false;
int prev_code1 = 0;

```

```

int prev_code2 = 0;

void work1_func(struct work_struct *work)
{
    printk(">> interrupt_2: -----
---\n");
    my_work_struct_t *my_work = (my_work_struct_t *)work;
    int code = my_work->code;

    printk(KERN_INFO "interrupt_2: work1 begin");

    if (!flag1)
    {
        printk(">> interrupt_2: Key is %s\n", lower_codes[code]);
    }
    else
    {
        printk(">> interrupt_2: Key is %s\n", upper_codes[code]);
    }
    prev_code1 = code;

    printk(KERN_INFO "interrupt_2: work1 end");
    printk(">> interrupt_2: -----
---\n");
}

void work2_func(struct work_struct *work)
{
    printk(">> interrupt_2: -----
---\n");
    my_work_struct_t *my_work = (my_work_struct_t *)work;
    printk(KERN_INFO "interrupt_2: work2 sleep begin");
    msleep(10);
}

```

```

    printk(KERN_INFO "interrupt_2: work2 sleep end");
    int code = my_work->code;
    if (!flag2)
    {
        printk(">> interrupt_2: Key is %s\n", lower_codes[code]);
    }
    else
    {
        printk(">> interrupt_2: Key is %s\n", upper_codes[code]);
    }
    prev_code2 = code;
    printk(">> interrupt_2: -----
    ---\n");
}

irqreturn_t my_irq_handler(int irq, void *dev)
{
    printk(">> interrupt_2: -----
    ---\n");
    int code;
    printk(KERN_INFO "interrupt_2: irq handler");

    if (irq == keyboard_irq)
    {
        code = inb(0x60);

        if (code >= 128)
        {
            printk(KERN_INFO "interrupt_2: called by keyboard
            _irq");

            work1->code = code - 128;
            work2->code = code - 128;
            if (work1->code == 58)
            {

```

```

        flag1 = !flag;
        flag2 = !flag;
    }
    printk(KERN_INFO "interrupt_2: key code is %d", work1->code);

    queue_work(my_wq, (struct work_struct *)work1);
    queue_work(my_wq, (struct work_struct *)work2);
}

return IRQ_HANDLED;
}

printk(KERN_INFO "interrupt_2: called not by keyboard_irq");
printk(">> interrupt_2: -----
---\n");

return IRQ_NONE;
}

static int __init my_workqueue_init(void)
{
    int ret;

    ret = request_irq(keyboard_irq, my_irq_handler, IRQF_SHARED,

                      0, (void *) my_irq_handler);

    if (ret)
    {
        printk(KERN_ERR "interrupt: request_irq error");
        return ret;
    }
}

```

```

else
{
    my_wq = alloc_workqueue("%s", __WQ_LEGACY | WQ_MEM_RE
CLAIM, 1, "interrupt_2");

    if (my_wq == NULL)
    {
        printk(KERN_ERR "interrupt_2: create queue erro
r");

        ret = GFP_NOIO;
        return ret;
    }

    work1 = kmalloc(sizeof(my_work_struct_t), GFP_KERNEL);
    if (work1 == NULL)
    {
        printk(KERN_ERR "interrupt_2: work1 alloc erro
r");

        destroy_workqueue(my_wq);
        ret = GFP_NOIO;
        return ret;
    }

    work2 = kmalloc(sizeof(my_work_struct_t), GFP_KERNEL);
    if (work2 == NULL)
    {
        printk(KERN_ERR "interrupt_2: work2 alloc erro
r");

        destroy_workqueue(my_wq);
        kfree(work1);
        ret = GFP_NOIO;
        return ret;
    }
}

```

```

        INIT_WORK((struct work_struct *)work1, work1_func);
        INIT_WORK((struct work_struct *)work2, work2_func);
        printk(KERN_ERR "interrupt_2: init");
    }
    return ret;
}

static void __exit my_workqueue_exit(void)
{
    synchronize_irq(keyboard_irq); // ожидание завершения обр
аботчика
    free_irq(keyboard_irq, my_irq_handler); // освобождение л
инии от обработчика
    flush_workqueue(my_wq);
    destroy_workqueue(my_wq);
    kfree(work1);
    kfree(work2);

    printk(KERN_INFO "interrupt_2: exit");
}

module_init(my_workqueue_init);
module_exit(my_workqueue_exit);

```